Thrashing Returns: Metastable Failures in Prefix-Cached Agent Serving

Akshat Valse* Ali Jannesari
Department of Computer Science, Iowa State University, Ames, IA, USA
avalse@iastate.edu jannesar@iastate.edu

Abstract

Prefix caching keeps an LLM agent session's KV across tool calls, so a hit prefills only new tokens. This can make the high-hit state metastable, in the physics sense. In a pre-registered paired test on SGLang 0.5.22 (Qwen2.5-7B-Instruct, one A100, 300k-token pool) at 1.35 and 1.65 times the measured all-miss capacity, a burst of 200 sessions left 6 of 6 runs at zero hits to the end, 7500 s later (5 trapped by the registered rule); without it 6 of 6 stayed good, and a client-side cap of 20 drained it in 6 of 6. The test also passed with SGLang's own 955,757-token pool and, registered, on vLLM 0.32.0 (a frozen mechanism check failed there); on an H200 the all-miss capacity rose 2.7-fold, the peak only 1.6-fold. In registered rescue tests that cap (64 with SGLang's pool), engaged in an established trap, rescued 9 of 9 runs with no session shed; released after the drain, every run stayed good at the same load; uncapped controls stayed at zero hits. A registered test put the basin boundary between bursts of 25 and 100 sessions. We derive closed-population throughput $\mu(n)$ from first principles, with misses swelling running requests' KV and nothing fitted to any $\mu(n)$, hit or trap measurement (another GPU or engine takes one speed factor): its all-miss onset is the sessions in tool calls plus one pool of running sessions, a uniformly faster GPU lowers its peak ratio, and exogenous-gap models without running KV cannot reach zero hits. In sample it reproduces the SGLang curves within 10% at the peak but misses vLLM's earlier cliff; its frozen predictions held for the rescue, the basin test and the H200's nominal from-empty edge (the last two on runs begun before freezing), not for the H200 curve's tail or 4 of 9 release times; others are pending. Rates are nominal; registered tests use one model, one GPU type and synthetic sessions (exponential tools, no shared prefixes).

1 Introduction

An LLM agent that uses tools sends its serving engine sessions of many turns. Each turn's prompt is the previous context plus the output of the tool just run, and between turns the session is idle while the tool runs. Prefix caching keeps the key–value (KV) state of the previous turn in GPU memory, so a follow-up turn whose prefix is still cached prefills only its new tokens [104]. In our workload a follow-up turn that hits prefills 551 tokens, while a miss prefills the whole prompt, about 10,500 tokens on average; over a session, prefill work with every follow-up hitting is about 16 times smaller than with every turn missing.

We show that this can make the engine's good state, in which almost every follow-up turn hits, *metastable* in the physics sense (it persists until a large enough perturbation; in the terms of Bronson et al. [10] it is a vulnerable state, and the all-miss state it falls into is the metastable failure state), and that this bad state can be undone at the same load. Let λ_{miss} be the all-miss capacity, the session throughput of a saturated engine when every turn misses, and λ_{ref} its estimate from saturated runs with caching disabled (0.0715 sessions/s on SGLang with a 300,000-token KV pool). In our measurements an empty server carried nominal arrival rates up to 2 times its own λ_{ref} for 3000 s with 91–93% of turns hitting, on SGLang and on vLLM with that pool, and up to 5.5 times λ_{ref} when SGLang sized its own KV pool (nominal rates on seed-0 grids, mostly one run per rate; §5.1). On SGLang at 300k, three fresh arrival seeds also carried 2.5 λ_{ref}, where the original seed did not, and ordered by realized arrival rate the from-empty edge there lies between 2.67 and 2.73 λ_{ref}. A transient burst admits more sessions than the KV pool can hold once their contexts grow. A session with no running request, whether in a tool call or waiting in the queue, holds no lock on its KV, so admitting waiting work evicts it, oldest first; when its next turn is admitted it misses, the miss costs an order of magnitude more prefill, the queue grows, the gap between a session's turns grows with it, and more idle KV is evicted before its owner returns. Every turn then pays a full prefill, and the backlog does not drain (Fig. 1).

Neither the collapse nor the loop that sustains it is new. Throughput that collapses when the working sets of the admitted jobs exceed memory is Denning's thrashing [26, 27], and a bad state that a feedback loop sustains after its trigger is gone is a metastable failure [10, 46]. Agent-serving systems evict, offload or retain the KV that sessions leave idle during tool calls [1, 8, 9, 58, 94], treat its loss as working-set thrashing [31, 53, 83], and describe, measure or bound the loop between eviction, cold prefill and queueing for closed or bounded populations of agents [3, 18, 45, 55, 102], where lowering the number of active agents also lowers the load. Under open arrivals on vLLM, KAIROS describes the same loop and asserts, in its design section and without a controlled test, that even a short overload can push an instance into a persistently inefficient operating region; the persistent thrashing regime

*Corresponding author.

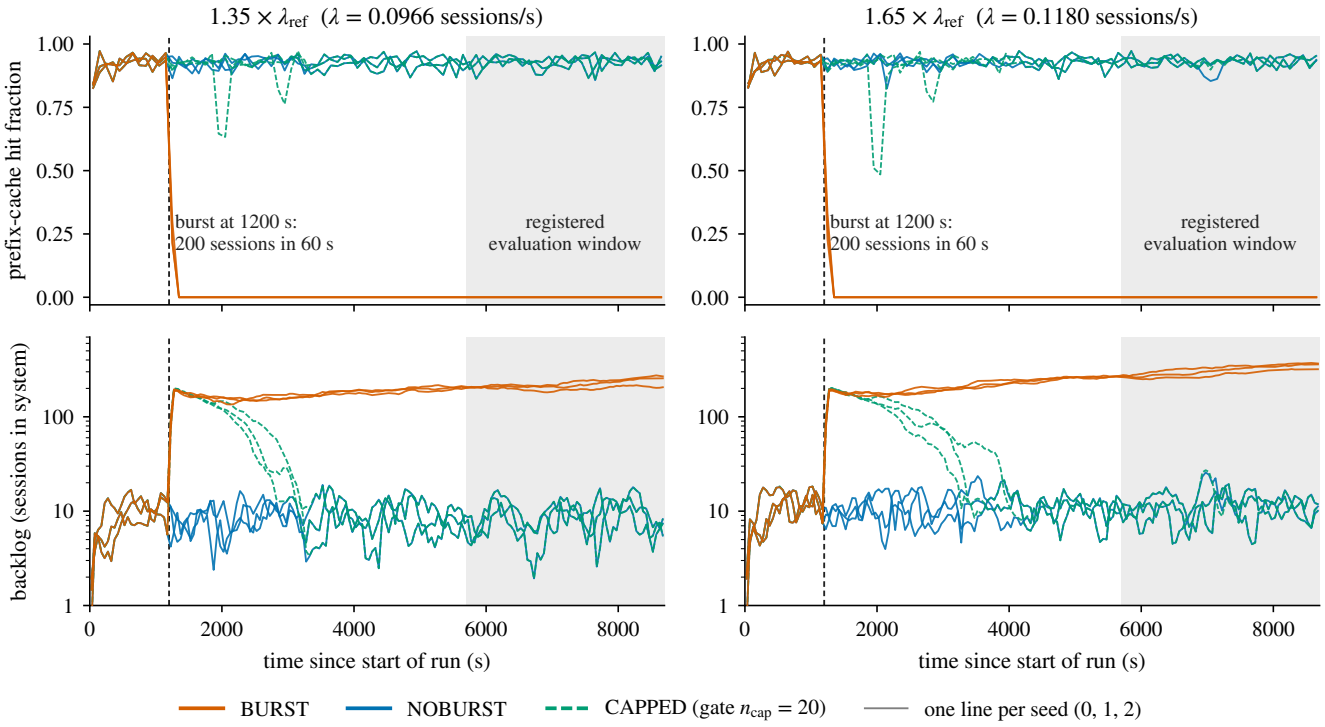


Figure 1: The registered paired test on SGLang (E2) at nominal rates of 1.35 λ_{ref} (left) and 1.65 λ_{ref} (right), λ_{ref} = 0.0715 sessions/s; over the evaluation window the three seeds delivered 1.27–1.45 and 1.61–1.75 λ_{ref}. Top: hit fraction (turns with more than half of the prompt cached) in 100 s windows. Bottom: sessions in the system (log scale), including those in tool calls. One thin line per seed and arm. Dashed: a burst of 200 sessions at 1200 s. Shaded: the registered evaluation window. The hit series is recomputed from the request logs and matches the registered per-run value to four decimals.

it shows is one run without its thrashing avoidance, against capped and no-control runs on the same replayed workload, with no burst trigger or matched no-burst arm, no fixed clock and no replication [101]. Section 9 gives the full positioning.

Results. We report four results; the paragraph after them says what is ours and what is not.

(1) **Registered open-system metastability on real engines** (§5.2, Fig. 1, Table 1; §5.5). At a fixed clock and pool, at Poisson arrival rates that the engine carried in the good state for the full 8700 s of every no-burst run, a burst of 200 sessions within 60 s left it at zero hits, with a backlog that did not drain, to the end of the run, 7500 s later, while a client-side cap at the peak of $\mu(n)$ drained the same burst. In two pre-registered SGLang tests (pools of 300,000 and 955,757 tokens) the burst arm (BURST) was at zero hits in 12 of 12 runs and trapped by the registered rule in 11, the no-burst arm (NOBURST) was good in 12 of 12 and the capped arm (CAPPED) recovered in 12 of 12. On vLLM 0.32.0 the registered analysis of a third test passes at both rates, but the frozen decision rule for its mechanism checks is not met, because its online detector also alarmed, during the drain, in two capped runs. On an H200 at the same pool the all-miss capacity rose 2.71-fold and the peak of $\mu(n)$ only 1.63-fold, so the window of rates in which the good state is metastable, whose upper end is the from-empty edge, narrowed from (1, 2.0) to (1, 1.3) times each GPU's λ_{ref} (nominal multiples on seed-0 grids), too narrow for our registration rule to place a test.

(2) **Rescue from an established trap, with persistence after release** (§5.3, Fig. 3, Table 2). In two registered tests the same cap, engaged 600 or 900 s after the burst inside an established trap, rescued 9 of 9 runs without shedding a session: hits began to return within three minutes, the backlog drained, and after the cap was removed every run stayed good at the same arrival process to the end of the run, matching its no-burst control, while the same-seed uncapped run stayed at zero hits. Which state the engine is in at a given load thus depends on its history, and a transient intervention moves it back to the good state, which then sustains itself. The cap was removed once the client queues had been empty for 300 s, so the test covers a drained state left without control, not a release while the cap binds. The price is waiting at the client, 386–569 s on average per paused session; counting all waiting, the rescued runs waited 4.4 to 5.1 times less than their uncapped controls (a lower bound, since those controls still had unfinished sessions at 8700 s). In client delay (pauses and admission waits), applying the cap from the

start costs about as much at the 300k pool and about 11 times less with the native pool.

(3) **A first-principles thrashing curve** (§6, Fig. 4, Table 3; §7, Table 5). Misses starve decode under prefill priority, so running requests reserve more of the pool and squeeze the idle KV that would give hits; this coupling produces the cliff, while the gap between a session's turns is still the tool time. Once every turn misses, the queue, not the tool time, sets that gap, which sustains the all-miss state (Proposition 5). We derive (with LLM coding agents, §6) $\mu(n)$ from the pool size, the workload and an iteration-cost model fitted only to server-log timing, with no parameter fitted to any $\mu(n)$, hit or trap measurement (the H200 and vLLM cells take one speed factor fitted to their all-miss token rate). For the deterministic model we prove a closed-form all-miss onset, the sessions in tool calls plus one pool of running sessions (Proposition 2), with the corollary that a uniformly faster GPU moves the onset out (Proposition 2(c)), a speed law under which such a GPU lowers the peak ratio $\mu^*/\lambda_{\text{miss}}$ (Proposition 3), and a bound showing that exogenous-gap models whose running requests hold little of the pool cannot reach the measured zero hits, which need running requests to hold almost the whole pool (Proposition 4). In sample, the model reproduces the SGLang curves within 10% at the peak and misses vLLM's earlier cliff. Of its predictions frozen before the results were read by the coding agents that derived it (the first author had seen the H200 values for $n \leq 32$), those for the rescue verdicts, the basin test BASIN-1, the H200's from-empty edge (on the nominal grid; in realized rates F2 was somewhat pessimistic, as on the A100) and its 0.5-hit crossing held, and those for the H200 curve's tail past the peak and 4 of the 9 rescue release times did not; its critical-burst prediction (BASIN-2) is pending. Section 6.4 gives the measured gap in both states, and §7 what each prediction is blind to.

(4) **A measured basin** (§5.4). In the registered basin test BASIN-1 at 1.65 λ_{ref} on the 300k pool, a burst of 25 sessions left the engine good and one of 100 trapped it, in 3 of 3 seeds each, so the boundary of the good state's basin lies between 25 and 100 sessions, as the simulator predicted before the runs were submitted (the model's predictions for it are blind by access only: its runs had started, and the 100-session runs had ended, before those predictions were pushed, and were read only after); BASIN-2, with bursts of 40, 50 and 60 sessions around the model's critical burst of 53.4, is pending in this version.

What is ours. The loop was measured for closed populations; under open arrivals KAIROS describes it, asserts without a controlled test that a short overload can leave an instance persistently inefficient, and shows one such run without its thrashing avoidance and with no burst trigger [101]. Ours is a controlled, replicated, registered test of that assertion at a fixed clock and pool with the trigger-and-control design of Huang et al. [46]: two persistent states at one capacity and offered load, against matched no-burst and capped arms on common random numbers, on two pools and two engines. Session caps, pausing at tool boundaries and hit-rate-triggered contraction are published as prevention or, in closed populations, as a way out of thrashing that lowers the load itself [18, 31, 45, 53]; KAIROS defers new agents once bursty arrivals break its bound but reports no recovery from an established trap [101]; and Carleial and Hellman proposed blocking new traffic once a bistable channel reaches a dangerously high backlog and resuming when it falls to a specified lower one, without evaluating it [14]. Ours is the measured rescue of an established open-system trap at the same offered load, with persistence after release, and with its condition (Corollary 1) and release times frozen beforehand. Metastability from a unimodal throughput curve and its cure by bounded admission are classical [14, 23, 56, 72], with the slowdown given as a function of occupancy, and mean-field models of multi-turn prefix caches take the gaps between turns as given [97]; ours are the derivation of the curve with running reservations coupled to the hit rate and, on the all-miss branch, the gap coupled to the queue, its closed forms, and their out-of-sample record. That the trap probability depends on the size of the trigger is part of metastability theory [10, 46, 48]; ours is a measured boundary on an LLM engine, predicted beforehand by the simulator.

Further results. Under request-level FCFS and stated assumptions, no online eviction rule, queued-prefix protection and FCFS-order TTL pins included, lets $\mu(n)$ exceed the all-miss capacity as $n \to \infty$ if the congested engine computes prompt tokens no faster than with caching disabled (Proposition 6, stated and proved in Appendix E). The prefill-saturation bound is MISA-T's [45]; ours is that request-level FCFS drives reuse to zero in congestion whatever the online eviction rule. Stock SGLang LPM falls under it in deep congestion through its 128-request fallback, and with the fallback disabled it avoided the all-miss collapse in an exploratory test, without recovering (§5.6). We also evaluate a pre-deployment procedure (§8): measure λ_{miss} with the cache off, $\mu(n)$ across the cliff and the from-empty edge, cap admitted sessions at the smallest n within 5% of the peak, inside the interval (n_1, n_2) of Corollary 1, and re-measure for each workload, pool, engine and GPU; its parts are prior art [18, 20, 29, 45, 68], and ours are the evaluation and the placement condition. Appendix I varies the tool-time law, the burst size and the horizon in a calibrated simulator and compares published mechanisms in an earlier one.

Registrations committed before their runs, with GitHub's push log of our private development repository, exported in `provenance/`, as the record of when each was pushed, and predictions frozen under pushed tags are how we tried to make these claims checkable. They are rigor, not a contribution; Appendices B and C list every registration and prediction round, with the slips in them.

The registered tests use one model, Qwen2.5-7B-Instruct, one GPU type, the A100 80GB, and synthetic sessions with exponential 5 s tool times and no shared prefixes. The H200 cell is a measurement, not a registered test, and the Llama-

3.1-8B cell of §7, whose outcome is pending, is a second model. The vLLM registration was committed before its jobs were submitted but reached GitHub only after the first two had started, so we call that test registered, not pre-registered, and its BURST arm reuses the seeds of earlier exploratory vLLM runs (§5.2).

2 Background and what is classical

Engines. LLM engines batch per model iteration: running requests decode one token per iteration and waiting requests join between iterations [99]; KV is stored in pages [57]. SGLang keeps cached prefixes in a radix tree, evicts least-recently-used (LRU) leaves, and can order waiting requests by longest prefix match (LPM) [104]. In SGLang 0.5.22 the default policy is FCFS and the default eviction rule LRU; LPM stops sorting its queue when more than 128 requests wait, a fallback that PEEK also documents [95]; a waiting request does not lock its cached prefix, which is locked when the request is admitted; and under FCFS, including LPM above that threshold, an LRU refresh touches the prefixes of the first 128 waiting requests once every 32 scheduling passes. vLLM 0.32.0 schedules FCFS by default, and a freed cached block keeps its hash until it is reallocated. Appendix E cites the source lines at the release tags. Our synthesis of these rules is that in both engines a session's KV is protected only while one of its requests runs.

Thrashing and metastability. Denning showed that admitting one more program to a memory near saturation can collapse efficiency, and that keeping the working sets of the active programs within memory, which limits their number, prevents it [26, 27]. We call throughput as a function of the closed population n the thrashing curve; the term is our framing. LRU does badly on loops larger than the cache [21, 84]. A metastable failure is a bad state that persists after its trigger is removed, sustained by a feedback loop under uncontrolled load; one of Bronson et al.'s case studies is a lost look-aside cache [10], and Huang et al. study such failures in production incidents and model them [46]. In their classification of sustaining effects, a trap in which the load itself lowers capacity is capacity-degradation amplification; one of their cases is garbage collection driven by queueing-induced memory pressure, and their experiments vary the trigger's magnitude against matched runs, the design we follow. In Bronson et al.'s terms, a system that serves its load but can be pushed by a trigger into the bad state is in a vulnerable state, and the bad state, kept by a sustaining effect after the trigger ends, is the metastable failure state. We also call the good state metastable, in the physics sense of a state that persists until a large enough perturbation. Denning measured the working-set window in a program's own virtual time, so that interruptions would not distort it [28]; an LRU pool over idle KV ages its entries in real time, so the time a session waits in the queue counts against its residency.

What is classical, and what is specific to agent serving. The collapse past the working-set level, a bad state that outlives its trigger, metastability from a unimodal throughput curve under open load and bistability when the load comes from a finite source, hysteresis, and bounded admission as the cure are classical, for random-access channels, thrashing multiprogramming systems and service systems whose customers return [14, 15, 23, 56, 72]; in all of them the slowdown is a given function of occupancy. So are the decomposition approximation behind §3.3 [23, 56] and multiprogramming-level load control [29, 43, 82]. Two features of agent serving make them apply: the reusable KV is exactly the idle KV, which memory-gated admission reclaims, and a hit is much cheaper than a miss. Appendix G.9 gives the full background.

3 A fluid model of the open system

3.1 Workload and notation

A session has $T \sim \min(\text{Geometric}(1/15), 60)$ turns, so $E[T] = 14.761$. Turn k sends a prompt of $L_k = L_0 + (k-1)(\Delta + d)$ tokens: a first prompt of $L_0 = 2000$ tokens, $\Delta = 550$ tokens of tool output and $d = 100$ decoded tokens per turn. Between turns the session waits for a tool for a time Z, independent across turns, exponential with mean 5 s in the workload of record. A turn *hits* if more than half of its prompt is served from cache; on the real engine a hit almost always reuses everything but the 550 new tokens and the last decoded token, so it prefills 551 tokens (all but 95 of the 185,613 hits in the registered paired test E2 of §5.2). First turns always miss, so in the long run the hit fraction is at most $1 - 1/E[T] = 0.932$; finite windows can lie slightly above it (up to 0.94 in our runs). With every turn missing, a session's prefill work is $W(T) = 2000\,T + 325\,T(T-1)$ tokens with $E[W] = 154{,}529$; with every follow-up hitting it is $2000 + 551(T-1)$, about 9,600 in expectation.

The engine has a KV pool of M tokens. In the *closed* system n sessions are present and a finished session is replaced at once. The thrashing curve $\mu(n)$ is the long-run session completion rate (sessions/s); $\mu^* = \max_n \mu(n)$ is attained at n^*, and $\mu_\infty = \lim_{n\to\infty} \mu(n)$ is the *congested service rate*, a long-run rate of the closed system (an open system in congestion can complete sessions faster than μ_∞ for a long time, §3.3). The all-miss capacity λ_{miss} is the throughput of the saturated engine with prefix caching disabled. In the *open* system sessions arrive as a Poisson process of rate λ, and $N(t)$ counts sessions in the system (waiting, running or in a tool call). A session rate is estimated either as sessions completed per second or, with much lower variance because session work is heavy-tailed in T, as turns per second divided by $E[T]$ ("turns-based").

3.2 The thrashing curve for agent sessions

Let E_{lb} be the time-averaged KV footprint of a session over its lifetime, a length-biased mean in which long sessions and late turns weigh more (during the tool call after turn k a session holds $L_k + d$ tokens). By the $H = \lambda G$ relation [12, 44], the

mean KV demand of n present sessions is nE_{lb}. For our workload E_{lb} is about 10.5k tokens when tool calls dominate the lifetime, so a 300k pool holds $N_{\text{lb}} = M/E_{\text{lb}} \approx 28.5$ sessions on average (weighting every turn equally, the last included, gives 10.6k and 28.4; the static caps 28 and 36 of the earlier-simulator comparison, Appendix I.2, are set from the latter, and the registered caps come from the rule of §4.3). While nE_{lb} is well below M, idle KV survives the tool call and $\mu(n)$ rises with n through batching. Beyond it, admitting a waiting turn evicts idle KV, which belongs to sessions in tool calls or already waiting in the queue; they miss when their next turn is admitted. The peak n^* lies below N_{lb} mainly because idle KV is evicted at a finite LRU age, so hits fall before mean demand reaches the pool, and to a lesser degree because demand fluctuates around its mean. The capacity ratio is the collapse predictor of AgentKV [3] and the basis of MISA-T's session cap [45]. Section 6 derives the curve from first principles; its all-miss onset at 300k, $n_b = 32.7$, is the sessions in tool calls plus one pool of running sessions (Proposition 2 of §6.2), so the measured peak, the capacity ratio and the onset are ordered $n^* = 24 < N_{\text{lb}} < n_b$.

3.3 The open system as a quasi-static approximation

Replacing the departure rate of the open system by its closed-system value at the current population, the classical decomposition approximation (§2), gives

$$\dot{n} = \lambda - \mu(n). \tag{1}$$

Extend μ from integer n to all $n \geq 0$ by linear interpolation, so that it is continuous and piecewise linear with $\mu(0) = 0$ and Eq. (1) has unique solutions, and assume that the stationary curve is *unimodal*: strictly increasing on $[0, n^*]$ to μ^*, non-increasing on $[n^*, \infty)$ with limit $\mu_\infty < \mu^*$, and strictly decreasing wherever $\mu > \mu_\infty$. This allows a tail on which $\mu = \mu_\infty$, as on the predicted curves of §6; the statements below are for $\lambda \neq \mu_\infty$. For $0 < \lambda < \mu^*$ let $n_1 < n^*$ solve $\mu(n_1) = \lambda$. If $\lambda < \mu_\infty$, n_1 is the only fixed point of Eq. (1), and it attracts every trajectory. If $\mu_\infty < \lambda < \mu^*$, there is one more, $n_2 > n^*$, which is unstable, and from any $n > n_2$, $n \to \infty$: the good state is metastable. If $\lambda > \mu^*$ there is no good state. A solution from a congested state $x_0 > n^*$ returns to n_1 if and only if $\mu > \lambda$ on $(n_1, x_0]$, the endpoint n_1 excluded because $\mu(n_1) = \lambda$; since $\mu > \lambda$ on $(n_1, n^*]$ and $\mu \geq \mu(x_0)$ on $[n^*, x_0]$, this holds if and only if $\lambda < \mu(x_0)$. As x_0 grows, $\mu(x_0)$ falls to μ_∞, so μ_∞ is the *recovery threshold* from deep congestion. After a finite burst x_0 is finite, so recovery above μ_∞ is possible; the threshold for a given burst depends on its size. Appendix E proves these statements and treats a curve without the monotone tail, such as our measured one.

In a finite system fluctuations can carry N past n_2 without a burst, and an experiment sees only escapes within its horizon. We therefore use a measured *from-empty edge* λ_{e}, the largest rate at which runs started from an empty engine stay stable over a stated horizon, and define the window

$$\mathcal{W} = \big(\mu_\infty,\ \min(\mu^*, \lambda_{\text{e}})\big). \tag{2}$$

Inside $\mathcal{W}$ the good state exists from empty, and a burst that crosses the basin boundary leaves the system in a regime whose service rate is below the arrival rate. That regime is transient (the backlog grows), not a second stationary state, so we say metastable rather than bistable; two stable states, the bistability of ALOHA-type channels [14] and the stochastic cusp of multiprogramming models [72], arise here only for the admitted population of a capped engine (c sessions at $\mu(c)$ while the gate grows; Corollary 1) or, when the closed curve folds (dispersed tool times, not our workload), in the closed system itself: Proposition 7 then gives a locally stable all-miss state and two queue-free states, and in our example one of these, the good state, is numerically stable (Fig. 5). Appendix E maps these quantities to Bronson et al.'s terms [10].

Eq. (1) assumes that the internal state relaxes quickly compared with changes in n, and it is not quantitative in congestion, where a trapped engine serves a young backlog of short turns; we use it for the window, as a long-run statement for the recovery threshold, and for the capped system's drain time, not for burst sizes or backlog growth (Appendix E).

3.4 Caps and a congestion bound

Without a cap, deep congestion is bounded by the all-miss capacity: under request-level FCFS and assumptions (A1)–(A3) of Appendix E, which include that the congested engine computes prompt tokens no faster than with caching disabled, no online eviction rule, queued-prefix protection and FCFS-order TTL pins included, lets $\mu(n)$ exceed λ_{miss} as $n \to \infty$, and the rate of hit turns tends to 0 (Proposition 6). It covers SGLang's stock LPM through its 128-request fallback, but not a pure resident-first order; Appendix E states and proves it and discusses its scope.

Proposition 1 (cap identity; elementary)**.** *Let a FIFO gate admit at most c sessions, each holding its slot from admission to the end of its last turn, tool calls included. In the closed system with n sessions the admitted sessions form a closed system of* $\min(n, c)$ *sessions, so* $\mu_{\text{cap}}(n) = \mu(\min(n, c))$.

Proof. When an admitted session finishes, the gate admits its head at once and the replacement joins the gate. □

Corollary 1 (cap rescue; quasi-static)**.** *Let μ be unimodal as in §3.3, let $\mu_\infty < \lambda < \mu^*$, and let $n_1 < n_2$ be the stable and the unstable fixed point of Eq.* (1)*. Under Eq.* (1) *with the departure rate μ_{cap} of Proposition 1, a FIFO cap c engaged when more than c sessions are present (the congested regime included) empties its gate if and only if $n_1 < c < n_2$, that is, $\mu(c) > \lambda$. Once the gate is empty the cap no longer binds, and removing it leaves the population in the basin $[0, n_2)$ of n_1, where it stays.*

The proof is in Appendix E. That the good state persists after the cap is removed is hysteresis of the system, which the rescue tests of §5.3 check on the engine; it differs from the dead band of a controller that pauses and resumes admission at two thresholds (Appendix I.2). A cap $c > n^*$ raises the congested level to $\mu(c)$ but leaves the window $(\mu(c), \mu^*)$ open. A burst

of b sessions that arrives when $n_1 < c$ sessions are present fills the $c - n_1$ free slots at once, so the gate empties after about $(n_1 + b - c)/(\mu(c) - \lambda)$; Appendix A compares this estimate, and the registrations' cruder $b/(\mu(c) - \lambda)$, with the engine. Appendix E treats a folding curve and the stochastic counterpart.

4 Methodology

4.1 Real-engine harness

All real-engine runs used one GPU per job on the Nova SLURM cluster at Iowa State University, an NVIDIA A100 80GB PCIe except in the H200 cell of §5.5 (one H200 NVL), with SGLang 0.5.22 (FlashInfer attention, or on the H200 FlashAttention 3, SGLang's default there; chunked prefill of 8192 tokens, at most 256 running requests, LRU eviction) serving Qwen2.5-7B-Instruct [80]. The KV pool was capped at 300,000 tokens or, in the native-pool runs, sized by SGLang at 955,757 tokens. Decoding was greedy, 100 tokens per turn (Appendix A lists every setting). The vLLM 0.32.0 runs used the same model, GPU, client and workload with a 300,000-token pool (§5.2).

An open-loop client on the same node sends each turn as a completion request whose prompt is a list of token ids: random content from a seeded generator, extended each turn by the token ids the server returned plus 550 new tool tokens, so a hit covers exactly the previous prompt and output; during a tool call the client sleeps. Cached tokens come from the server's usage report, which the engine's counters and log confirmed in the closed-population runs (Stage C, §4.2). Before every run the cache is flushed and checked with a canary: a 1024-token request that reused 1023 tokens before the flush (1008 on vLLM, which caches whole 16-token blocks) must reuse at most 15 after; it reused none. In the cache-off calibration runs (E0a, §4.2) the check only confirms that the engine is idle. Each arm of a registered test ran in its own job on a fresh server.

The BURST arm adds 200 fresh sessions within 60 s at $t_b = 1200$ s to the Poisson stream; NOBURST is the same stream without them; CAPPED is BURST plus a client-side FIFO gate admitting at most n_{cap} sessions, each holding its slot until its last turn ends (the gate of Proposition 1); the rescue tests of §5.3 engage such a gate only after the trap is established. The burst equals 12 to 38 minutes of base arrivals at the registered rates, and its 2000-token first prompts alone (400k tokens) exceed the 300k-token pool; we took its size from our earlier simulator, where bursts of 100 or more trapped every work-conserving policy, so the test asks whether a burst large enough to cross the basin boundary traps the engine, not how likely such a burst is (§5.4). The arms of one seed share arrival times, turn counts and tool times (common random numbers), so BURST differs from NOBURST by the burst and CAPPED from BURST by the gate, up to GPU timing nondeterminism: each arm ran on a fresh server in its own job, and the arms of one seed did not always share a node. Seeds 0, 1 and 2 are reused across rates, pools, policy cells, engines and tests: for a given seed the 200 burst sessions are identical everywhere, the arrival times at a lower rate are a subset of those at a higher rate, and the i-th session to arrive has the same turn count, tool times and client-generated tokens everywhere, so runs that share a seed are not independent across these.

4.2 Staged protocol

Each stage's inputs came from the previous one (job IDs in Table 6; Appendix A gives the sequence and the details). *Stage C* measured $\mu(n)$ in closed populations, with all n sessions starting at $t = 0$, one run per n and an 1800 s window after 900 s of warmup. *E0a* measured λ_{miss} with the radix cache disabled; its turns-based value λ_{ref} (0.0715 sessions/s at 300k, 0.0718 with the native pool, 0.0657 on vLLM) is the unit of every arrival rate on that engine and pool, and numeric multiples of λ_{miss} in this paper are multiples of that pool's λ_{ref}. *E0b* ran Poisson arrivals from an empty engine for 3000 s per rate, with arrival seed 0 at every rate; a run is stable if its final queue is below 50 and its queue rises by at most 10 over the last 1500 s. The registration was then filled and committed, and *E2*, the registered paired test, ran the three arms; the native pool and vLLM repeated the sequence, and the later registered tests follow the paired-test protocol.

4.3 Decision rule and registration

Each run of a registered test lasts 8700 s and is judged over $[t_b + 4500, 8700]$ s; in the rule, λ_{miss} is the cell's λ_{ref}. A BURST run is TRAPPED if $\lambda > \lambda_{\text{miss}}$ and (a) its uncached prefill token rate is within ±15% of the cell's E0a all-miss rate (11,243 tokens/s at 300k) and (b) the slope of its backlog exceeds $0.5(\lambda - \lambda_{\text{miss}})$, the slope being the least-squares slope of 10 s bin means of the sessions in the system (arrived and not ended, including any held at the gate) over the window. A run is RECOVERED if over the last 300 s its hit fraction exceeds 0.8 and its outstanding backlog (requests in flight plus sessions held at the gate) is below 20; NOBURST is good if RECOVERED. A run that is neither is DRAINING if its slope is negative and INDETERMINATE otherwise; the labels take precedence in the order trapped, recovered, draining, indeterminate. A rate PASSES if, in at least two of three seeds, BURST is trapped, NOBURST good and CAPPED recovered in that same seed. Criterion (a) uses tokens because a trapped engine with a growing backlog completes sessions faster than λ_{miss} while it prefills at the all-miss token rate (§3.3); Appendix A gives the evidence.

The registration was filled by formula from committed copies of the Stage C curve, the E0a calibration and the E0b bracket: the window $(1,\ 2.0)\ \lambda_{\text{ref}}$, its upper edge E0b's largest all-stable multiple; the rates one third and two thirds of the way across it, rounded to 0.05 (1.35 and 1.65 λ_{ref}); and the cap, the smallest n with $\mu(n) \geq 0.95\mu^*$, which gives $n_{\text{cap}} = 20$. We call a test pre-registered when its registration file was committed and pushed to GitHub before its runs, with GitHub's push log of our development repository as the record (not an entry in a third-party registry); that repository is private, so readers see

the log only as our export of it in `provenance/` (Appendix D), which cannot be checked against GitHub independently. The E2 and native tests meet this; the vLLM registration reached GitHub after its jobs were submitted, before any could have ended, so that test is registered, not pre-registered; and on the H200 the rule could not place its test points, so that cell has no registered test (§5.5). Appendix B gives the fill, its enforcement by the job script and the push time of every registration, and Appendix C every prediction round with its time record.

4.4 The calibrated simulator

Our discrete-event simulator models a continuous-batching engine whose iteration cost has five parameters fitted only to SGLang's per-batch timing lines in server logs, never to $\mu(n)$; it failed its declared level test, missing one Stage C cell by 42%, but reproduced the curve's shape. LLM coding agents used it to make blind predictions for the registered tests, each in its own copy of the repository and instructed not to read the results they predicted; every round was committed and pushed under a tag before its results were read by its authors, and what each round is blind to differs (Appendix C gives the model, the validation, the rounds and their time record). Round predictions are labelled Bk: B1 the all-miss capacity, B2 the from-empty edge, B3 the paired test, B4 the closed curve, with suffix n for the native pool and v for vLLM (Appendix C.1, Table 13). Our earlier simulator (rounds 1–5), which is not calibrated to the engine and uses assumed costs (all-miss capacity 0.0474 sessions/s), is used for the mechanism comparison of Appendix I.2, in units of its own λ_{miss}. Each experiment cell was also run by its own Claude Code agent, which submitted its jobs and kept a log in `notes/LOG.md` (the cell's operating agent), and a further, coordinating agent dispatched the cells and drafted the paper; the first-principles model of §6 was derived and frozen by agents in the same way. In this paper "session" always means a workload session.

5 Results on the real engines

5.1 The thrashing curve and the from-empty edge

With the 300k pool (Fig. 2, left), $\mu(n)$ rises from 0.176 sessions/s at $n = 16$ to 0.224 at 20 and 0.233 at 24, with hit fractions 0.93, 0.93 and 0.89, and then collapses: the run at $n = 32$ mixes a degraded and a recovered regime (hit fraction 0.51), and at $n = 48$ none of the 20.0M prompt tokens in the window was served from cache. The hit fraction crosses 0.5 at $n = 32.2$ (interpolated between the runs at 32 and 48), about 3.7 sessions above the capacity ratio $N_{\text{lb}} \approx 28.5$ and just below the all-miss onset $n_b = 32.7$ of the first-principles model, which adds the sessions in tool calls (4.9 here) to a pool of running sessions and predicts this curve without using its points (§6). The all-miss level implied by token throughput is roughly flat at 0.072–0.081 sessions/s, so the collapse from the peak is about threefold. With the native pool (Fig. 2, middle), 955,757 tokens or 3.19 times the 300k pool, μ peaks at 0.479 sessions/s at $n = 64$ (0.477 at 80) with 92–93% of turns hitting, and the hit fraction crosses 0.5 near $n = 93$; the all-miss capacity is unchanged within its standard error (0.0718 ± 0.0037 against 0.0715 ± 0.0038 sessions/s turns-based). The 300k pool limit did not create the cliff; it moved it. The ratio $\mu^*/\lambda_{\text{ref}}$ is 3.26 at 300k and 6.67 with the native pool (the registered values, the peak in completed sessions over the turns-based λ_{ref}; 3.15 and 6.48 turns-based). Token-level hit rates have been reported to collapse as concurrent agents grow on SGLang [18] and on vLLM [45, 102]; what we add is $\mu(n)$ in completed sessions per second, the quantity the open-system model uses, at two pool sizes and, below, on a second engine. Beyond the peak neither curve is monotone, which we attribute to the synchronized start of the closed runs; Appendix G gives this and the other details of both curves.

The all-miss capacity and the from-empty edge. E0a gives $\lambda_{\text{ref}} = 0.0715 \pm 0.0038$ sessions/s turns-based (standard error over three seeds), our estimate of λ_{miss}, and $11{,}243 \pm 84$ all-miss prefill tokens/s; the token rate varies by ±1.2% across seeds and the session rate by ±17%, because the engine is bound by prefill tokens and the work of a session is heavy-tailed. From an empty cache and queue (Fig. 6 in Appendix G) the engine stayed stable for 3000 s at 1.25, 1.5, 1.75 and 2.0 λ_{ref}, ending each run with an empty queue and 91–93% of turns hitting, and was unstable at 2.5 and 3.0 λ_{ref}; on this grid, which used the single arrival seed 0, the edge lies between 2.0 and 2.5 λ_{ref} in nominal rates, and the registered window was $(1, 2.0)\,\lambda_{\text{ref}}$. An exploratory replication with fresh seeds 3, 4 and 5 found 2.0 and 2.5 λ_{ref} stable in 3 of 3 seeds each, so the seed-0 bracket is not replicated in nominal multiples; ordered by realized arrival rate, the twelve from-empty runs at this pool are stable up to 2.665 λ_{ref} and unstable from 2.726, still below $\mu^*/\lambda_{\text{ref}}$, and the registered points, 1.35 and 1.65 λ_{ref}, lie inside the stable range on either reading. With the native pool the edge lies between 5.5 and 6.5 times the native λ_{ref} in nominal rates, so that window is $(1, 5.5)\,\lambda_{\text{ref}}$, $4.5\,\lambda_{\text{ref}}$ wide against 1.0 at 300k, although its unstable run received more arrivals than the measured peak, so overload alone can explain its instability. Each rate is a single 3000 s run, so slower escapes are not seen.

5.2 The registered paired tests

The registered analysis of E2, run on all 18 runs with the code frozen at the registration commit, passes at both rates (Table 1, top); by the definition of §4.3 the test is pre-registered. Every BURST run had a hit fraction of 0.000 over the evaluation window, an uncached prefill rate of 1.009–1.029 times the all-miss rate and a mean backlog of 200–332 sessions, and in Fig. 1 each is at zero hits from the 100 s window starting at 1300 s to the end of the run. One ($1.35\,\lambda_{\text{ref}}$, seed 1) is stuck in the same way, but its backlog was flat over the window, so the rule calls it indeterminate: its seed delivered 5.8% fewer arrivals than nominal in the window, and the trapped engine

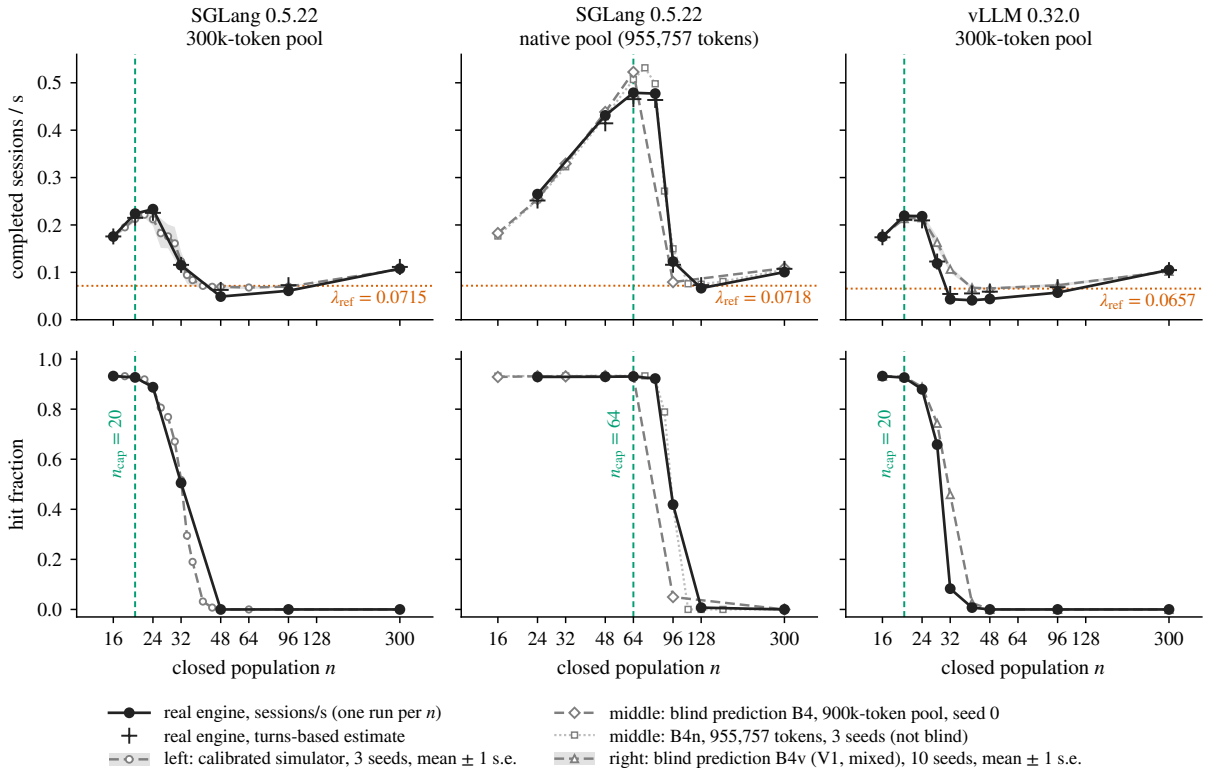


Figure 2: Closed-population thrashing curve, real engine (black, filled; one run per n) against the simulator (gray, open, dashed). Top: $\mu(n)$ in completed sessions/s (plus signs: turns-based estimate); bottom: hit fraction. Left: SGLang, 300k-token pool; calibrated simulator, mean of 3 seeds ±1 s.e. Middle: SGLang, native pool (955,757 tokens); blind prediction B4 at 900k tokens (seed 0); light gray, dotted: B4n at the real pool (3-seed mean), which is not blind (Appendix C.1). Right: vLLM 0.32.0, 300k-token pool; blind prediction B4v (variant V1: token costs scaled to vLLM's all-miss rate; mixed batching), mean of 10 seeds ±1 s.e. (§5.2). Dotted: each cell's measured λ_{ref}, the turns-based E0a estimate of λ_{miss}. Green dashed: the registered caps, $n = 20$, 64 and 20.

completed sessions about as fast as they arrived. The pass at 1.35 λ_{ref} therefore rests on 2 of 3 seeds, the minimum the rule allows. Every NOBURST and CAPPED run is good (hit fractions 0.921–0.934, backlogs of 8–12 sessions, most asleep in tool calls); in the window CAPPED cannot be told apart from NOBURST, because the gate had drained the burst 1547–2693 s after it. Counting each seed once, since both rates use the same three seeds with the same burst and nested arrivals, a post-hoc one-sided sign test of BURST's hit fraction against NOBURST's gives $p = 1/8$, the smallest value three seeds allow. The trapped runs still completed 1.13–1.35 λ_{ref}, above λ_{miss}, because their young backlog has short turns; only the uncached prefill rate sits at the all-miss level, which is why the registered rule uses tokens (§3.3). So at loads that the engine carried from empty, 200 sessions arriving within a minute, an order of magnitude more than the 8–12 present in the good state, moved it into an all-miss state that it did not leave within 7500 s, and a cap of 20 sessions, within 5% of the measured peak of $\mu(n)$, prevented that, at the price of admission delay that the registered criteria do not measure (349–543 sessions per CAPPED run waited at the gate, 405–471 s on average). Good NOBURST arms are the expected outcome even for a metastable good state: by the heuristic F2 of §6.3, a run at these rates leaves it on its own within 8700 s with probability about 0.5% at 1.35 and 5% at 1.65 λ_{ref}. Appendix G gives the arrivals, the backlog after the burst, the drain and the cost of the cap in detail.

Native pool. A second registration, filled by the same formulas from the native curve, E0a calibration and E0b bracket, gives the window (1, 5.5) λ_{ref}, the rates 2.5 and 4.0 λ_{ref} and $n_{cap} = 64$, flagged ambiguous because $\mu(80)$ is within 0.35% of $\mu(64)$; it reached GitHub before the first native E2 job was submitted. The registered analysis passes at both rates, and every run is in the predicted state (Table 1): all six BURST runs were trapped, all six NOBURST runs were good and all six CAPPED runs recovered (Fig. 7 and the details in Appendix G). Pooled post hoc over the two pre-registered SGLang tests (a descriptive count: they share three seeds' session sequences), NOBURST was good and CAPPED recovered in 12 of 12 runs each, and BURST was at zero hits in 12 of 12 and trapped by the rule in 11 of 12.

Table 1: The three registered tests: the two pre-registered SGLang tests, E2 (300k-token pool, cap 20, $\lambda_{\text{ref}} = 0.0715$ sessions/s) and native E2 (955,757-token pool, cap 64, $\lambda_{\text{ref}} = 0.0718$ sessions/s), and the registered vLLM test (300k-token pool, cap 20, $\lambda_{\text{ref}} = 0.0657$ sessions/s). States for seeds 0, 1, 2: T trapped, I indeterminate, D draining, G good, R recovered. Hit fraction, uncached prefill rate in multiples of the all-miss rate, mean backlog (sessions in the system) and its slope (sessions/s) over the evaluation window; ranges over seeds; good-arm slopes in absolute value. Slope thresholds: 0.0125, 0.0232, 0.0538 and 0.1077 at 1.35, 1.65, 2.5 and 4.0 λ_{ref} on SGLang, 0.0115 and 0.0213 at 1.35 and 1.65 λ_{ref} on vLLM. Rates are nominal; the base arrivals realized in the window, which the three arms of a seed share, were 1.45, 1.27, 1.42; 1.73, 1.61, 1.75; 2.73, 2.48, 2.58; 4.28, 3.89, 4.09; and on vLLM 1.45, 1.24, 1.41 and 1.76, 1.56, 1.74 λ_{ref} for seeds 0, 1, 2.

$\lambda/\lambda_{\text{ref}}$	arm	states	hit	tokens	backlog	slope
300k-token pool (E2)						
1.35	BURST	T I T	0.000	1.009–1.025	200–232	0.0036–0.0222
	NOBURST	G G G	0.930–0.934	0.08–0.09	7.9–9.7	<0.001
	CAPPED	R R R	0.930–0.934	0.08–0.09	7.9–9.7	<0.001
1.65	BURST	T T T	0.000	1.025–1.029	283–332	0.0277–0.0362
	NOBURST	G G G	0.921–0.934	0.10–0.13	10.9–11.8	<0.002
	CAPPED	R R R	0.929–0.934	0.10–0.11	10.9–11.7	<0.002
native pool (native E2)						
2.5	BURST	T T T	0.000	1.074–1.078	557–601	0.079–0.085
	NOBURST	G G G	0.929–0.935	0.15–0.16	15.6–18.5	<0.003
	CAPPED	R R R	0.929–0.935	0.15–0.16	15.6–18.5	<0.003
4.0	BURST	T T T	0.000	1.106–1.124	1098–1139	0.146–0.164
	NOBURST	G G G	0.929–0.935	0.24–0.27	28.2–33.6	<0.004
	CAPPED	R R R	0.929–0.935	0.24–0.27	28.1–33.5	<0.004
vLLM 0.32.0, 300k-token pool (registered, not pre-registered)						
1.35	BURST	T D T	0.000	0.995–1.012	201–222	−0.00005 to 0.0170
	NOBURST	G G G	0.930–0.931	0.07–0.09	7.0–8.4	<0.001
	CAPPED	R R R	0.930–0.931	0.07–0.09	7.0–8.4	<0.001
1.65	BURST	T I T	0.000	0.948–1.018	281–330	0.0183–0.0341
	NOBURST	G G G	0.932–0.933	0.10–0.11	9.3–10.8	<0.001
	CAPPED	R R R	0.932–0.933	0.10–0.11	9.3–10.9	<0.001

A second engine: vLLM. vLLM 0.32.0 ran with the SGLang cell's settings where they exist (FCFS, prefix caching on, chunked prefill of 8192 tokens, at most 256 running requests, a 300,000-token pool). Its all-miss capacity is 8% below SGLang's ($\lambda_{\text{ref}} = 0.0657 \pm 0.0030$ sessions/s); its closed curve has SGLang's shape with an earlier, steeper cliff (peak 0.219 sessions/s at $n = 20$; hit fraction crossing 0.5 near $n = 29.1$); and its from-empty edge lies between 2.0 and 2.5 times its λ_{ref} (Fig. 2, right). A registration filled from these by the SGLang formulas gives the window (1, 2.0) λ_{ref}, the rates 1.35 and 1.65 λ_{ref} and $n_{\text{cap}} = 20$. It reached GitHub only after the 18 jobs were submitted, when two had started and none had ended, so the test is registered, not pre-registered; and its BURST arm reuses the seeds, and so the arrivals, of six earlier exploratory vLLM BURST runs, all of which had been at zero hits after the same burst, so it does not replicate them independently. The registered analysis passes at both rates (Table 1, bottom): every BURST run had zero hits over the evaluation window, and NOBURST stayed good and CAPPED recovered in all six runs each; the two BURST runs that the rule does not call trapped (seed 1, the seed with the fewest arrivals) were stuck at the all-miss token rate but failed its slope test. Of the mechanism checks frozen before any run had ended, those on the gap between a session's turns (§6.4), on a full pool rewritten at the all-miss rate and on an engine-side latency rule held in all 18 runs, but the online detector, a follow-up hit fraction below 0.2 over 60 s, alarmed 100–120 s after every burst and also in two CAPPED runs while their gates drained, so the frozen decision rule is not met and we do not claim the detector (Appendix G). Across the three registered tests, BURST was at zero hits in 18 of 18 runs and trapped by the rule in 15, NOBURST was good in 18 of 18 and CAPPED recovered in 18 of 18; the tests share seeds, so these counts are descriptive.

5.3 Rescue from an established trap

The paired tests show that a cap present when the burst arrives prevents the trap. Two further registered tests, R1 and R4 (the labels of the plan that defined them, which defined no other rescue test; Table 9 lists every registration we filled), ask whether a cap can undo an established trap at the same arrival rate without shedding work, and whether the good state then persists without it. Each run replays the arrival plan of the paired test's BURST run byte for byte, as a fresh run on a fresh server, until a client-side cap is engaged at t_{on}: R1 on the 300k pool at 1.35 and 1.65 λ_{ref} with cap 20 from $t_{\text{on}} = 1800$ s, and R4 with the native pool at 4.0 λ_{ref} with cap 64 from 2100 s, the registered CAPPED caps, engaged 600 and 900 s after the burst. At t_{on} more sessions than the cap are already admitted, so sessions over it give up their slot at their next tool-call boundary and wait at the client, together with new arrivals;

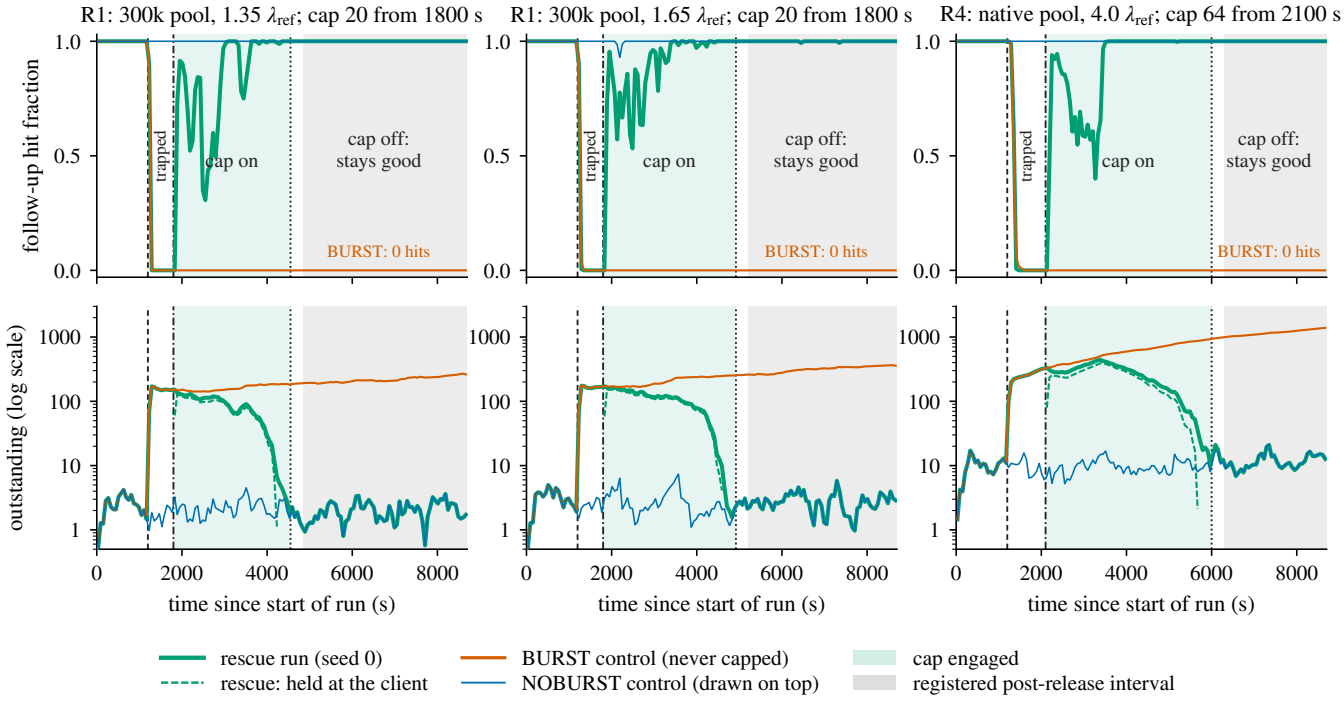


Figure 3: Trap, rescue, release and persistence: the registered rescue tests R1 and R4, seed 0 of each cell. Each rescue run (green) replays the arrival plan of a paired test's BURST run (a fresh run on a fresh server) until a client-side cap is engaged at t_{on} (dash-dotted; 1800 s for R1 on the 300k pool, cap 20; 2100 s for R4 with the native pool, cap 64), and the cap is removed at t_{rel} (dotted; 4550, 4920 and 6000 s) once both client queues have been empty for 300 s. The same-seed BURST (vermillion) and NOBURST (blue) runs of the paired tests share its arrivals. Dashed: the burst of 200 sessions at 1200 s. Green band: cap engaged. Gray: the registered post-release interval $[t_{rel} + 300, 8700)$ s. Top: follow-up hit fraction (turns after a session's first with more than half of the prompt cached, so the good state is near 1, not 0.93 as in Fig. 1) in 60 s bins. Bottom: outstanding work, requests in the engine plus sessions held at the client (log scale); dashed green: the sessions the rescue run holds at the client, waiting for admission or paused. Seeds 1 and 2 behave alike (Tables 2 and 16).

paused sessions are readmitted first, each queue in FIFO order, and once the admitted set has fallen to the cap the gate is that of Proposition 1. The cap is removed at the first 10 s tick, no earlier than $t_{on} + 600$ s, at which both client queues have been empty for 300 s. A run counts as rescued if it was trapped at t_{on} (a follow-up hit fraction below 0.2 over the preceding 300 s and more than 50 requests or sessions outstanding), the cap was released by 7200 s, and over $[t_{rel} + 300,\ 8700)$ every 300 s window had a follow-up hit fraction of at least 0.5, the whole interval above 0.8, an uncached token rate below half the all-miss rate, and the registered RECOVERED test held over the last 300 s; a window below 0.2 is a relapse. A cell passes if at least 2 of its 3 seeds are rescued and every same-seed BURST control stays stuck; two relapses, or two releases after 7200 s, among R1's six runs would refute it, and a failed session would withdraw the claim that nothing was shed. Both registrations reached GitHub at 22:39:02 CDT on 2026-10-30, 22 minutes before the jobs were submitted (Appendices A and B).

The registered analysis passes in both cells (Table 2, Fig. 3). Every run was trapped when the cap was engaged (follow-up hit fraction 0.000–0.002 over the preceding 300 s, with 139–172 requests or sessions outstanding at 300k and 298–324 with the native pool), and all 9 runs were rescued, with no relapse, every release before 7200 s and no failed session. Hits returned within three minutes of the engage (in every run the first 60 s bin with a follow-up hit fraction of 0.8 or more was the one from 120 to 180 s after it) but fluctuated until the backlog drained: in later 60 s bins the follow-up hit fraction fell again to minima of 0.13–0.60 per run (the last bin below 0.8 began 780–1620 s after the engage), and over 300 s bins it stayed at or above 0.56 in R1 and dipped to 0.47–0.57 in R4 (Appendix G.3, Table 16). Draining the backlog took longer; the cap last held a session back 301–310 s before each release, which came at 3980–4920 s on the 300k pool and 5980–6140 s with the native pool.

With the cap removed, every run stayed good at the same arrival process for the remaining 3780–4720 s (300k) and 2560–2720 s (native) of the run. Over $[t_{rel} + 300,\ 8700)$ each rescued run matched its NOBURST control, which never saw the burst: the same follow-up hit fraction and mean outstanding work (Table 2), the same turn latency in the engine (1.38–1.55 s against 1.38–1.67 s at 300k, 2.13–2.49 s against 2.13–2.51 s with the native pool), follow-up turn counts within 7 of each other, and the same number of sessions unfinished at 8700 s;

Table 2: The registered rescue tests R1 and R4 (seeds 0, 1, 2; Fig. 3); ranges over seeds. Rescued: runs that meet the registered rule, with relapses in parentheses. After release: follow-up hit fraction and mean outstanding work (requests in flight plus sessions held at the client) over $[t_{\text{rel}} + 300,\ 8700)$ s for the rescue run and, over the same interval, for its same-seed controls, which share its arrivals. Delay: session-hours of pauses and admission waits at the client. The rows below the release time are descriptive, computed after the registered verdict.

	R1, 300k $1.35\ \lambda_{\text{ref}}$	R1, 300k $1.65\ \lambda_{\text{ref}}$	R4, native $4.0\ \lambda_{\text{ref}}$
cap; engaged at	20; 1800 s	20; 1800 s	64; 2100 s
rescued (relapses)	3/3 (0)	3/3 (0)	3/3 (0)
release t_{rel} (s)	3980–4550	4430–4920	5980–6140
after release: follow-up hit fraction			
rescue	0.999–1.000	0.994–1.000	1.000
NOBURST	0.999–1.000	0.991–1.000	1.000
BURST	0.000	0.000	0.000
after release: mean outstanding work			
rescue	1.9–2.2	2.5–2.8	8.8–11.8
NOBURST	1.9–2.2	2.5–3.1	8.8–11.8
BURST	195–219	272–317	1133–1178
sessions paused	121–130	138–143	220–251
delay (session-h)	38–65	50–78	222–230

each rescued run completed exactly 200 sessions more than its control, so every burst session finished. Over the same interval the same-seed BURST control, which received the same burst and was never capped, stayed at a follow-up hit fraction of 0.000 with 195–317 (300k) and 1133–1178 (native) requests outstanding, growing in 8 of the 9 runs (the $1.35\ \lambda_{\text{ref}}$ seed-1 control stayed flat at about 200, as in §5.2), and turns that took 159–231 s and 523–532 s. At one arrival process the engine thus sat in either of two states that each persisted to the end of the run, and a transient intervention moved it from the trapped one to the good one, which then sustained itself without the cap: hysteresis on a real engine, as Corollary 1 predicts for a cap inside (n_1, n_2). Persistence here means that the drained state sustains itself with the cap removed: the release rule waits until the cap no longer binds, so the test does not cover a release while it still binds. After release the population was at or above the cap for 0–4.5% of the time at 300k (at most 27 sessions against 20), and with the native pool it stayed below it (at most 43–55 against 64).

The rescue costs waiting, not sessions. In the rescue runs 121–143 (300k) and 220–251 (native) sessions were paused, on average 386–569 s per paused session, and 196–344 and 1050–1101 sessions waited at admission, 408–683 s on average and at most 1318 s: 38–78 and 222–230 session-hours of delay at the client. Counting pauses, admission waits and the latency of every request, the rescued runs waited 4.4, 5.1 and 4.6 times less in total than their uncapped controls (cell means; a lower bound, since the controls' 206–1404 unfinished sessions were still waiting at 8700 s). Prevention is cheaper: the CAPPED arm of the paired tests, which applies the same cap from the start, delayed sessions by 40–65 session-hours at 300k, about as much as the rescue, and by 18–21 with the native pool, about 11 times less, because there the rescue starts from about 300 outstanding and its client queue peaks near 400. These comparisons, the persistence measures above and Fig. 3 are descriptive, computed after the registered verdict (Appendix G.3, Table 16). The blind simulator and the frozen model both predicted the rescue; both drain the native gate too fast, and R4 released 800–1750 s after the simulator's per-seed predictions and 19–22% after the model's, outside its ±15% tolerance, as was one of the six R1 releases (+18.8%; §7).

5.4 The basin of the good state

The paired tests use one burst size, 200 sessions, which in our earlier simulator trapped every work-conserving policy. Two registered BURST-only tests at $1.65\ \lambda_{\text{ref}}$ on the 300k pool vary it, with the same seeds: BASIN-1 with 25 and 100 sessions and BASIN-2 with 40, 50 and 60. BASIN-1 passed: a burst of 100 sessions left the engine stuck (hit fraction 0.000 over the evaluation window) and trapped by the registered slope rule in 3 of 3 seeds, and a burst of 25 left it good (RECOVERED, hit fraction 0.92–0.93, within 0.003 of the no-burst run with the same arrivals) in 3 of 3. The good state of the real engine therefore has a finite basin, whose boundary lies between 25 and 100 sessions in every seed, as in the calibrated simulator, where 25 sessions never trapped and 50 trapped about half the time (Appendix I.1). BASIN-2 tests the first-principles model's critical fresh burst of 53.4 sessions, from a heuristic of §6.3 that counts a burst session at its peak footprint; its outcome is pending in this version. BASIN-1's registration reached GitHub 68 minutes before its jobs were submitted; §7 scores the frozen predictions.

5.5 A faster GPU narrows the window

The H200 cell repeats the 300k SGLang cell on one H200 NVL with SGLang's default attention backend for that GPU (FlashAttention 3; the A100 cells used FlashInfer), so the comparison is of GPU and default backend together. Against the A100, the all-miss rate rises 2.71 times ($\lambda_{\text{ref}} = 0.194$ sessions/s, turns-based), the peak of the closed curve only 1.63 times ($\mu^* = 0.367$ sessions/s at $n = 32$, turns-based; 1.65 times in completed sessions) and the from-empty edge, the largest all-stable nominal multiple of each GPU's seed-0 grid, 1.76 times: the engine stayed stable from empty at 1.1, 1.2 and 1.3 λ_{ref} and was unstable at 1.4, 1.55 and 1.75 λ_{ref}. Reseeds were stable at $1.3\ \lambda_{\text{ref}}$ in 3 of 3 seeds and at 1.4 in 2 of 3. The seed-0 grid received 7–8% more arrivals than nominal, and, as on the A100, stability over all ten runs is monotone in the realized rate, with the edge between 1.496 and 1.503 λ_{ref}; the 1.4 seed that stayed stable received 1.43. That bracket is only four sessions wide and the 1.503 run ended just above the queue threshold (50.4 against 50), so the data neither show a sharp edge nor need a stochastic band to explain them. The window shrinks from $(1,\ 2.0)\ \lambda_{\text{ref}}$ on the A100 (nominal, on the seed-0 grid) to $(1,\ 1.3)\ \lambda_{\text{ref}}$, too narrow for our registration rule: it places the test points at 1.10

and 1.20 λ_{ref}, outside the band (1.1, 1.17) that keeps them at least 10% from the window's ends, so no registration, paired test or decisive claim exists for this GPU. The cliff moves out: the hit fraction crosses 0.5 near $n = 38.9$, against 32.2 on the A100, and past it falls gradually, to 0.46 at $n = 40$ and 0.16 at 52, reaching zero only between 52 and 96. Memory and tool time do not speed up with the GPU, so a faster GPU raises the all-miss capacity more than the peak, as the speed law of §6 predicts for a uniform speed-up; §7 scores the model's frozen predictions for this cell. KAIROS shows qualitatively that a slower clock tips a fixed arrival rate into thrashing [101], and VAMP locates KV capacity boundaries on H100 and H200 GPUs [55]; what we add is the measurement in units of λ_{miss} and the opposite movement of the onset and the peak ratio.

5.6 Other orderings (exploratory)

Three other engine cells ran the BURST arm at the registered rates and seeds, with no control arms (Fig. 8 and details in Appendix G). Disabling SGLang's waiting-prefix refresh made no difference for FCFS. Stock LPM partly helps: from the 200-session burst it recovered in 4 of 6 runs at 1.35 λ_{ref} but in 1 of 6 at 1.65. On our reading of its source it stops re-sorting its queue when more than 128 requests wait, and in each run that ended trapped the last hit came when the queue last stood at or below 128. With that fallback disabled by a job-local hook (LPM-INF, exploratory, with its decision rule and the simulator's prediction frozen beforehand), LPM did not collapse on the two arrival streams on which stock LPM had collapsed in 4 of 4 runs (token hit fraction, the share of prompt tokens served from cache, pooled by completion time over the evaluation window: 0.59 and 0.42 against 0.000; unlike the turn hit fraction it has no 0.932 ceiling), but no run recovered by the registered rule; with one rate and two informative seeds this is directional evidence, and it does not separate LPM's ordering from the LRU touch of its prefix matching.

6 A first-principles model of the thrashing curve

Section 3.3 takes $\mu(n)$ as measured. Here we *predict* it, together with the all-miss level, the peak, the from-empty edge, the trapped state and the cap condition, from the pool size M, the workload of §3.1, the tool-time law, and the iteration-cost model of our simulator (§4.4), which was fitted only to server-log iteration timings. No parameter is fitted to any $\mu(n)$ point, from-empty outcome or trap measurement. Every proposition below is about a deterministic fluid model, and the layers of §6.3, the trapped-state layer included, are heuristics; we do not prove that the engine converges to the model. The model, its code and the proofs below were derived by LLM coding agents under the authors' direction (see Use of AI tools); these agents worked from every committed result of the three measured cells and the registered protocols, and saw no H200, Llama, pool-sweep, basin, fresh-seed, LPM-INF or rescue result before freezing the predictions of §7 (Appendix C). The blindness of each prediction is stated with respect to these agents and, where it differs, to the first author.

6.1 Model

Let $w_k = P(T \geq k)/E[T]$ be the turn mix, $E_k = L_k + d$ the KV of a session after turn k, $\bar{E} = \sum_k w_k E_k = 10{,}569$ tokens, $A_1 = \sum_k P(T \geq k+1)E_k = 144{,}960$ the KV a session carries into tool calls over its life, $E[E_T] = 11{,}045$ its final KV, $\bar{z} = E[Z]$, and $\bar{z}' = \bar{z}(1 - 1/E[T])$ its tool time per turn. The tool time Z has distribution F_Z and density f. *Server.* At turn completion rate ν (turns/s) and follow-up hit probability h, turn k needs prefill GPU time $t_k(h) = t_k^m - h\,\mathbf{1}\{k \geq 2\}(t_k^m - t_k^h)$, with t_k^m, t_k^h the miss and hit prefill times of the cost model (a hit prefills $\Delta + 1 = 551$ tokens). SGLang runs prefill and decode in separate iterations with prefill priority, so prefill takes a fraction $p = \nu \sum_k w_k t_k(h)$ of the GPU and the number R_D of decoding requests solves $R_D(1-p) = \nu(d-1)\tau_D(R_D)$, where τ_D is the cost model's decode iteration time. A turn occupies the engine for $S_k = t_k(h) + S_D$ with $S_D = R_D/\nu$ and reserves its KV E_k throughout, so running requests hold $K_{\text{run}}(\nu, h) = \nu \sum_k w_k E_k S_k$ tokens. *Memory.* Idle KV (sessions in tool calls, and the KV of finished sessions, which the engine cannot know is dead) is evicted least recently used first.

Assumption MF (Che [17, 38])**.** *In stationarity an idle item is evicted at a deterministic idle age R.*

By $H = \lambda G$ the idle KV is then $\mu\mathcal{D}(R)$ with $\mathcal{D}(R) = A_1 E[\min(Z, R)] + E[E_T]R$, and the pool is full, so

$$M = K_{\text{run}}(\nu, h) + \frac{\nu}{E[T]}\mathcal{D}(R), \qquad h = F_Z(R), \qquad n = \nu\,(\bar{S} + \bar{z}'), \qquad \mu = \nu/E[T], \tag{3}$$

the last two being Little's law with $\bar{S} = \sum_k w_k S_k$. Admission is first-come-first-served, work-conserving and memory-gated (assumption (S4) of Appendix F), so requests queue only when running reservations alone fill the pool ($K_{\text{run}} = M$, hence $R = 0$ and $h = 0$). With $K_{\text{run}} = 0$ and no queue, Eq. (3) is the characteristic-time equation of Yao et al. [97] term for term. Three additions make the hit rate feed back on itself. Running reservations depend on h, because under prefill priority misses starve decode. Throughput is set by a server and a closed population, not by an exogenous arrival rate. And there is a memory-gated all-miss branch. On every partially hitting state the reuse gap is still the exogenous tool time; only on the all-miss branch does the queue set it. The cliff therefore comes from the first addition, with exogenous gaps; the queue-set gap sustains the all-miss branch.

6.2 Results

Proposition 2 (stationary states and the all-miss onset)**.** *Let ν_b solve $K_{\text{run}}(\nu, 0) = M$, $\mu_b = \nu_b/E[T]$, and $E_S = \sum_k w_k E_k S_k / \sum_k w_k S_k$ at $(\nu_b, 0)$. (a) Each $R > 0$ gives exactly one queue-free stationary state; under (S4), the states*

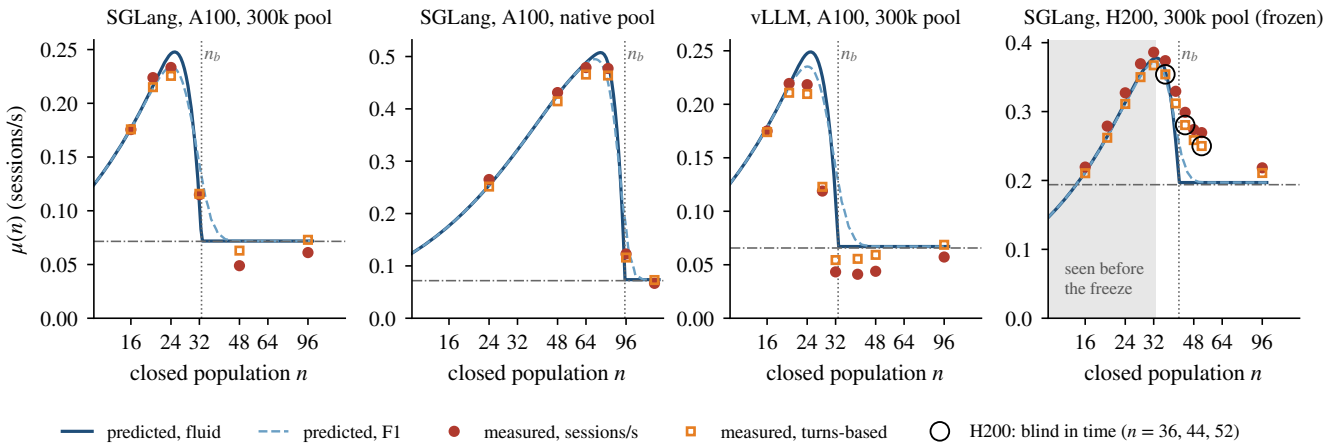


Figure 4: Predicted thrashing curves against the engine. Lines: the first-principles model (solid, fluid; dashed, with the demand-fluctuation layer F1). Its inputs are the pool, the workload and a cost model fitted to server-log timing only; vLLM and the H200 also get one speed factor fitted to their all-miss token rate. Points: measured $\mu(n)$, sessions/s (filled) and turns-based (open). Dotted: the predicted all-miss onset n_b (Eq. (4)); dash-dotted: the measured λ_{ref}. No measured $\mu(n)$ point enters any prediction. The first three panels are in sample: the agents that derived the model knew every point. The fourth is out of sample: the H200 curve was predicted under tag `r13-theory-pred` before those agents read it; the runs at $n = 36$, 44 and 52 (ringed) were submitted after the freeze and are blind in time, those at $n \le 32$ (shaded) had been seen by the first author before it, and the others are blind by access only (§7). The n axis is logarithmic, as in Fig. 2, and ends at 140, so the $n = 300$ runs are not plotted. The measured points below λ_{ref} beyond the cliff ($n = 48$–96 at 300k, 128 with the native pool, 32–96 on vLLM) are not described by the stationary model; at 300k we attribute them to the synchronized start of the closed runs (§5.1).

with a queue are exactly the all-miss states ($h = 0, \nu = \nu_b$); $\mu(n) = \mu_b$ for all large n, so μ_b is the model's λ_{miss} and μ_∞ (the model's counterpart of Proposition 6 of Appendix E under its assumption (A4), that the congested engine computes prompt tokens as fast as with caching disabled). (b) All-miss states exist exactly for $n \ge n_b$, where

$$n_b = \nu_b \bar{z}' + M/E_S, \tag{4}$$

the sessions in tool calls at the all-miss rate plus one pool's worth of running sessions. (c) If every GPU cost is divided by s (a uniformly faster GPU), then $\nu_b \mapsto s\nu_b$, E_S is unchanged, and $n_b(s) = n_b(1) + (s-1)\,\nu_b(1)\,\bar{z}'$.

At 300k, $n_b = 4.94 + 27.76 = 32.70$ (native pool: 94.86; vLLM: 32.83). Parts (a) and (c), Proposition 3(b)–(c) and Proposition 7 rest on Lemmas 2 and 3, which are for the separate-iteration (SGLang) server; for vLLM's mixed iterations n_b follows from the identity of part (b), which holds whenever the mixed server's ν_b exists, that is, while its chunk budget does not bind (Appendix F). The pool part M/E_S has the form of MISA-T's cap, a reserved block budget over a conservative session length [45], and of AgentKV's collapse predictor (from its abstract) [3]. Here E_S weights each turn's KV by its service time at the all-miss rate (about 10.8k tokens at 300k), so $M/E_S = 27.8$ lies just below $N_{\text{lb}} = M/E_{\text{lb}} \approx 28.5$ of §3.2 and $M/\bar{E} = 28.4$; all three are capacity ratios of this form. What is new is the tool-call term and the identification of n_b as the onset of the all-miss branch. On the two SGLang cells the measured 0.5-hit crossing (32.2 and 93.4) lies just below n_b; on vLLM it lies at 29.1, nearer the pool part (28.2) than n_b. Part (c) says that a uniformly faster GPU moves the onset *outward*: on an H200 at the same pool ($s = 2.742$, fitted to its all-miss token rate) $n_b = 41.3$. On the H200 the measured crossing moved out, from 32.2 to about 38.9, though past it the engine lost its hits more gradually than the model (§5.5, §7).

Proposition 3 (speed and tool time enter only through $s\bar{z}$)**.** *Write $\omega_b = \mu^*/\mu_b$ for the peak ratio and $Z = \bar{z}Y$ with Y fixed. (a) $\lambda_{\text{miss}}(C/s) = s\,\lambda_{\text{miss}}(C)$ and $\omega_b(C/s, \bar{z}) = \omega_b(C, s\bar{z})$ exactly, for cost model C. (b) ω_b is non-increasing in $s\bar{z}$, strictly while $\omega_b > 1$. (c) $\omega_b \to 1$ as $s\bar{z} \to \infty$.*

A uniformly faster GPU therefore lowers the peak ratio, the upper end of the window when the from-empty edge does not bind: the good state is limited by memory and tool time, neither of which speeds up. The same holds for slower tools. On the H200 the measured peak ratio against λ_{ref} was 1.89 (turns-based), against 3.15 on the A100 and 1.92 predicted (§7; the H200 is not a uniform speed-up: it also changes the attention backend, and its 64-to-32 all-miss token-rate ratio is 1.046 against 1.019 under uniform scaling, §6.6); this comparison is not blind to the first author, who had seen the H200 values for $n \le 32$ before the freeze.

Proposition 4 (with exogenous gaps, zero hits need running requests to hold the pool)**.** *Suppose that the reuse gap of every follow-up is its tool time Z, with no queue wait, that running requests hold $K_{\text{run}} \ge 0$ tokens of the pool, and that idle KV is evicted at a deterministic idle age R (Assumption MF), so that $M - K_{\text{run}} = \lambda_0\,\mathcal{D}(R)$ at session throughput λ_0. Then*

$R \geq (M - K_{\text{run}})/(\lambda_0 E[T]\bar{E})$, *so the follow-up hit probability* $F_Z(R)$ *is at least* $F_Z\big((M - K_{\text{run}})/(\lambda_0 E[T]\bar{E})\big)$*; and if* F_Z *is continuous and strictly increasing, a follow-up hit probability of at most* h *requires* $K_{\text{run}} \geq M - \lambda_0\, \mathcal{D}(F_Z^{-1}(h))$.

Proof. Since $\min(Z, R) \leq R$ and $A_1 + E[E_T] = \sum_k P(T \geq k)E_k = E[T]\bar{E}$, we have $\mathcal{D}(R) \leq E[T]\bar{E}\, R$, which gives the first bound, F_Z being non-decreasing. A hit probability of at most h means $R \leq F_Z^{-1}(h)$, and $\mathcal{D}$ is increasing. □

With $K_{\text{run}} = 0$ the balance is the characteristic-time equation of Yao et al. [97], and the proposition says that a hit probability near zero needs running requests to hold almost the whole pool. Our model is an exogenous-gap closure of this kind on every queue-free state, and this is how its cliff arises: running reservations grow as hits fall, so on the queue-free branch K_{run}/M rises continuously towards 1 as $R \downarrow 0$ (on the 300k pool 0.28 at the fluid peak, 0.89 at $n = 32$ and 0.97 at $n = 32.5$, with hit fractions 0.89, 0.37 and 0.15, all with no queue), and $K_{\text{run}} = M$ holds exactly only on the all-miss branch, where requests queue (Proposition 2(a)). Evaluated retrospectively on each closed run, with nothing fitted, exogenous-gap closures with little running KV never reach the measured zeros: at $n = 48$ on the 300k pool, where no turn hit, Yao et al.'s fixed point at the measured throughput gives a hit fraction of 0.93, the same equation with its rate set by Little's law at zero service time (about 11 times the measured throughput) 0.50 and Che's approximation with the tool time as the gap 0.55, and the proposition's bound with $K_{\text{run}} = 0$ at the measured throughput is 0.93, while the model, whose running reservations fill the pool there, gives 0.000 (Appendix F gives every measured point past the peak of the three cells). The model is not exact on the cliff either: its fluid hit fraction differs from the measured one by more than 0.1 at four of those 15 points, 300k at $n = 32$ (0.51 measured, 0.37 fluid), vLLM at $n = 28$ (0.66; 0.81 fluid, 0.72 with F1) and $n = 32$ (0.08; 0.44 fluid) and the native pool at $n = 96$ (0.42; 0.00 fluid, 0.36 with F1), and by more than 0.3 at the last two; at the other 11 it is within 0.04.

Evaluated numerically, the fold criterion for the closed curve (Proposition 7, stated with a proof sketch in Appendix F) finds every predicted cell with the registered exponential tool times fold-free, so on these cells metastability is a property of the open system; dispersed tool times with the same 5 s mean can make the closed curve fold, and in our example (Fig. 5) the all-miss state and a good state are then both stable at one population, but none of our engine runs used such a law.

On the predicted curves, which are fold-free and, numerically, unimodal in the sense of §3.3 (with $\mu = \mu_b$ on $[n_b, \infty)$), Corollary 1 applies with $n_2 \leq n_b$, and on $[n_b, \infty)$ Eq. (1) makes the backlog grow at $\lambda - \mu_b$ (Appendix F). The model does not use this for the trap: its trapped-state predictions come from the round-robin layer RR of §6.3. The registered caps lie inside (n_1, n_2): 20 in $(8.5, 32.2)$ at 1.35 and in $(10.4, 31.8)$ at 1.65 λ_{ref} on the 300k pool, 64 in $(27.8, 90.0)$ at 4.0 λ_{ref} with the native pool. The upper end n_2 inherits the quasi-static assumption, which is weakest near n_b (§3.3); for these caps only $n_1 < c < n_b$ matters.

Proposition 5 (trapped identity)**.** *In the all-miss FCFS regime, where every present session completes one turn per scheduler cycle, the follow-up gap divided by* $R_M = M/\text{churn}$ *(churn = uncached prompt plus completion tokens per second) equals* $\sum_i (L_{k_i} + d)/M$ *over the sessions present, for any cost model.*

To the extent that a trapped run meets this one-turn-per-cycle hypothesis, its gap$/R_M$ is the total next-turn context in the system over the pool. The engine meets the hypothesis only approximately (arrivals, departures and a growing backlog): on the three 300k BURST runs at 1.35 λ_{ref} the measured backlog and prompts give 7.24, 6.43 and 6.81 against measured values of 6.92, 6.35 and 6.79, within 5%. The proposition says nothing about the good state; there we approximate the median gap by the median tool time plus the mean service time, so that gap$/R_M \approx (\bar{z} \ln 2 + \bar{S})\,\text{churn}/M$. Section 6.4 compares both with the engine. The cycle argument is EfficientAgent's: in a backlogged pool the other agents' contexts are referenced between two turns of one agent, so the reuse distance is about the pool's total context (its Eq. 5, an approximation for a host KV tier in a closed pool) [83]. Proposition 5 gives its exact time-domain form, gap$/R_M$, on the all-miss FCFS branch of an open system, for any cost model.

6.3 Heuristic layers (nothing fitted)

Four heuristic layers, with nothing fitted, extend the model where the propositions do not reach; Appendix F defines them. *RR*, the trapped layer, replaces Eq. (1) in the trap by a fluid FCFS round robin in which every follow-up misses and every present session completes one turn per cycle; the trapped rows of Table 3 and the model column of Table 4 come from it. *F1* averages the curve over the KV-demand fluctuation of n sessions. *F2* predicts the from-empty edge from a one-dimensional diffusion of the demand. *F3* gives the probability that a burst leaves a run stuck; burst sessions are fresh, so a burst counts by its peak footprint (4,331 tokens per session, at turn 12), not by its number. *F4* gives the release time of a cap engaged inside the trap.

6.4 The sustaining variable on the engines

At the same arrival rate the good and the trapped state differ in how long a session's KV lies idle between turns compared with how long the pool keeps idle KV. Let the gap of a follow-up turn be the time from the completion of its session's previous turn to its own completion, the tool time plus the turn's latency, and let $R_M = M/\text{churn}$, where churn is the rate at which new KV is written (uncached prompt tokens plus completion tokens per second). Under LRU an idle prefix is evicted by the time M newer tokens have been written, so a follow-up whose idle time exceeds R_M misses: Denning's criterion, an interval between references longer than the window [27], with Che's characteristic time as the window [17]. The gap exceeds the idle time by the turn's own service, and SGLang's refresh of waiting prefixes (§2) resets the age of some of them, so gap$/R_M$ is a proxy for the criterion, not the criterion itself.

Table 3: First-principles predictions against measurements (`results/theory/merged/validation_table.md`, frozen at tag `r13-theory-pred`). Inputs: pool, workload, server-log cost model; vLLM also one speed factor fitted to its own all-miss token rate. Measured μ is turns-based. The first row is the model's 32-session cache-off rate, its analogue of λ_{ref}, against λ_{ref} (vLLM: fitted); the model's all-miss level μ_b (Proposition 2(a)) is 0.0718, 0.0737 and 0.0670 sessions/s. The peak ratio is taken against the measured λ_{ref} here ($\omega_{\text{ref}} = \mu^*/\lambda_{\text{ref}}$), as in the scoring rules, not against μ_b as in Proposition 3. The trapped rows come from the round-robin layer RR (§6.3). [§]Measured flat top: grid points within 5% of the measured maximum (the frozen scoring rule uses 2%, which gives 24 alone at 300k). [¶]Fluid curve, the frozen primary; F1 gives 30.5, 92.0 and 30.7. Pairs of values are for the two registered rates of each cell. [†]The F2 edge criterion was chosen knowing these two brackets (in-sample). [‖]Nominal multiples on the single seed-0 grid; with fresh seeds 2.5 λ_{ref} was stable in 3 of 3 runs, and over all 12 runs the edge in realized arrival rates lies between 2.665 and 2.726 λ_{ref} (§7). [‡]Pushed before the registered vLLM verdict was computed (see text); every other measured value was known to the agents that derived the model. vLLM trapped values are from the exploratory BURST runs, whose arrivals the registered BURST arm repeats.

	SGLang 300k pred / meas	SGLang native pred / meas	vLLM 300k pred / meas
cache-off $\mu(32)$ (turns)	0.0718 / 0.0715	0.0718 / 0.0718	fitted (0.0670 / 0.0657)
μ^* F1 (fluid)	0.234 (0.248) / 0.226	0.495 (0.508) / 0.466	0.235 (0.249) / 0.211
peak n F1 / flat top[§]	24 / 20–24	69.5 / 64–80	24 / 20–24
0.5-hit crossing n[¶]	31.5 / 32.2	93.7 / 93.4	31.8 / 29.1
peak ratio $\mu^*/\lambda_{\text{ref}}$ F1	3.27 / 3.15	6.89 / 6.48	3.58 / 3.21
from-empty bracket	$[2.0, 2.5]^{\dagger}$ / $[2.0, 2.5]^{\|}$	$[5.5, 6.5]^{\dagger}$ / [5.5, 6.5]	[2.5, 3.0] / [2.0, 2.5]
trapped backlog	215, 304 / 200–232, 283–332	565, 1089 / 557–601, 1098–1139	206, 288 / 203–227, 277–309
trapped token ratio	1.006, 1.016 / 1.009–1.025, 1.025–1.029	1.067, 1.079 / 1.074–1.078, 1.106–1.124	0.992, 1.002 / 0.997–1.007, 1.025–1.031
good-state hit	0.932 / 0.921–0.934	0.932 / 0.929–0.935	0.932[‡] / 0.930–0.933
capped drain (s)	1576, 1941 / 1547–1904, 2021–2693	493, 815 / 512–583, 837–937	1449, 1725[‡] / 1521–2027, 1848–2735

Table 4 gives its median in every arm of the three registered tests, computed by the script frozen for the vLLM mechanism checks. In every good-state run the ratio is 0.010–0.028: the gap is 4.9–5.9 s, mostly tool time, and R_M is 188–505 s. In every BURST run it is 4.1–9.7: R_M falls to 26–78 s because the trapped engine rewrites the pool at the all-miss rate, and the gap rises to 166–516 s, of which the mean tool time is 4.8–5.1 s and the rest is the turn's latency, mostly time in the queue (a turn's own service takes seconds). Turn latency, not tool time, carries the gap past R_M. The trapped value is close to the identity of Proposition 5, which holds exactly when every present session completes one turn per cycle of the FCFS queue: then gap/R_M equals the total next-turn context of the sessions present divided by M, and the round-robin layer RR, with nothing fitted to any trapped measurement (it uses the arrival rate, the burst and the server-log cost model), gives values within 4% of the measured means (Table 4). These are correlations across two states at one load; whether a temporary increase in tool latency, with no burst, can trigger the trap is the registered TOOLSLOW test of §7, which would not separate the gap from the capacity loss that the slowdown also causes. Section 9 places this measurement against EfficientAgent's backlogged-pool working set and CacheRoute's revisit criterion.

Table 4: The sustaining variable: median gap between a session's turns over the pool's turnover time $R_M = M/\text{churn}$, over the evaluation window, ranges over seeds (good arms: NOBURST and CAPPED). Gap: median, seconds. Model: Proposition 5 applied to the turn mix and backlog that the round-robin layer RR of the first-principles model predicts (§6.3; tag `r13-theory-pred`; for vLLM validated on the exploratory runs, whose arrivals the registered BURST arm repeats). Computed by `analysis/r11/loopvars_frozen.py` (commit `f1338e3`), unchanged.

		good arms		BURST		
test	$\lambda/\lambda_{\text{ref}}$	gap/R_M	R_M (s)	gap/R_M	gap (s)	model
SGLang 300k	1.35	0.016–0.020	255–307	6.35–6.92	166–178	6.93
	1.65	0.022–0.028	188–230	8.18–9.67	210–249	8.98
SGLang native	2.5	0.010–0.012	446–505	4.14–4.57	322–354	4.45
	4.0	0.018–0.022	272–310	6.68–6.96	504–516	7.06
vLLM 300k	1.35	0.014–0.017	286–337	6.44–6.79	183–196	6.64
	1.65	0.019–0.022	229–264	8.16–8.78	231–261	8.52

6.5 Validation

Table 3 compares the predictions with the three measured cells. Inputs and outputs are separated: for the SGLang cells the model receives M, the workload and the cost model and predicts everything else; for vLLM it also receives one speed factor fitted to vLLM's own all-miss token rate, so its λ_{miss} is not a prediction. The agents that derived the model knew every measured value in the table except the vLLM cells marked ‡, whose predictions were pushed at 02:05:30 CDT, three minutes before the registered vLLM verdict was computed and, by the log of the agent that operated the registered vLLM test (`notes/LOG.md`), before any of that test's run directories was opened; the theory's blindness declaration does not cover these runs (§5.2). On the turns-based estimator, which measures the model's $\nu/E[T]$, the F1 peak is 3.6% and 6.3% high on the SGLang cells and the fluid peak 9.9% and 9.0%. Assumption MF is the likeliest cause: the memory balance fixes the mean of $\mathcal{D}(R)$, and the hit

probability $F_Z \circ \mathcal{D}^{-1}$ is concave in it, since its derivative $f(R)/(A_1 P(Z > R) + E[E_T])$ decreases for exponential tool times (indeed for any non-increasing hazard), so by Jensen a deterministic R overstates hits near the peak. The vLLM cell is a partial miss: its cliff comes earlier and steeper than predicted. Figure 4 shows the curves. Out of sample (§7, Table 5), the frozen predictions held for the rescue verdicts, BASIN-1, the H200's from-empty edge (in nominal multiples) and its 0.5-hit crossing, and missed the H200 curve's tail past the peak and 4 of the 9 rescue release times; BASIN-2, which tests the critical burst, is pending.

6.6 Weaknesses

Every proposition is about the deterministic model; we have no limit theorem that takes the engine, or the simulator, to it. Assumption MF is a law of large numbers for many small objects, and our pools hold few: one session's mean footprint is 3.5% of a 300k pool, and the mean live demand of the predicted (F1) peak population is 85% of the pool on the A100 at 300k and on vLLM, 99% at 150k and 114% on the H200, where it already exceeds the pool. The frozen notes named this lack of concentration, worst on the H200, as the likeliest failure there and expected it to bias μ^* upward; the H200 peak was matched, but past the peak the measured hit fraction stays above the prediction, the opposite of the overstatement of hits near the peak that a deterministic R implies for these exponential tool times, and we have not traced this miss to a specific assumption (§7). The H200 predictions divide every A100 cost by one factor; uniform scaling predicts a 64-to-32-session ratio of all-miss token rates of 1.019, against 1.046 measured, and a variant that fits both gives the low end of the frozen band. On vLLM the cliff comes earlier and steeper than predicted (crossing at 29.1 against 31.8; $\mu(28)$ 0.123 against 0.225 sessions/s, turns-based), and its plateau below λ_{miss} is not reproduced. The heuristics have their own misses: F2 was pessimistic about the 300k from-empty edge on fresh seeds, having been chosen knowing the seed-0 bracket, whose unstable run had received 2.73 λ_{ref}; and F4, like the simulator, drains the native-pool gate too fast (R4 released 19–22% late). Unimodality, fold-freeness and the stability of the queue-free states are numerical; RR and F1–F4 are heuristics; choosing this model over a second derivation, and F2 over another edge criterion, were selections made on the three measured cells; and the cost model was fitted to the server logs of the 300k Stage C runs, so that comparison is out of sample in its target but not in its data source. Proposition 7 has only a proof sketch, and its criterion has not been evaluated for any measured tool-time law. The basin predictions rest on quantities near n_2, the critical fresh burst (53.4 sessions) and the stationary estimate $n_2 - n_1$ (21.3), which $\mu(n)$ does not determine, so BASIN-2 tests the heuristic F3, not a consequence of the propositions.

6.7 Relation to prior models

Yao et al. [97] give a mean-field (Che-type) LRU hit ratio for multi-turn sessions with exogenous gaps between turns and a unique fixed point, and note that end-to-end latency may change the observed gaps without modelling it; Eq. (3) reduces to theirs when nothing runs and nothing queues, and Proposition 4 shows that with exogenous gaps and little running KV the measured zeros cannot arise. Here the feedback runs through service time and reserved KV: misses starve decode under prefill priority, which swells running reservations and squeezes idle KV, turning their unique fixed point into a curve with a rise, a peak, a cliff at Eq. (4) and an all-miss branch on which the queue sets the reuse gap. MISA-T describes the eviction–cold-prefill–queue loop in words and derives a session cap of the form of the pool part of Eq. (4) [45], and AgentKV predicts the collapse from the same kind of capacity ratio [3]; neither gives equilibria, a rescue condition or a fold criterion, and a capacity ratio leaves out the sessions in tool calls, a term that grows with GPU speed (Proposition 2(c)). The closed curve is a Denning thrashing curve [26], and Corollary 1 is the classical decomposition argument [23] applied to a predicted $\mu(n)$. What we believe is new: a predicted thrashing curve for prefix-cached agent serving, the onset identity and its speed corollary, the $s\bar{z}$ law for the peak ratio, the exogenous-gap bound, a fold criterion (proof sketch), and the exact time-domain form of the trapped identity. Appendix F relates the model further to the Che approximation and its analyses, to CacheRoute, to classical couplings of cache loss and queueing and to metastable-failure models, and states the limits of our literature search.

7 Frozen predictions and their tests

The registered tests of §5.2 check rules written before them. For the later tests, predictions were also frozen: by the calibrated simulator (§4.4) and by the first-principles model of §6, under tag `r13-theory-pred` (commit `997d2c4`, pushed at 02:05:30 CDT on 2026-10-31) and `r13-theory-pred2` (pred2 below; commit `cfa7712`, pushed at 04:20:20 CDT), which calls the model's frozen functions unchanged through new drivers written for the later tests (`analysis/theory/pred2/`), with primaries, estimators, tolerances and refutation rules fixed in advance (`notes/THEORY_r13_scoring.md`). Each was committed under its tag before any of its results was read by the agents that made it; what each prediction is blind to differs and is stated with it. Table 5 summarizes the outcomes in hand, and Appendix H gives every prediction in full (Tables 17 and 18). Every outcome is reported against these rules, misses included.

H200. The model's only H200 inputs were that GPU's E0a measurements, through one speed factor, $s = 2.742$, fitted to its all-miss token rate (Proposition 2(c)). It predicted a peak ratio of 1.92 at $n \approx 32.5$ (against 3.27 predicted and 3.15 measured on the A100), a 0.5-hit crossing at 37.3 ± 3 sessions, an all-miss onset at 41.3 and a from-empty edge at 1.39 λ_{ref}. The engine gave 1.89 at $n = 32$ (turns-based; 1.99 in completed sessions), a crossing near 38.9 and, on the original seed-0 runs, an edge in $[1.3, 1.4)$ λ_{ref} in nominal multiples,

Table 5: Frozen predictions and the outcomes in this version. "Simulator": the calibrated simulator of §4.4; "theory": the model of §6. Blind in time: pushed before the runs were submitted or started; by access only: the runs had ended, or were in progress, at the push, and were first read after it. Appendix H gives every prediction with its scoring rule.

Test	Frozen prediction	Outcome
H200, 300k pool (a measurement, not registered)	Theory: peak ratio 1.92, 0.5-hit crossing 37.3 ± 3, all-miss onset 41.3, from-empty edge 1.39 λ_{ref}. Peak not blind to the first author; n = 36, 44, 52 blind in time; the other runs blind by access only	Peak ratio 1.89, crossing ≈ 38.9, edge in [1.3, 1.4) λ_{ref} nominal (contains 1.39, a hit by the frozen grid rule; in realized rates 1.496–1.503, so F2 is about 7% pessimistic, as on the A100); no refutation rule fires. Tail softer than predicted from n = 40 (first zero hits between n = 52 and 96), so the predicted onset (41.3) came early
Fresh-seed E0b, 300k (exploratory)	Simulator: stable in 6 of 6 (blind in time). Theory (F2, not declared for this test): unstable with probability 0.69 per run at 2.5 λ_{ref}	Stable in 6 of 6: the seed-0 bracket is not replicated. F2 pessimistic
BASIN-1, 1.65 λ_{ref} (registered)	Simulator: pass (blind in time). Theory: this outcome with probability 0.879 (blind by access only; the 25-session runs were in progress at the push)	PASS: 100 sessions trapped and 25 absorbed, 3 of 3 seeds each; simulator right in 6 of 6 runs
Rescue R1, 300k (registered)	Simulator: pass, per-seed release times. Theory: rescued, release at 3830 and 4360 s (±15%). Both blind in time	PASS, 6 of 6 rescued, no relapse. Releases within 600 s of the simulator's in 6 of 6 and within ±15% of the theory's in 5 of 6
Rescue R4, native pool (registered)	Simulator: pass, per-seed release times. Theory: rescued, release at 5040 s (±15%). Both blind in time	PASS, 3 of 3 rescued, no relapse. Releases 800–1750 s after the simulator's and 19–22% after the theory's, outside its ±15% (a miss or a refutation, depending on the frozen rule read; see text)
LPM-INF, 1.65 λ_{ref} (exploratory)	Simulator: no collapse on the seeds on which stock LPM collapsed (blind in time)	No collapse (token hit fraction 0.59 and 0.42); level missed (0.08–0.10 predicted); no recovery
Pending	H200 with a 600k pool; pools of 150k and 600k; Llama-3.1-8B; BASIN-2; SHED; TOOLSLOW	—

which contains 1.39 (a hit by the frozen grid rule), and no refutation rule fires. That grid received 7–8% more arrivals than nominal, and in realized rates the edge lies between 1.496 and 1.503 λ_{ref} (§5.5); on the realized-rate basis on which the A100 edge is scored below, F2 was pessimistic here too, by about 7%. The tail past the peak is softer than predicted (Fig. 4, right): from n = 40 the engine keeps more hits than the model (0.46, 0.32, 0.22 and 0.16 at n = 40, 44, 48 and 52, against 0.27, 0, 0 and 0), so the predicted onset comes early, and we have not traced this to a specific assumption (§6.6). Blindness differs by quantity: the agent that operated the H200 cell saw its closed-curve values for $n \leq 32$, which include the predicted peak, before the freeze and showed them to the first author, though not to the agents that derived the model, so the peak ratio is not blind to the first author; the runs at n = 36, 44 and 52 are blind in time, and the other runs and the original from-empty runs, which had ended before the freeze but were first read after it, are blind by access only.

Basin. For BASIN-1 the simulator predicted a pass and matched all six per-run calls, with hit fractions within 0.006 and backlog slopes within 0.0005 sessions/s of the engine's; its predictions were pushed 68 minutes before the jobs were submitted. The model gave the observed outcome the probability $1.000 \times 0.958^3 = 0.879$; none of its BASIN-1 predictions is blind in time: the 100-session runs had ended 0.2–1.6 h before the push, and the 25-session runs, submitted at 21:12, were 6230–7807 s into their 8700 s, past the burst and inside the scoring window, so all six are blind by access only. For BASIN-2 the model's per-seed probabilities of a stuck run are 0.14, 0.55 and 1.0 at 40, 50 and 60 sessions; conditioned on each seed's burst (pred2, pushed before the BASIN-2 jobs were submitted), the registered bracketing rule holds with probability 0.66.

Rescue. Both prediction sets were pushed before any rescue run started (the simulator's at 21:30:40 CDT on 2026-10-30, the model's at 02:05:30 on 2026-10-31; the first run started at 02:23:44), so all nine runs are blind in time to both. A short test run of the cap's client gate (a smoke job, not one of the nine registered runs) had ended before the model's push; the model made no prediction for it, and by their record the agents that derived the model had no Nova access and opened no rescue output before the push. The simulator predicted a pass in both cells (R1 rescued in 20 of 20 seeds at each rate, R4 in 16 of 20) and no relapse, and every pass/fail call held; its per-seed release times on the exact arrival plans were within 600 s of the engine's in 6 of 6 R1 runs (late by a median of 375 s), but in R4 the engine released 800–1750 s later than predicted, still inside the simulator's 20-seed band. The model predicted rescue in every cell, relapse probabilities of 0.002 and 0.025 for R1 and about 0 for R4, and releases at 3830, 4360 and 5040 s (±15%): R1 released within the tolerance in 5 of 6 runs, all late (+1.6% to +18.8%), and R4 outside it in 3 of 3, 19–22% late. The frozen record disagrees on what such a release means: the scoring declaration (`notes/THEORY_r13_scoring.md`) calls a release outside ±15% a miss and makes only NOT RESCUED refute, while the headline table of `notes/PREDICTIONS_r13_theory.md`, frozen with it, lists the release time as falsifying, and for R4 as the only falsifier. Under that table the R4 release prediction, and the R1 prediction at 1.35 λ_{ref} seed 0, are refuted. Both models drain

the native gate too fast; the model's stated weak point was R4's first phase, which needed the paused sessions to complete at no less than 51% of their fluid rate. Two choices were made before any rescue run with knowledge of the simulator's predictions: R4's engage time was moved from 2400 to 2100 s after the simulator had predicted 13 of 20 rescued at 2400 s, and the agents that froze the model's rescue predictions knew the simulator's.

From-empty edge with fresh seeds. The simulator predicted all six fresh-seed runs stable, blind in time, and was right; the frozen replication rule, which needed 2.5 λ_{ref} unstable in at least two seeds, is not met. The model's heuristic F2, applied to these runs, was pessimistic: its frozen probability that a run at 2.5 λ_{ref} is unstable is 0.69 per seed, which gives all three stable a probability of 0.03, and its edge, 2.34 λ_{ref}, lies below the realized-rate bracket. These values are in the frozen file but were not declared for this test, and pred2's table for it was computed after the runs had ended.

LPM-INF. The simulator predicted, 65 minutes before the jobs were submitted, that LPM without its fallback avoids the all-miss collapse on the two seeds on which stock LPM collapsed. It got the direction and missed the level (token hit fraction 0.08–0.10 predicted, 0.42–0.59 measured), as flagged in advance: it has no partial prefix hits.

Earlier rounds. The first three simulator rounds predicted the registered paired tests and the curves and edges behind them (Appendix C.1, Table 13). B3, B3n and B3v predicted that the paired tests would pass (B3n at its own test points, 6–16% above the registered rates), and they did, with trapped token rates and good-arm hit fractions close to the engine's. B4 and B4v got the peak's level within 9% and 2%; B4's cliff came early, mostly because it was predicted for a smaller pool, and B4v's late. B2 and B2v got the from-empty edge in nominal multiples, while the primary mode of B2n put the native edge at least one grid step too high. What each round is blind to differs (§4.4, Appendix C).

Pending. The outcomes of the H200 cell with a 600k pool, the pool sweep at 150k and 600k tokens (which tests the model's numerically predicted pool dependence of the peak ratio and the onset, Fig. 9(b), discriminates between F2 and a second edge criterion and, with the 600k H200 cell, tests Proposition 3 at a second pool), the Llama-3.1-8B cell, the second basin test, BASIN-2 (bursts of 40, 50 and 60 around the model's critical burst of 53.4), the SHED test (arrivals stopped for 600 or 3600 s, refusing about 58 or 348 sessions, against none for the cap) and the TOOLSLOW test (a temporary 10 or 30 s slowdown of tool calls, with no burst) are pending in this version; their frozen predictions are in Appendix H.

8 A diagnostic, and what it implies

Choosing the number of admitted jobs from the throughput curve is the classical selection of a multiprogramming level [29, 43, 82]; session-based admission, client-side concurrency limits and cold-start tests at maximum concurrency are established practice [11, 20, 68]; and capacity ratios and stack-distance estimators size admission or a KV tier, the latter for a target hit rate or when turn gaps are exogenous [3, 59, 83, 97]. What the window adds is that the from-empty edge, a finite-horizon stability limit in the sense of Kleinrock and Lam's first exit time [56], and not only μ^*, bounds the operating range; that a cap protects against history, not only throughput, and can undo it; and that the right cap moves with the tool-time law, the token profile, the pool and the GPU. Only the evaluation, the placement of the cap at the client and the placement condition below are ours.

The window of Eq. (2) comes from three measurements that an operator can make before deployment, on the engine, model, pool and workload that will be served. (1) *All-miss capacity*: run a saturated closed population with prefix caching disabled at two populations (for example 32 and 64 sessions), take the larger uncached prefill token rate, which is stable across seeds, and convert it to sessions by the mean work per session; if sessions share a prefix, measure with caching on and each turn's private context fresh instead. Under request-level FCFS this bounds the recovery threshold from deep congestion (Proposition 6, Appendix E). (2) *Closed-population curve*: measure $\mu(n)$ on a grid that crosses the cliff, and do not read μ_∞ off the largest n of a short run with a synchronized start (here $\mu(300)$ was 51% above λ_{ref}). (3) *From-empty edge*: run open arrivals from an empty engine at increasing rates over the horizon of interest, with more than one arrival seed or in realized rates (at 300k, 2.5 λ_{ref} was unstable on one seed and stable on three others). Appendix G gives the pitfalls of each measurement.

If $\mu^* > \lambda_{\text{miss}}$, then in the fluid model of §3.3, under request-level FCFS and the assumptions of Proposition 6, every operating point in $(\lambda_{\text{miss}}, \min(\mu^*, \lambda_{\text{e}}))$ is one sufficiently large burst away from the congested state unless admission is bounded; a steady-state load test there passes without testing recovery. On the engine a burst of 200 sessions, 6 to 29 times the 7–34 sessions present in the good state, left every registered BURST run at zero hits (15 of the 18 trapped by the registered rule), and at 1.65 λ_{ref} a burst of 100 also trapped the engine while one of 25 did not. Without a cap, Proposition 6 and the fluid model predict that recovering from deep congestion requires cutting the arrival rate below λ_{miss}, which was 31% of peak throughput at 300k tokens and 15% with the native pool; no completed real run has lowered the rate after a trap (the registered SHED test is pending). A cap does it at the offered load. Corollary 1 says that a cap drains a burst, or an established trap, if it lies inside (n_1, n_2), fastest where $\mu(c) - \lambda$ is largest, at the peak; on the engine it prevented the trap in all 12 registered SGLang CAPPED runs and undid an established one in 9 of 9 rescue runs (§5.3). A cap at the capacity boundary $N_{\text{lb}} \approx 28$, the size a capacity-ratio rule gives at 300k, would

sit on the steep side of the curve, within a few sessions of n_2, where the quasi-static model is least reliable; we take the smallest n within 5% of the peak, because the curve has one run per n, and check it with a burst test. We recommend a cap because it needs no engine change and is the one mechanism we tested on the engine, not because it dominates: in the earlier simulator, 20 s TTL pins with a pinned-first order reached a higher $\mu(300)$ than any static cap (Appendix I.2). A cap does not make the burst free: it moves the burst's cost to the client as admission delay and pauses, minutes per affected session, instead of turns that take several minutes for the rest of the run, and preventing the trap costs less than undoing it with the native pool. Whether the gate should queue or reject sessions is a deployment choice we did not evaluate.

The cap must be measured again when the workload or configuration changes. On the engine it moved from 20 to 64–80 with the native pool, stayed at 20–24 with vLLM, and on an H200 moved to 32 (the smallest-within-5% rule gives 28) while the window narrowed to $(1, 1.3)\,\lambda_{\text{ref}}$, too narrow for our registration rule to place a burst test. In the calibrated simulator it moved to 32 or 40 with tool-time laws from real agent traces and fell to 12 with mini-SWE-agent's turn and token counts, and a cap sized for one setting can fail in either direction: too small, it holds the engine below the offered load; too large, it sits on the collapsed side of the curve (Appendix I). In that simulator the paired test passed at all ten points of five tool-time laws and at five of six points with tool times from real agent traces, whose dispersion is far larger than ours, and in the earlier simulator pure LPM, PEEK's released scheduler and TTL pins with a pinned-first order raised $\mu(300)$, our proxy for the congested service rate, above λ_{miss}, while FCFS with or without queued-prefix protection did not (Appendix I). An online detector is a separate question: a hit-rate threshold (with high KV usage) is CONCUR's thrashing signal [18] and a confirmed decline below a learned reference hit rate is MISA-T's [45], hit rate is among Bronson et al.'s characteristic metrics [10], and learned early indicators have been proposed [66]. Our frozen version of the first alarmed after every vLLM burst but also twice during capped drains (§5.2), and with a shared system prompt a detector would have to count private-suffix reuse. We do not claim a detector. Here the three stages took 14–16 A100-hours per pool and engine, and a three-arm burst check about 7.5 more per rate and seed.

9 Related work

Thrashing, caching and metastability. The thrashing collapse and the working-set remedy are Denning's [26, 27], and Wilkes analysed the dynamics of paging [92]; the decomposition analysis of thrashing instability, its stochastic cusp [72, 98] and the choice of a multiprogramming level are classical (§2); LRU's failure on loops is classical [21, 84], as is its weakness on weak-locality patterns [50]. Metastable failures were named by Bronson et al. [10] and studied in production incidents and modelled by Huang et al. [46], whose experiments vary a trigger's magnitude or duration against otherwise matched runs and check whether the bad state persists after the trigger is removed (in runs of 500 s, triggered at 60 s, in their replicated-database retry experiments, and for an hour after the trigger in their look-aside cache experiments), the template of our paired test. They were analysed with queueing models, simulation and emulation by Isaacs et al., who also compute a basin for retry storms with a continuous-time Markov chain [48], and with continuous-time Markov chains by Alvaro et al. [2]; predicted with a queueing model for replicated storage by Habibi et al. [42]; attributed to destabilizing feedback cycles by Farahbakhsh et al. [35, 36]; avoided with bounded concurrency and load shedding in a database simulator by Charapko et al. [16]; and anticipated with learned indicators by Majumdar et al. [66]. An AI system's operating loop has been modelled with cusp bistability and hysteretic recovery, and in an instrumented pipeline experiment it collapses just below a spinodal computed from the measured service law and recovers with a backlog-delayed hysteresis [89]; there the feedback is uncertified output re-entering the load rather than cache loss, and load reduction alone need not restore operation, whereas our rescue keeps the offered load. AWS Lambda's container cache can fail to refill once it is empty; Brooker et al. also note that a drop in its hit rate raises latency and, by Little's law, the concurrency and working set the cache must hold, relate this to Denning's thrashing, and guard against it with a concurrency limit and cold-start tests at maximum concurrency [11]. Our trap is the case in which the pool is full and rewritten before reuse. Gibbens, Hunt and Kelly's networks with random alternative routing are a classical case in which a state-dependent priority rule, trunk reservation, removes a congested mode and its hysteresis [41]; their modes are both stable, whereas our congested regime is transient, and the closer analogue of our cap is multiprogramming-level load control (§2).

Queueing theory of LLM serving. Ao et al. give fluid-guided scheduling with threshold admission [4] and show that in a decode-only model, which abstracts away prefix reuse, the eviction-free fixed point can be unstable, with rate-limited admission and length mixing as remedies; its benchmark GPU experiments disable prefix caching [5]. In its open-arrival simulations, every load tested between the worst-cycle throughput, the analogue of our λ_{miss}, and the eviction-free capacity accumulated backlog. On our engine, backlog accumulated from an empty start only above the from-empty edge; inside our window the good state persisted from empty for the length of each run, and a burst moved the system out of it. Dai et al. prove throughput optimality of work-conserving policies without cache state, and give a cyclic-routing example where work-conserving scheduling fails [24]; Nie et al. give stability conditions under KV memory constraints for independent one-shot requests whose KV cache is freed at completion [73]; Dong and Cao study flow-controlled admission [33]. Dexter et al. analyse FCFS and longest-prefix-match scheduling under prefix reuse and latency constraints [30]. Yao et al. derive a mean-field LRU hit ratio for multi-turn conversations with exogenous gaps between turns, and note that end-to-end la-

tency may change the observed gaps [97]; §6.7 relates our model to theirs and to the Che approximation, whose criterion CacheRoute states for LLM prefix caches and whose single-characteristic-time form it finds misses a 70B engine's hit rate [19]. In operations research, service systems whose service slows in congestion, or whose customers return, show bistability and are protected by admission control [15, 32, 47]; a session that returns after a tool call is such a returning customer, but there the slowdown is a given function of occupancy.

Prefix and KV mechanisms for agents. PEEK protects prefixes needed by queued requests and adds a cache-aware scheduler [95]; it documents SGLang's 128-request LPM fallback and runs its heavy-load cells at that depth, and its "scheduling-induced thrashing" is LPM breaking ties by arrival order. Continuum keeps a program's KV across a tool call for a time-to-live chosen by a cost model, which may be zero, and reports mean tool times of 925 ms on SWE-Bench (standard deviation 3550 ms) and 1923 ms on BFCL [58, 78]. Sutradhara observes, under Poisson load on vLLM, that prefix hit rates collapse despite context reuse, and attributes this to LRU eviction of the contexts of requests waiting on tool calls [9]. CONCUR controls the number of active agents in closed batch inference with AIMD, halving its window, to exit the thrashing regime, when KV usage exceeds 0.5 and the hit rate is below 0.2, pausing agents at generation and tool boundaries, and reports SGLang's hit rate falling from 77.7% at batch 32 to 35.4% at 40 [18]; MARS sets an admission window by AIMD from GPU KV pressure and CPU tool pressure, and reports that under heavier load its baselines' SLO-aware goodput collapses to near zero while their token throughput stays high [91]. On vLLM, Yuan et al. report the prefix-cache hit rate falling from 98.85% to 16.78% as the maximum number of concurrent Terminal-Bench agents goes from 15 to 20 [102], and VAMP calls the sharp rise in time to first token once reusable prefixes outgrow the KV pool the prefix-cache cliff, and measures stock vLLM at a 0.19% prefix hit rate on a recorded SWE-bench agent replay [55]. ThunderAgent describes agent collapse as working-set thrashing and pauses whole programs when their KV demand, with programs in tool calls counted at a weight that decays with tool time, exceeds the capacity; it reports that baseline systems collapse in throughput as the number of parallel workflows grows, a closed population [53], and NVIDIA Dynamo's port of its scheduler aims to avoid thrashing when memory cannot hold every agent's working set [31]. EfficientAgent applies Denning's working-set principle and load control to the host KV tier of a fixed pool of agents; its abstract describes state written to the host tier and evicted before reuse by other agents' traffic during tool waits, and it reports that waiting behind other agents lengthens the gap between an agent's turns [83]. AgentKV, from its abstract, predicts the onset of collapse at high concurrency from available KV capacity over per-instance context demand and prevents it by predictive admission [3]. MISA-T, for RL rollouts under a concurrency ceiling, reports vLLM Router's prefix-cache hit rate falling from 92.4% to 4.5% at a high ceiling, describes the positive feedback behind it, a loop between eviction, cold prefill, queue length and session overlap, notes that at prefill saturation the request rate is bounded by prefill throughput over the mean uncached tokens per request, and interrupts the loop with an adaptive admission cap for new sessions, derived from observed KV demand and overload pressure and contracted temporarily on a confirmed hit-rate decline [45]; it has no open arrivals, bursts or recovery test. KAIROS manages GPU frequency and admission for agentic serving under open arrivals on vLLM (Poisson in its characterization, Sec. 3.1; a replayed recorded agent workload in its evaluation, Sec. 7) and reports thrashing at a fixed lowered maximum clock (its Sec. 3.4 and Fig. 5: 660 against 1680 MHz at 0.08 agents/s), when its controller lowers the clock (its Figs. 10–11), and when the arrival rate rises (its App. A.4: 0.6 against 0.5 jobs/s), and avoids it by bounding the aggregate context of admitted agents (its Eq. 13: at most 0.95 of capacity, readmitting below 0.9) [101]. Its design section (Sec. 6.2.4) describes thrashing as a feedback loop in which throughput drops, requests accumulate and resident context grows, and states that consequently even a short overload can push the instance into a persistently inefficient operating region; it gives this as the motivation for its concurrency bound and does not test it in a controlled experiment. In its Figs. 10–11, at 0.05 jobs/s, the run without thrashing avoidance stays in what KAIROS calls a persistent thrashing regime, completing 0.046 jobs/s, while a baseline at the maximum clock carries the same rate. Its SLO-aware boost (its Eq. 12) restores the maximum clock when an agent falls below its throughput target, but no frequency trace is shown, so whether the trigger was removed is not reported; its three configurations share one replayed workload, but there is no burst trigger and so no matched no-burst arm, no fixed clock and one run per configuration, and when bursty arrivals have already broken its bound KAIROS defers new agents and boosts the clock but reports no recovery from an established trap. Many more systems offload, prefetch, retain, schedule or evict agents' KV, or simulate agent serving; Appendix G lists them.

Position. The loop is published under a concurrency ceiling or for a closed population (MISA-T, CONCUR, Yuan et al., VAMP, ThunderAgent [18, 45, 53, 55, 102]), at fixed concurrency (AgentKV, from its abstract [3]), and under open arrivals, where KAIROS reports it after a lowered clock or a higher arrival rate, asserts (its Sec. 6.2.4) that a short overload can leave an instance persistently inefficient, and shows a persistent thrashing regime in one run without its thrashing avoidance and with no burst trigger, whose clock during the thrashing is not reported [101]. Session-aware admission, pausing at tool boundaries and hit-rate thrashing signals are implemented or proposed [18, 31, 45, 51, 53, 60, 91, 101], to prevent the trap or, in closed populations, to leave it by lowering the number of active agents, which lowers the load itself; where these controllers use two thresholds (ThunderAgent's watermarks, Dynamo's pause and resume levels, KAIROS's 0.95 and 0.9), the thresholds form a controller's dead band; and shedding load is the usual response in production [46]. We

claim none of these. We add (i) a controlled, replicated, registered test of KAIROS's open-system assertion at a fixed clock and pool, following the trigger design of Huang et al. [46]: two persistent states at one capacity and offered load, entered by a transient burst, against matched no-burst and capped arms on common random numbers, on two pools and two engines; (ii) the rescue of an established open-system trap at the same offered load without shedding, and the persistence of the good state after the cap is removed, hysteresis of the system rather than of a controller, which Carleial and Hellman proposed for bistable channels but did not evaluate [14]; (iii) a first-principles thrashing curve in which hit-dependent running reservations produce the cliff and the queue sets the reuse gap on the all-miss branch, with the measured sustaining variable in both states at one load, a cost-model-free identity for its trapped value (the exact time-domain form of EfficientAgent's backlogged-pool cycle argument [83]; CacheRoute states the underlying revisit criterion for prefix caches whose revisit times are set by routing [19]), a bound showing that exogenous-gap models without running reservations cannot reach the measured zero hits, and out-of-sample predictions frozen before the H200, basin and rescue results were read by the agents that derived it (the first author had seen the H200 closed-curve values for $n \leq 32$, which include the predicted peak); and (iv) a measured basin boundary. Against the look-aside cases of the metastability literature, where a cache that has lost its contents fails to refill [10, 36, 46], our pool is full and rewritten before reuse, an instance of what Huang et al. call capacity-degradation amplification. Metastability from a unimodal throughput curve, and bounded admission as its cure, are classical (§2). Ao et al.'s decode-only model abstracts away prefix reuse; extending Yao et al.'s model so that the KV held by running requests, and their service time, depend on the hit rate covers our setting, and §6 does this.

10 Limitations and threats to validity

Scope of the registered evidence. All registered SGLang evidence is for SGLang 0.5.22, Qwen2.5-7B-Instruct and one A100, at two pool sizes: the 300k-token pool limit and the 955,757-token pool SGLang sizes itself, with which the cliff moved out and the registered tests passed too, so the pool limit did not create the cliff. vLLM 0.32.0 ran only with the 300k-token pool, and its paired test is registered, not pre-registered, with a BURST arm that repeats the workloads of earlier exploratory runs (§5.2). The H200 cell is a measurement, not a registered test, and compares GPU and default attention backend together. The rescue tests engage the registered cap at a fixed time, not on an online signal, with three seeds per cell; they test persistence once the cap no longer binds, with no perturbation after release, and their cost and persistence comparisons are descriptive. The 600k-pool H200, Llama-3.1-8B, pool-sweep, BASIN-2, shedding and tool-slowdown tests have pending outcomes in this version. Larger models, parallelism, offloading and GPUs other than the A100 and the H200 are untested.

Synthetic sessions and clients. Sessions are synthetic (random tokens, fixed tool outputs and output lengths, geometric turn counts, exponential 5 s tools) and share no prefixes, which removes the case PEEK's scheduler targets. Tool times in real agent traces have medians 15–33 times shorter than ours and coefficients of variation of 4.3–4.7 against 1, and outputs per call 2.7–5.9 times our 100 tokens; in the simulator such tool-time laws keep the trap but move the window and the cap, which the engine has not tested. With a shared system prompt the ratio of all-miss to all-hit prompt work, about 16, would barely change, but the hit fraction of a trapped engine would stay near the shared share, so hit-based signals would have to count private-suffix reuse. Our clients never retry or abandon a request; retries would add load in the trapped state and abandonment would remove it, so either could change whether the engine recovers. Appendix G gives the trace statistics and the other details.

Few runs, finite horizons, nominal rates. The closed curves and the from-empty sweeps have one run per point, so the peak's location is within noise and single runs near the cliff land on either side of it. The registered tests, the grid, the vLLM runs and the rescue tests share the same three seeds, so their runs are not independent across rates, pools, engines or tests. All rates are nominal; each from-empty sweep used a single arrival seed, and at 300k the seed-0 bracket was not replicated on fresh seeds, although stability is monotone in the realized rate, with the edge between 2.665 and 2.726 λ_{ref}; on the H200 it is likewise monotone over ten runs, with the edge between 1.496 and 1.503 λ_{ref} (the 1.503 run ended just above the queue threshold). "Does not leave" means within 7500 s on the engine and 30,000 s in the simulator, and the 3000 s horizon of the from-empty runs cannot see slow escapes. The registered rule loses margin at both ends of the window: the slope test near λ_{miss}, and the token test at high multiples. Appendix G discusses the native cap, the coincidence of the cliff with a change of job and node in both SGLang curves, and other uncontrolled factors.

Models and simulators. The fluid model is used for the window, the recovery threshold and drain times, and through the heuristics F2–F4 for the from-empty edge, burst sizes and release times; it is not quantitative in congestion and is least reliable near the cliff, and the frozen SHED and TOOLSLOW predictions use it there. Proposition 6 is a limit statement under stated assumptions, including private prefixes, no offloading, i.i.d. tool times unknown to the engine and an engine that in congestion computes prompt tokens no faster than with caching disabled, which our engine meets only to within a few percent at the stationary turn mix and within 12% over the windows we measured; that recovery without a cap needs a rate below λ_{miss} is untested on the engine, pure LPM's congested rate is a conjecture, and the capped system's stochastic stability is not proved. The first-principles model's propositions hold for a deterministic fluid model with no limit theorem to the engine; it misses the H200's tail past the peak and vLLM's earlier cliff, its native-pool release times came

late, and its derivation and edge criterion were selected on the measured cells (§6.6). The calibrated simulator failed its level test at one cell, omits partial prefixes and the waiting-prefix refresh, frees a finished session's KV at once and understated LPM; the mechanism comparison uses an older simulator with assumed costs, configured pins, ablations and a surrogate of CONCUR, so its rates do not carry over to the engine or to the named systems (Appendix I).

The registration record. Appendix C lists the slips: the vLLM registration reached GitHub after two of its jobs had started; some prediction rounds were pushed after their jobs were submitted, or specified after related data were known; the H200 peak values were seen by the first author before the freeze; some of the model's predictions are blind by access only; and R4's engage time and the model's rescue predictions were chosen or frozen with knowledge of the simulator's predictions. No registration or prediction is in a third-party registry, and none has a timestamp other than GitHub's push log of a private repository, which readers can see only in our export.

11 Conclusion

Prefix caching turns multi-turn agent serving into a thrashing workload: the cache that makes follow-up turns cheap belongs to sessions idle in tool calls or waiting in the queue, and admission reclaims it. On SGLang with two pool sizes and on vLLM, at rates the engine carried in the good state for the full length of every no-burst run, a burst of 200 sessions moved the engine into an all-miss state that it did not leave within 7500 s, while a client-side cap at the peak of $\mu(n)$ drained the same burst; two of these tests were pre-registered and the third registered, and all three pass by their registered rule, although on vLLM the frozen decision rule for the mechanism checks is not met. The same cap, engaged inside an established trap, returned the engine to the good state in 9 of 9 registered runs without shedding a session, and with the cap removed, once the client queues had stayed empty for 300 s, the good state persisted at the same load while the uncapped controls stayed at zero hits: which state the engine is in depends on its history, not only on its load. Bursts of 25 sessions were absorbed and bursts of 100 trapped the engine, so the good state has a finite basin. A first-principles model in which misses swell running reservations and squeeze idle KV, and the queue sets the gap between a session's turns once every turn misses, with no parameter fitted to any $\mu(n)$, hit or trap measurement, reproduces the SGLang curves (in sample) and the trapped states, shows why exogenous-gap cache models without running KV cannot reach the measured zero hits, and predicted, before the results were read, the rescue, the 25/100-session basin test and, in nominal multiples, an H200's from-empty edge, though not that GPU's softer tail past the peak or 4 of the 9 rescue release times. On the H200 the peak ratio fell from 3.15 to 1.89, in the direction of the model's speed law, and the from-empty edge, which sets the window's upper end on both GPUs, fell to 1.3 λ_{ref} in nominal multiples, as the heuristic F2 predicted (1.39). These results are for synthetic sessions with exponential 5 s tool times and no shared prefixes, and several frozen tests are pending in this version. Operators should measure $\mu(n)$ for the workload they serve, cap admitted sessions at the smallest population within a few percent of its peak, accepting that a burst then waits at the client, and re-measure when the workload, pool, engine or GPU changes; a cap engaged after a collapse still restores the good state, but preventing the collapse costs no more client delay, and much less with the native pool.

Code and data availability

The public repository [pending: URL of the public repository] holds the code (simulator, load generator, analysis, the first-principles model of §6 and the Slurm job scripts), the experiment configurations, every registration file (`configs/agentload_prereg_*.yaml`), the frozen predictions with their tags (`notes/PREDICTIONS*.md`, `configs/r1*_predict.yaml`, `configs/r13_theory_pred2.yaml`, `results/theory/merged/`, `results/theory/pred2/`), all real-engine data with the job logs (`results/nova/`), the research log (`notes/LOG.md`), the scripts behind every figure and derived table, and the source of every number in the paper (`paper/NUMBERS.md`); this version corresponds to tag `arxiv-v1`. Its history is a redacted copy of the authors' development repository, whose commit hashes this paper cites; `provenance/` maps them to their public counterparts and holds the timing evidence, the hash-chain proofs of the registrations and predictions, and the cluster's accounting records (Appendix D). The development repository is private; its GitHub push log, the record behind our pre-registration claims, is available to readers only as our export of it in `provenance/`.

Use of AI tools

LLM coding agents (Anthropic Claude through Claude Code, including Claude Sonnet subagents, and OpenAI Codex sessions) expedited this work under the authors' direction. They wrote and tested much of the simulator, load generator and analysis code; prepared and submitted the cluster jobs, each experiment cell operated by its own Claude Code agent and the cells coordinated by a further agent that also drafted the text, under authorizations the first author gave for each experiment or set of experiments; made the blind simulator predictions (§4.4); derived the first-principles model of §6, wrote its code and proofs, and froze its predictions; specified the prediction rounds and, afterwards, checked the first round's tool-call transcripts; proposed the registration rules and drafted and filled every registration file; designed most of the later experiments (the basin, rescue and LPM-INF tests, BASIN-2, TOOLSLOW, SHED and the pool-size sweep), which the first author authorized as a set rather than one by one; made, in

a separate Codex session, the independent verification of the first round, which reused the prediction agents' copy of the repository rather than a fresh one; ran adversarial reviews of the claims, the proofs and their novelty against the literature; audited the citations; read the engine sources, partly through a web tool that returns a language model's summary of each page; and drafted and revised the text. The authors chose the research question, chose and approved the early experiments one at a time (Stage C, E2, the native-pool and vLLM tests), authorized the later ones, reviewed the results, and read and checked the manuscript, including the proofs, before posting; they are the final decision makers on its content and take full responsibility for it. Every commit, including those the agents made, is recorded under the first author's identity; a history rewrite on 2026-10-22 removed the agents' co-author trailers from commit messages and changed nothing else (`provenance/commit-map.tsv` maps old hashes to new ones).

Acknowledgments

The experiments ran on the Nova cluster at Iowa State University. The research reported in this paper is partially supported by the HPC@ISU equipment at Iowa State University, some of which has been purchased through funding provided by NSF under MRI grants number 1726447 and MRI2018594.

References

[1] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiying Zhang. InferCept: Efficient intercept support for augmented large language model inference. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 81–95, 2024.

[2] Peter Alvaro, Rebecca Isaacs, Rupak Majumdar, Kiran-Kumar Muniswamy-Reddy, Mahmoud Salamati, and Sadegh Soudjani. Formal analysis of metastable failures in software systems. arXiv:2510.03551, 2025.

[3] Anonymous. AgentKV: Predictive admission prevents KV-cache collapse in agentic LLM serving. ICLR 2027 submission, `https://openreview.net/forum?id=zLtopd1ATE`, 2026.

[4] Ruicheng Ao, Gan Luo, David Simchi-Levi, and Xinshang Wang. Optimizing LLM inference: Fluid-guided online scheduling with memory constraints. arXiv:2504.11320 (v4, 13 June 2026), 2025.

[5] Ruicheng Ao, Jing Dong, Gan Luo, and David Simchi-Levi. Service-induced congestion in memory-constrained LLM serving. arXiv:2606.15555 (v3, 20 October 2026), 2026.

[6] Mayank Arya, Agrim Bari, Anjaly Parayil, Yogesh Simmhan, Renee St. Amant, Victor Rühle, and Saravan Rajmohan. Understanding KV residency in production-scale agentic LLM workloads. arXiv:2610.22863 (v1, 16 October 2026), 2026.

[7] L. A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2): 78–101, 1966.

[8] Zhuohang Bian, Feiyang Wu, Zhuoran Li, Teng Ma, and Youwei Zhuo. TokenCake: A KV-cache-centric serving framework for LLM-based multi-agent applications. arXiv:2510.18586; to appear at EuroSys '27, 2025.

[9] Anish Biswas, Kanishk Goel, Srivarshinee S, Jayashree Mohan, Alind Khare, Anjaly Parayil, Ramachandran Ramjee, and Chetan Bansal. Sutradhara: An intelligent orchestrator-engine co-design for tool-based agentic inference. arXiv:2601.12967 (v3, 22 April 2026), 2026.

[10] Nathan Bronson, Abutalib Aghayev, Aleksey Charapko, and Timothy Zhu. Metastable failures in distributed systems. In *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS '21)*, Ann Arbor, MI, 2021. doi: 10.1145/3458336.3465286.

[11] Marc Brooker, Mike Danilov, Chris Greenwood, and Phil Piwonka. On-demand container loading in AWS Lambda. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 315–328, 2023. arXiv:2305.13162 (v2).

[12] Shelby L. Brumelle. On the relation between customer and time averages in queues. *Journal of Applied Probability*, 8(3):508–520, 1971. doi: 10.2307/3212174.

[13] Shiyi Cao, Yichuan Wang, Ziming Mao, Pin-Lun Hsu, Liangsheng Yin, Tian Xia, Dacheng Li, Shu Liu, Yineng Zhang, Yang Zhou, Ying Sheng, Joseph Gonzalez, and Ion Stoica. Locality-aware fair scheduling in LLM serving. arXiv:2501.14312, 2025.

[14] A. Carleial and M. Hellman. Bistable behavior of ALOHA-type systems. *IEEE Transactions on Communications*, 23(4):401–410, 1975. doi: 10.1109/TCOM.1975.1092823.

[15] Carri W. Chan, Galit Yom-Tov, and Gabriel Escobar. When to use speedup: An examination of service systems with returns. *Operations Research*, 62(2): 462–482, 2014. doi: 10.1287/opre.2014.1258.

[16] Aleksey Charapko, Murat Demirbas, Matt Broadstone, Daniel Gomez Ferro, and Akshat Vig.

Towards designing an execution control system with metastability resilience. In *Proceedings of the 35th International Conference on Computer Communications and Networks (ICCCN)*, pages 1–9, 2026. doi: 10.1109/ICCCN69946.2026.11662935.

[17] Hao Che, Ye Tung, and Zhijun Wang. Hierarchical Web caching systems: Modeling, design and experimental results. *IEEE Journal on Selected Areas in Communications*, 20(7):1305–1314, 2002. doi: 10.1109/JSAC.2002.801752.

[18] Qiaoling Chen, Zhisheng Ye, Tian Tang, Peng Sun, Boyu Tian, Guoteng Wang, Shenggui Li, Yonggang Wen, Zhenhua Han, and Tianwei Zhang. CONCUR: High-throughput agentic batch inference of LLM via congestion-based concurrency control. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 17881–17891, 2026. arXiv:2601.22705.

[19] Huang Cheng. CacheRoute: Planned prefix-affinity routing for large-scale LLM serving. arXiv:2608.19677 (v1, 20 August 2026), 2026.

[20] L. Cherkasova and P. Phaal. Session-based admission control: A mechanism for peak load management of commercial web sites. *IEEE Transactions on Computers*, 51(6):669–685, 2002. doi: 10.1109/TC.2002.1009151.

[21] Hong-Tai Chou and David J. DeWitt. An evaluation of buffer management strategies for relational database systems. In *Proceedings of VLDB 85*, pages 127–141, Stockholm, 1985.

[22] Neil Chowdhury, James Aung, Jun Shern Chan, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubeh, Mia Glaese, Carlos E. Jimenez, John Yang, Leyton Ho, Tejal Patwardhan, Kevin Liu, and Aleksander Madry. Introducing SWE-bench Verified. OpenAI blog post, 13 August 2024 (updated 24 February 2025); `https://openai.com/index/introducing-swe-bench-verified/`, 2024.

[23] P. J. Courtois. Decomposability, instabilities, and saturation in multiprogramming systems. *Communications of the ACM*, 18(7):371–377, 1975. doi: 10.1145/360881.360887.

[24] J. G. Dai, Tianze Deng, Yueying Li, and Tianyi Peng. Throughput-optimal scheduling algorithms for LLM inference and AI agents. arXiv:2504.07347 (v3, 18 May 2026), 2025.

[25] Asit Dan, Daniel M. Dias, and Philip S. Yu. The effect of skewed data access on buffer hits and data contention in a data sharing environment. In *Proceedings of the 16th International Conference on Very Large Data Bases (VLDB)*, pages 419–431, Brisbane, Australia, 1990.

[26] Peter J. Denning. Thrashing: Its causes and prevention. In *Proceedings of the AFIPS Fall Joint Computer Conference*, pages 915–922, 1968.

[27] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5): 323–333, 1968. doi: 10.1145/363095.363141.

[28] Peter J. Denning. The locality principle. *Communications of the ACM*, 48(7):19–24, 2005. doi: 10.1145/1070838.1070856.

[29] Peter J. Denning, Kevin C. Kahn, Jacques Leroudier, Dominique Potier, and Rajan Suri. Optimal multiprogramming. *Acta Informatica*, 7(2):197–216, 1976. doi: 10.1007/BF00265771.

[30] Gregory Dexter, Shao Tang, Ata Fatahi Baarzi, Qingquan Song, Tejas Dharamsi, and Aman Gupta. LLM query scheduling with prefix reuse and latency constraints. arXiv:2502.04677 (v3; v1 7 February 2025), 2025.

[31] Ishan Dhanani, Jamie Li, and Karen Chung. Session-aware agentic inference with NVIDIA Dynamo. PyTorch Blog, 8 October 2026; `https://pytorch.org/blog/session-aware-agentic-inference-with-nvidia-dynamo/`, 2026.

[32] Jing Dong, Pnina Feldman, and Galit B. Yom-Tov. Service systems with slowdowns: Potential failures and proposed solutions. *Operations Research*, 63(2): 305–324, 2015. doi: 10.1287/opre.2015.1346.

[33] Zhuolun Dong and Junyu Cao. Flow-controlled scheduling for LLM inference with provable stability guarantees. arXiv:2604.11001, 2026.

[34] Ronald Fagin. Asymptotic miss ratios over independent references. *Journal of Computer and System Sciences*, 14(2):222–250, 1977. doi: 10.1016/S0022-0000(77)80014-7.

[35] Ali Farahbakhsh, Andreas Haeberlen, Qingjie Lu, Lorenzo Alvisi, Robbert van Renesse, and Shir Cohen. Modeling metastability. In *Proceedings of the 24th ACM Workshop on Hot Topics in Networks (HotNets '25)*, pages 19–24, College Park, MD, USA, 2025. ACM. doi: 10.1145/3772356.3772426.

[36] Ali Farahbakhsh, Qingjie Lu, Lorenzo Alvisi, Andreas Haeberlen, and Robbert van Renesse. Characterizing metastable faults and failures. arXiv:2606.00942 (v2, 2 June 2026), 2026.

[37] Guy Fayolle, Erol Gelenbe, and Jacques Labetoulle. Stability and optimal control of the packet switching broadcast channel. *Journal of the ACM*, 24(3): 375–386, 1977. doi: 10.1145/322017.322019.

[38] Christine Fricker, Philippe Robert, and James Roberts. A versatile and accurate approximation for LRU cache performance. 24th International Teletraffic Congress (ITC 24); arXiv:1202.3974 (v1, 17 February 2012), 2012.

[39] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 111–126, Santa Clara, CA, 2024. USENIX Association. arXiv:2403.19708.

[40] Nicolas Gast and Benny Van Houdt. Transient and steady-state regime of a family of list-based cache replacement algorithms. In *Proceedings of the 2015 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, pages 123–136, 2015. doi: 10.1145/2745844.2745850.

[41] R. J. Gibbens, P. J. Hunt, and F. P. Kelly. Bistability in communication networks. In G. R. Grimmett and D. J. A. Welsh, editors, *Disorder in Physical Systems*, pages 113–127. Oxford University Press, 1990.

[42] Farzad Habibi, Tania Lorido-Botran, Ahmad Showail, Daniel C. Sturman, and Faisal Nawab. MSF-Model: Queuing-based analysis and prediction of metastable failures in replicated storage systems. In *Proceedings of the 43rd International Symposium on Reliable Distributed Systems (SRDS)*, pages 12–22, Charlotte, NC, USA, 2024. IEEE. doi: 10.1109/SRDS64841.2024.00013.

[43] Hans-Ulrich Heiss and Roger Wagner. Adaptive load control in transaction processing systems. In *Proceedings of the 17th International Conference on Very Large Data Bases (VLDB '91)*, pages 47–54, Barcelona, 1991. Morgan Kaufmann.

[44] Daniel P. Heyman and Shaler Stidham, Jr. The relation between customer and time averages in queues. *Operations Research*, 28(4):983–994, 1980. doi: 10.1287/opre.28.4.983.

[45] Zetao Hong, Song Yuan, Yuanhao Ding, Yibo Zhu, Daxin Jiang, Zhibin Wang, and Chen Tian. Scheduling mixed RL rollouts beyond prefix locality. arXiv:2608.11152 (v1, 11 August 2026), 2026.

[46] Lexiang Huang, Matthew Magnusson, Abishek Bangalore Muralikrishna, Salman Estyak, Rebecca Isaacs, Abutalib Aghayev, Timothy Zhu, and Aleksey Charapko. Metastable failures in the wild. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 73–90, 2022.

[47] Armann Ingolfsson, Eman Almehdawe, Ali Pedram, and Monica Tran. Comparison of fluid approximations for service systems with state-dependent service rates and return probabilities. *European Journal of Operational Research*, 283(2): 562–575, 2020. doi: 10.1016/j.ejor.2019.11.041.

[48] Rebecca Isaacs, Peter Alvaro, Rupak Majumdar, Kiran-Kumar Muniswamy-Reddy, Mahmoud Salamati, and Sadegh Soudjani. Analyzing metastable failures. In *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS '25)*, pages 172–178, Banff, AB, Canada, 2025. ACM. doi: 10.1145/3713082.3730380.

[49] Van Jacobson. Congestion avoidance and control. In *Symposium Proceedings on Communications Architectures and Protocols (ACM SIGCOMM '88)*, pages 314–329, 1988. doi: 10.1145/52324.52356.

[50] Song Jiang and Xiaodong Zhang. LIRS: An efficient low inter-reference recency set replacement policy to improve buffer cache performance. *ACM SIGMETRICS Performance Evaluation Review*, 30(1): 31–42, 2002. doi: 10.1145/511399.511340.

[51] Youhe Jiang, Fangcheng Fu, Binhang Yuan, Krishna Malladi, Ehsan K. Ardestani, Zhan Shu, Adnan Aziz, and Yi Xu. Coda: Exploiting admission flexibility for coding-agent serving. arXiv:2610.03088 (v1, 2 October 2026), 2026.

[52] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR 2024)*, 2024.

[53] Hao Kang, Ziyang Li, Weili Xu, Xinyu Yang, Yinfang Chen, Junxiong Wang, Beidi Chen, Tushar Krishna, Chenfeng Xu, and Simran Arora. ThunderAgent: A fast, simple, and program-aware agentic inference system. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 55630–55657, 2026. arXiv:2602.13692.

[54] Samuel Karlin and Howard M. Taylor. *A Second Course in Stochastic Processes*. Academic Press, New York, 1981. ISBN 978-0-12-398650-4.

[55] Jinpyo Kim, Mingi Kwon, Younghoon Min, Jongryool Kim, and Jishen Zhao. Dynamic HBM repartitioning

for multi-turn MoE serving. arXiv:2609.13537 (v1, 11 September 2026), 2026.

[56] Leonard Kleinrock and Simon S. Lam. Packet switching in a multiaccess broadcast channel: Performance evaluation. *IEEE Transactions on Communications*, 23(4):410–423, 1975. doi: 10.1109/TCOM.1975.1092814.

[57] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, pages 611–626. ACM, 2023. doi: 10.1145/3600006.3613165.

[58] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Huanchen Zhang, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live. arXiv:2511.02230 (v7, 8 September 2026), 2025.

[59] Luchang Li, Shuaishuai Wang, Zhao Ruan, Dongfang Li, and Bozhao Gong. The KV cache working set: Online capacity planning for LLM inference systems. arXiv:2609.27746 (v1, 23 September 2026), 2026.

[60] Shuhuai Li, Shengdong Ye, Dongdong Ge, Yinyu Ye, and Jianghao Lin. TAO: Tool-aware joint scheduling for LLM-based agentic serving. arXiv:2610.37230 (v1, 27 October 2026), 2026.

[61] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of LLM-based applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 929–945, Santa Clara, CA, 2024. USENIX Association. arXiv:2405.19888.

[62] Banruo Liu, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Ricardo Bianchini, and Esha Choukse. Agentic coding in the wild: Characterizing GitHub Copilot traces at production scale. arXiv:2608.00101 (v1, 30 July 2026), 2026.

[63] Yipeng Liu, Yingqiang Zhang, Feifei Li, and Huanchen Zhang. Ask the tool, don't guess: Agent tool calls hold their progress, and the serving system should read it. arXiv:2609.18849, 2026.

[64] Yiyu Liu, Minlan Yu, and Juncheng Yang. When fancy eviction fails: Rethinking cache replacement for LLM prefix reuse. arXiv:2609.28870 (v2, 1 October 2026), 2026.

[65] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. Agentix: An efficient serving engine for LLM agents as general programs. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, pages 2443–2459, Renton, WA, 2026. USENIX Association. arXiv:2502.13965 (as Autellix).

[66] Rupak Majumdar, Mahmoud Salamati, Nikhil Singh, and Sadegh Soudjani. Learning metastable dynamics. arXiv:2609.14712 (v1, 13 September 2026); HSCC '26, 2026.

[67] Valentina Martina, Michele Garetto, and Emilio Leonardi. A unified approach to the performance analysis of caching systems. In *IEEE INFOCOM 2014 – IEEE Conference on Computer Communications*, pages 2040–2048, 2014. doi: 10.1109/INFOCOM.2014.6848145.

[68] Ben Maurer. Fail at scale. *ACM Queue*, 13(8):30–46, 2015. doi: 10.1145/2838344.2839461.

[69] Jiaying Meng and Bojie Li. Metronome: Bound the cache, keep the beat for real-time interaction model serving. arXiv:2607.02640 (v1, 2 July 2026), 2026.

[70] Utopia Meng, Unicornt Zhao, Derek Li, Goalen Gao, and Frank Du. Talaria: Session-aware serverless serving of hundred-billion-parameter LLMs. arXiv:2607.17181 (v1, 19 July 2026), 2026.

[71] John Nagle. Congestion control in IP/TCP internetworks. RFC 896, 6 January 1984; `https://www.rfc-editor.org/rfc/rfc896`, 1984.

[72] Randolph Nelson. Stochastic catastrophe theory in computer performance modeling. *Journal of the ACM*, 34(3):661–685, 1987. doi: 10.1145/28869.28878.

[73] Chengyi Nie, Nian Si, and Zijie Zhou. A queueing-theoretic framework for stability analysis of LLM inference with KV cache memory constraints. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 93016–93031, 2026. arXiv:2605.04595.

[74] Zaifeng Pan, Ajjkumar Dahyalal Patel, Yipeng Shen, Zhengding Hu, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, pages 139912–139931, 2025. doi: 10.52202/085713-4208. arXiv:2507.07400.

[75] Zaifeng Pan, Yipeng Shen, Zhengding Hu, Zhuang Wang, Aninda Manocha, Zheng Wang, Zhongkai Yu, Yue Guan, and Yufei Ding. ScaleSim: Serving large-scale multi-agent simulation with invocation distance-based memory management. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 95898–95913, 2026. arXiv:2601.21473.

[76] Zaifeng Pan, Chris Wu, Zhengding Hu, Xinwei Qiang, Zhongkai Yu, and Yufei Ding. RR-Evict: Fine-grained prefix cache eviction beyond LRU for agentic LLM serving. arXiv:2609.32278 (v2, 2 October 2026; under review), 2026.

[77] Sungjoon Park, Changue Jung, Kyungno Joo, Mincheol Kang, Jaehyung Ahn, Sehwan Lee, and Sangjoon Kim. ServeTwin: A benchmark-validated simulator for distributed LLM architecture exploration. arXiv:2610.02732 (v1, 2 October 2026), 2026.

[78] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392, 2025.

[79] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — A KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 155–170, Santa Clara, CA, 2025. USENIX Association. Technical-report version: arXiv:2407.00079, titled "Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving".

[80] Qwen Team. Qwen2.5 technical report. arXiv:2412.15115, 2024.

[81] Rakibul Hasan Rajib, Mengxin Zheng, and Qian Lou. AgentServeSim: Serving-system simulation and policy search for LLM agent programs. arXiv:2606.09613 (v3, 5 September 2026), 2026.

[82] Bianca Schroeder, Mor Harchol-Balter, Arun Iyengar, Erich Nahum, and Adam Wierman. How to determine a good multi-programming level for external scheduling. In *Proceedings of the 22nd International Conference on Data Engineering (ICDE '06)*, page 60, 2006. doi: 10.1109/ICDE.2006.78.

[83] Kunming Shao, Jierun Chen, Jiangnan Yu, Xiao-Hui Li, Chaofan Tao, Yanli Wang, Huanxin Lin, Kwang-Ting Cheng, Chi Ying Tsui, and Haoli Bai. EfficientAgent: What makes KV cache offloading work for concurrent agents? arXiv:2609.33762 (v1, 27 September 2026), 2026.

[84] Yannis Smaragdakis, Scott Kaplan, and Paul Wilson. EELRU: Simple and effective adaptive page replacement. In *Proceedings of the 1999 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, pages 122–133, 1999. doi: 10.1145/301453.301486.

[85] M. S. Squillante and E. D. Lazowska. Using processor-cache affinity information in shared-memory multiprocessor scheduling. *IEEE Transactions on Parallel and Distributed Systems*, 4 (2):131–143, 1993. doi: 10.1109/71.207589.

[86] Shriniwas Ramesh Suram. Cacheable by design? training mixture-of-experts routers for locality against the edge memory-bandwidth wall: A pre-registered negative result with a systems measurement study. arXiv:2608.18261 (v1, 18 August 2026), 2026.

[87] SWE-agent. mini-SWE-agent. GitHub repository, release v2.0.0 (11 February 2026); `https://github.com/SWE-agent/mini-swe-agent`, 2026.

[88] Shubham Tiwari, Tapan Chugh, Nash Rickert, Simon Peter, Ratul Mahajan, and Haiying Shen. CacheWise: Understanding workloads and optimizing KVCache management for efficiently serving LLM coding agents. arXiv:2606.16824, 2026.

[89] Pieter van Rooyen. First-order recoverability collapse in self-referential information decoders: The operating loop of an AI system as a driven nonequilibrium steady state. arXiv:2606.24861 (v3; v1 23 June 2026), 2026.

[90] Michael Wang, Yikang Yue, Shaobo Li, Yirui Eric Zhou, Chen Wang, and Jian Huang. Benchmarking LLM serving systems for agentic AI workloads with XPerf. arXiv:2608.20370, 2026.

[91] Yifei Wang, Hancheng Ye, Yechen Xu, Cong Guo, Chiyue Wei, Qinsi Wang, Dongting Li, Tingjun Chen, Hai "Helen" Li, Danyang Zhuo, and Yiran Chen. MARS: Efficient, adaptive co-scheduling for heterogeneous agentic systems. arXiv:2604.26963 (v2, 15 June 2026), 2026.

[92] M. V. Wilkes. The dynamics of paging. *The Computer Journal*, 16(1):4–9, 1973. doi: 10.1093/comjnl/16.1.4.

[93] Fangzhou Wu, Sandeep Silwal, and Qiuyi (Richard) Zhang. Randomization boosts KV caching, learning balances query load: A joint perspective. In *International Conference on Learning Representations (ICLR 2026)*, 2026. arXiv:2601.18999.

[94] Tian Xia, Hanchen Li, Zhifei Li, Xiaokun Chen, Hao Kang, Yifan Qiao, Yi Xu, and Ion Stoica. Idleness is relative: Exploiting tool-call idle windows for offloading in agentic systems with MORI. arXiv:2606.00866, 2026.

[95] Bing Xie, Zhipeng Wang, Masahiro Tanaka, and Zhen Zheng. PEEK: Predictive queue-informed KV cache management for LLM serving. arXiv:2607.02525 (v2, 12 September 2026; under review), 2026.

[96] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 50528–50652, 2024. doi: 10.52202/079017-1601. arXiv:2405.15793.

[97] Heyuan Yao, Chutong Gao, Yuan Lyu, Izzy Grosof, and David Simchi-Levi. Multi-turn LLM conversations under the least-recently-used policy: Mean-field asymptotics and hit ratio approximation. arXiv:2609.02027, 2026.

[98] Tokumi Yokohira and Hideo Miyahara. Analysis of catastrophic phenomena in a multiprogramming system. *Systems and Computers in Japan*, 20(6): 60–68, 1989. doi: 10.1002/scj.4690200607.

[99] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, Carlsbad, CA, 2022. USENIX Association.

[100] Lingfan Yu, Jinkun Lin, and Jinyang Li. Stateful large language model serving with Pensieve. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*, pages 144–158. ACM, 2025. doi: 10.1145/3689031.3696086. arXiv:2312.05516.

[101] Yichao Yuan, Mosharaf Chowdhury, and Nishil Talati. KAIROS: Stateful, context-aware power-efficient agentic inference serving. arXiv:2604.16682 (v1, 17 April 2026), 2026.

[102] Yichao Yuan, Ankita Nayak, Souvik Kundu, and Nishil Talati. Agentic AI workload characterization. In *Proceedings of the 2026 IEEE International Symposium on Workload Characterization (IISWC)*, 2026. arXiv:2605.26297 (v2, 21 September 2026), listed there as "Agentic AI Workload Characteristics".

[103] Rui Zhang, Chaeeun Kim, Shaoting Feng, Kuntai Du, Yuhan Liu, Yi Zhong, Cheng-Wei Ching, Junchen Jiang, and Liting Hu. Learning agent execution for KV-cache management in agentic serving. arXiv:2608.14624, 2026.

[104] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 62557–62583, 2024. doi: 10.52202/079017-2000.

[105] Kan Zhu, Mathew Jacob, Chenxi Ma, Yi Pan, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. TraceLab: Characterizing coding agent workloads for LLM serving. arXiv:2606.30560 (v2, 30 June 2026), 2026. Trace release v0.0.2, 24 July 2026: `https://github.com/uw-syfi/TraceLab`.

A Protocol details

Table 7 lists the workload and engine settings of the real-engine runs.

The sequence of stages. *Stage 0*, a smoke test of server and client on one GPU, came first; its server logs and those of Stage C calibrated the simulator (§4.4). After E2 we repeated the sequence with the native pool (Stage C, E0a, E0b, a second registration and *native E2*) and ran an exploratory policy grid. On vLLM we first measured λ_{miss} and ran the BURST arm alone under an exploratory registration, and later ran its own Stage C and E0b, a third registration and its paired test (§5.2). An exploratory replication of E0b later used seeds 3–5 at two rates (§5.1). Later cells repeat the first stages on an H200 at the same pool (§5.5), with Llama-3.1-8B and with pools of 150k and 600k tokens, and registered tests of a rescue (§5.3), of other burst sizes (§5.4), of a pause in arrivals and of a tool slowdown follow the paired-test protocol; §7 lists their frozen predictions and the outcomes in hand.

Stage 0. The smoke test (job 17060165) verified that every engine flag exists; that cached tokens are reported (a repeated 64-token prompt had 63 tokens cached); that the server returns output token ids; that contexts up to 40,350 tokens are accepted; that the KV pool is exactly 300,000 tokens; that client and server agree on prompt length (0 mismatches over 3191 requests); and that the flush empties the cache (canary 1023 cached tokens before, 0 after).

Stage C and E0a. Closed population; all sessions start at $t = 0$ and a finished session is replaced at once; 900 s warmup and an 1800 s measured window; one run per n (seed 0); flush and canary check between runs. The native-pool and vLLM closed curves used the same protocol, with the populations of Table 6. E0a disables the radix cache and runs 32 sessions with seeds 0, 1 and 2, plus 64 sessions with seed 0 as a saturation check. Its per-run values over the window were (sessions/s, turns-based sessions/s, uncached tokens/s): seed 0, 0.0567, 0.0643, 11,076; seed 1, 0.0817, 0.0773, 11,342; seed 2, 0.0767, 0.0730, 11,312; 64 sessions, 0.0544, 0.0676, 11,431. The spread comes from in-flight composition: the mean turn index of the requests in the window was 15.9, 13.2 and 14.1 for seeds 0, 1 and 2. vLLM E0a ran the same protocol with vLLM's prefix caching off: seed 0, 0.0539, 0.0603, 10,208; seed 1, 0.0728, 0.0705, 10,487; seed 2, 0.0700, 0.0662, 10,352; 64 sessions, 0.0467, 0.0631, 10,531.

E0b. Poisson session arrivals from an empty cache and queue at 1.25, 1.5, 1.75, 2.0, 2.5 and 3.0 λ_{ref}; one 3000 s run per rate; stable if the final queue is below 50 and the fitted rise of the queue over the last 1500 s is at most 10. All runs used arrival seed 0, so they share one random-number stream and the arrivals at each rate are a subset of those at the next higher rate. The six runs received 302, 359, 416, 472, 585

Table 6: Real-engine stages (SLURM job IDs on Nova). Submit, start, end and node of every job through the vLLM E0b are in `provenance/sacct_cited_jobs_2026-10-30.txt`, start, end and node of the vLLM paired test in its data directory (`sacct_17166121-17166138.txt`); those of the BASIN-1 and fresh-seed jobs and of the rescue jobs in theirs (`sacct_17168500-17168507.txt`, `sacct_1716896x.txt`); the records of the other later jobs are [pending: refreshed accounting export].

Stage	Jobs	What it measures
Stage 0	17060165	smoke test
Stage C	17062103	closed $\mu(n)$, 300k pool, FCFS, $n \in \{16, 20, 24, 32\}$ (node gpu-1)
	17062104	FCFS, $n \in \{48, 96, 300\}$ (gpu-3)
	17062105	LPM, $n \in \{48, 300\}$ (gpu-2)
E0a	17072634–37	λ_{miss}, cache off
E0b	17084750–55	from-empty stability
E2	17093387–404	registered paired test, 2 rates × 3 seeds × 3 arms
native C	17096856	$\mu(n)$, native pool, $n \in \{24, 48, 64, 80\}$ (gpu-2)
	17096857	$n \in \{96, 128, 300\}$ (gpu-1)
native E0a	17096858	λ_{miss}, native pool
grid	17097086–103	BURST arm, 3 policy cells
vLLM	17097972–74	install, GPU check, smoke
native E0b	17100938–39	from-empty stability, native pool
vLLM E0a	17100940	λ_{miss}, vLLM, cache off
native E2	17102173–90	registered paired test, native pool, 2 rates × 3 seeds × 3 arms
vLLM BURST	17102318–23	BURST arm, vLLM, exploratory
vLLM C	17162959	$\mu(n)$, vLLM, 300k pool, $n \in \{16, 20, 24\}$ (gpu-3)
	17162960	$n \in \{28, 32, 40\}$ (gpu-3)
	17162961	$n \in \{48, 96, 300\}$ (gpu-2)
vLLM E0b	17162962–63	from-empty stability, vLLM
vLLM E2	17166121–38	registered paired test, vLLM, 2 rates × 3 seeds × 3 arms
Rounds 11 to 14 (Tables 17 and 18; some outcomes pending)		
H200	17166807	smoke, H200 NVL, 300k pool
	17166810–13	E0a and its 64-session check
	17166818–22	Stage C, n = 16–300 (nine runs, with the next row)
	17166828, 30, 32, 34	Stage C, the rest of the nine runs
	17168377–82	E0b
	17169668–71	E0b reseeds, added after 17168377–82 were read (post hoc)
	17169612–14	Stage C, n = 36, 44, 52
	17169672–80	Stage C, 600k pool
	17169682–87	E0b, 600k pool
Llama	17166808	smoke, Llama-3.1-8B
	17166814–16	Stage C
	17166823, 29, 31, 33	E0a and its 64-session check
	17169565–70	E0b
pools	17168864–69	Stage C, 150k and 600k pools
	17168870–81	E0b, 150k and 600k pools
BASIN-1	17168500–05	bursts of 100, then 25
E0F	17168506–07	from-empty, seeds 3–5
LPM-INF	17168743–45	BURST, no LPM fallback
BASIN-2	17169893–901	bursts of 40, 60, then 50
TOOLSLOW	17169902–07	δ = 10, then 30 s
SHED	17169908–11	τ = 600, then 3600 s
rescue	17168965	smoke test of the gate (never decisive)
	17168966–71	R1, 300k pool, 2 rates × 3 seeds
	17168972–74	R4, native pool, 3 seeds

Table 7: Workload and engines of the real-engine runs.

First prompt L_0	2000 tokens
Tool output per turn Δ	550 tokens
Output per turn d	100 tokens, greedy, `ignore_eos`
Turns T	$\min(\text{Geometric}(1/15), 60)$, $E[T] = 14.761$
Tool time Z	exponential, mean 5 s
Prompt at turn k	$2000 + 650(k-1)$ tokens
Output feedback	server-returned token ids
Engine	SGLang 0.5.22, FlashInfer
Scheduling	FCFS, waiting-prefix refresh on
Batching	chunked prefill 8192, ≤ 256 running
KV pool	300,000 tokens; native 955,757
Context limit	49,152 tokens
Model	Qwen2.5-7B-Instruct, rev. a09a3545
GPU	A100 80GB PCIe, 81,920 MiB, driver 580.178.04
vLLM	0.32.0, FCFS (default), prefix caching on chunked prefill 8192, ≤ 256 running 300,000 tokens (18,750 blocks of 16)

and 697 sessions, 8.3–12.6% above λT and about 2.1 Poisson standard deviations above it in every run. Native E0b used the same rule at 1.5, 2.5, 3.5, 4.5, 5.5 and 6.5 times the native λ_{ref} (361, 587, 811, 1034, 1257 and 1478 sessions, 5.6–11.7% above λT), and vLLM E0b the multiples of the 300k runs of vLLM's λ_{ref} (279, 331, 383, 435, 539 and 642 sessions, 8.6–13.3% above λT; stable through 2.0, final queue 81.6 and 247.1 at 2.5 and 3.0).

The fresh-seed E0b (E0F; jobs 17168506–07) repeated 2.0 and 2.5 λ_{ref} on the 300k pool with arrival seeds 3, 4 and 5 under the same rule. All six runs were stable, with final queue 0, so the frozen replication rule, which needed 2.0 λ_{ref} stable in all three seeds and 2.5 λ_{ref} unstable in at least two, is not met. The nominal bracket of 2.0 to 2.5 λ_{ref} therefore comes from the single seed-0 grid: over seeds 0 and 3–5, 2.0 λ_{ref} is stable in 4 of 4 runs and 2.5 λ_{ref} in 3 of 4. Measured instead in realized arrival rates (base arrivals in the 3000 s run over 3000 s $\times$ λ_{ref}), the twelve from-empty runs of seeds 0 and 3–5 are stable up to 2.665 λ_{ref} and unstable from 2.726 λ_{ref}, monotone in the realized rate, so the edge lies between 2.665 and 2.726 λ_{ref}. The seed-0 run at 2.5 λ_{ref} is the one at 2.726: it received 9% more arrivals than its nominal rate. Either way, the registered rates of 1.35 and 1.65 λ_{ref} lie inside the window. The simulator's blind prediction was stable in all six runs; the heuristic F2 of §6.3 gave each 2.5 λ_{ref} run a probability of 0.69 of being unstable and was too pessimistic. This test is exploratory: it has no registration of its own and does not change the E2 registration, whose window was fixed by the seed-0 bracket.

E2. One arm per 8700 s job, each on a fresh server; burst of 200 fresh sessions within 60 s at $t_b = 1200$ s for BURST and CAPPED; CAPPED gate $n_{\text{cap}} = 20$; evaluation window $[t_b + 4500, 8700]$ s. The registered rule (Table 8) was applied by the analysis tool frozen at the registration commit, with the same parameter overrides that the jobs themselves used. Native E2 used the same protocol with the native registration's rates, reference rates and gate $n_{\text{cap}} = 64$. Twelve of its 18 runs started after the cluster checkout moved from the registration commit to `a78ce07`, which adds only the vLLM registration, its input file and two notes files (the research log and the status notes), with no code change, and its verdict ran on that commit; two of its 18 servers sized the pool one token smaller, at 955,756. The vLLM paired test uses the same protocol with the vLLM registration and $n_{\text{cap}} = 20$. It uses the same seeds, rates and reference rate as the exploratory vLLM BURST runs, so its BURST arm repeats their workloads rather than replicating them independently.

Why the rule uses tokens. Criterion (a) uses tokens because a trapped engine with a growing backlog serves mostly young sessions, whose turns are short, so at the all-miss token rate it completes more turns per second than the stationary all-miss mix, and sessions faster than λ_{miss} (§3.3): on traps simulated in our earlier simulator (§4.4), a session-rate test fired on at most 1 of 29 at 2 λ_{miss} and none of 39 at 2.5, while the token test fired on all 84 traps at 1.5–2.5 λ_{miss}.

Drain-time estimates. The registrations' feasibility check used $b/(\mu(c) - \lambda)$, which ignores the $c - n_1$ free slots that a burst fills at once: 1571 s at 1.35 and 1889 s at 1.65 λ_{ref} for E2, 668 s and 1043 s at 2.5 and 4.0 λ_{ref} for the native test, and 1529 s and 1800 s for vLLM. With the populations present at the burst (5–12 sessions at 300k, 11–32 with the native pool, 5–11 on vLLM), the fluid estimate $(n_1 + b - c)/(\mu(c) - \lambda)$ of §3.4 is 1453–1508 s and 1757–1804 s, 491–518 s and 840–876 s, and 1414–1460 s and 1674–1692 s. The CAPPED gate's queue emptied 1547–1904 s and 2021–2693 s after the burst on E2, 3–52% longer than this estimate; 512–583 s and 837–937 s with the native pool, between 4% shorter and 13% longer; and 1521–2735 s on vLLM, 1.05–1.63 times it.

Rescue (R1, R4). One rescue run per 8700 s job, each on a fresh server, from a frozen clone at the registration tag; the client's arrival plan is the BURST plan of the same seed and rate, byte for byte, so until t_{on} the rescue run replays the BURST run's arrival plan (a fresh run on a fresh server, not the same run). At t_{on} the gate starts to bind with cap c by attrition and pause: a session whose tool call ends while more than c sessions are admitted gives up its slot and joins a resume queue, which is served before the queue of new arrivals, each in FIFO order; a paused session's KV is not protected while it waits. The cap is removed at the first 10 s tick at or after $t_{\text{on}} + 600$ s at which both queues have been empty for 300 s; there is no forced release. The registered per-run rule uses the follow-up hit fraction, the share of a session's turns after its first, attributed by completion time, with more than half of the prompt cached: (a) VALID if the follow-up hit fraction over $[t_{\text{on}} - 300, t_{\text{on}})$ is below 0.2 and more than 50 requests or sessions are outstanding at t_{on} (requests in flight plus sessions waiting at the gate or paused), on the registered hardware; (b) RELEASED by 7200 s; (c) POST-GOOD over $[t_{\text{rel}} + 300, 8700)$: every 300 s window, tiled from the start with a short remainder covered by the window

ending at 8700 s, has a follow-up hit fraction of at least 0.5, the whole interval above 0.8, an uncached token rate below 0.5 times the all-miss reference, and the registered RECOVERED test holds over the last 300 s; a window with no completed follow-up fails (c) without being a relapse. RESCUED is (a), (b) and (c); RELAPSE is any such window below 0.2. A same-seed BURST control, from the paired test, is STUCK if its follow-up hit fraction over $[t_{on}, 8700)$ is below 0.2 and it is not RECOVERED over the last 300 s. A cell passes if at least 2 of its 3 seeds are RESCUED and every same-seed control is STUCK; R1 is refuted if 2 of its 6 runs relapse or are not released by 7200 s, and R4 if 2 of its 3 relapse; a refutation takes precedence over a pass, and a failed session in any counted run withdraws the claim that nothing was shed. The pairing requires common random numbers with the committed BURST and NOBURST runs (the BURST plan byte-identical, the NOBURST plan the rescue plan without the burst sessions), equal resolved rates, references, hardware and burst time. Each job ran only a mechanical check of the gate (`analysis.rescue check-run`: engage within 10 s of t_{on}, release on the 10 s grid after the 300 s hold, no request errors), which passed in all ten jobs; no hit fraction of a rescue run was read before all runs of its cell had ended. The controls ran on 2026-10-24 to 26 as part of the paired tests, not alongside the rescue runs. A smoke job under its own registration checked the gate's mechanics before the registered runs and is never decisive.

Table 8: The registered decision rule.

Rate estimator	uncached prefill tokens/s
Reference rate	E0a all-miss, 11,243 tokens/s (native: 11,282; vLLM: 10,349)
λ_{miss} in the rule	the cell's λ_{ref}
Rate tolerance	±0.15
Backlog slope	least squares over 10 s bin means of sessions in the system, gate included
Minimum window	600 s
Trapped	$\lambda > \lambda_{miss}$, rate within tolerance, and slope $> 0.5\,(\lambda - \lambda_{miss})$
Recovered	hit > 0.8 and outstanding < 20, last 300 s
Draining	slope < 0
Indeterminate	none of the above
Precedence	trapped, recovered, draining, indeterminate
Hit metric	turns with cached/prompt > 0.5
Pass	$\geq 2/3$ of ≥ 3 seeds with all three arm conditions met in the seed

Backlog definitions. The figures and Table 1 report the backlog as sessions in the system (arrived minus ended), which includes sessions asleep in tool calls (8–12 in the good state at 300k, 16–34 with the native pool). The registered RECOVERED test uses the outstanding backlog (requests in flight plus sessions held at the gate). The simulator's predictions count waiting requests plus the gate queue. The rescue rule's outstanding work also counts sessions paused by the gate (Fig. 3).

B The registrations

The registration was filled by formula from committed copies of the Stage C curve, the E0a calibration and the E0b bracket. The window was (1, 2.0) λ_{ref}, its upper edge E0b's largest all-stable multiple (below $\mu^*/\lambda_{ref} = 3.26$), a nominal value from E0b's single seed-0 grid; fresh seeds later put the from-empty edge, in realized arrival rates, between 2.665 and 2.726 λ_{ref}, and both registered points lie inside the stable range on either reading (§5.1). The rates followed a rule proposed in the runbook and adopted in the project status notes on 2026-10-22, the day before E0b was submitted (Appendix B): one third and two thirds of the way across the window, rounded to 0.05, which gives 1.35 and 1.65 λ_{ref} (0.0966 and 0.1180 sessions/s). The cap was the smallest n with $\mu(n) \geq 0.95\mu^*$: $n_{cap} = 20$, since $\mu(20) = 0.224$ is about 4% below $\mu(24) = 0.233$, inside the rule's 5%. The file was committed and reached the project's GitHub repository at 15:09:54 CDT on 2026-10-24, before the E2 jobs were submitted at 15:13 (Table 10). We call a test pre-registered when its registration file was committed and pushed to GitHub before its runs; this is not an entry in a third-party registry. Registration and the frozen predictions of §4.4 are rigor, not a contribution: they let a reader check that each rule and each prediction existed before its outcome. The design follows the model-to-system pipeline of Isaacs et al., in which an abstract model chooses the experiments that then test it [48], and pre-registered gates have been used in ML-systems measurement before [86]. Appendix B lists every registration we filled, with its status, and Appendix C every prediction round with its time record; the files carry no timestamp other than GitHub's push log. The repository we publish is a redacted copy of the development repository whose `provenance/` directory ties every registration file, by hash chains, to the development repository's push log (availability statement). Commit times are set by the committer, so the external record is GitHub's push log, archived as
`provenance/github_activity_2026-10.json`
(Appendix C). The job script refuses to start unless the file is committed and unmodified and the job's cell, rates, cap, reference rates, hardware and protocol match it, and writes its SHA-256 into every run; the analysis calls a verdict decisive only for the registered cell and rates with every run bound to the file and started after its commit (Appendix B); the label records this binding, not statistical strength, and the analysis checks commit order, not push time, so its own label "pre-registered" does not by itself meet our definition. The native registration was filled and enforced the same way and reached GitHub 10 minutes before its jobs were submitted (§5.2). The vLLM registration was filled from vLLM's own Stage C curve, E0a calibration and E0b bracket, and does not meet the definition: it was committed 37 s before its 18 jobs were submitted, and the cluster's record of the submission names the file but not its hash, but it reached GitHub 2 min after the submission, when two of the jobs had started and none could have ended, so we call this test registered, not

pre-registered (§5.2). On the H200 the rule could not be applied: its window was (1, 1.3) λ_{ref} (the largest all-stable E0b multiple, below $\mu^*/\lambda_{\text{ref}} = 1.99$), and the points the rule gives, 1.10 and 1.20 λ_{ref}, lie outside the band (1.1, 1.17) λ_{ref} that its 10% margin at each edge of the window leaves, so the fill returned INCOMPLETE and that cell has no registered test (§7).

The registration is `configs/agentload_prereg_2026-10-24.yaml`, status FILLED, committed in `d279b7b` at 15:09:12 CDT on 2026-10-24 and pushed to GitHub at 15:09:54, before the E2 jobs were submitted at 15:13:39, SHA-256 `2a6d680a9615bba962a5673d6db6b3e4 ca5102bf778ca974cea46147f5cc0ac6`. Its fields:

- **Cell and inputs.** Engine cell `sglang-fcfs-r1` (SGLang FCFS, waiting-prefix refresh on); committed copies of the Stage C curve, the E0a calibration and the E0b bracket under `prereg/inputs/2026-10-23/`; protocol 8700 s, burst of 200 within 60 s at 1200 s.
- **Measured.** $\mu(n)$ in sessions/s: 0.1756 ($n = 16$), 0.2239 (20), 0.2333 (24), 0.1150 (32), 0.0489 (48), 0.0611 (96), 0.1078 (300). $\mu^* = 0.2333$; $n_{\text{cap}} = 20$ by the smallest-within-5% rule (argmax 24; flagged ambiguous). $\lambda_{\text{miss}} = 0.07167$ (sessions) and 0.07153 (turns-based, $= \lambda_{\text{ref}}$); all-miss rate 11,243 tokens/s; E0b largest all-stable multiple 2.0, first unstable 2.5, monotone (arrival seed 0 only; the fresh seeds of Appendix A came later); hardware A100 80GB PCIe.
- **Predicted boundaries.** Recovery threshold λ_{ref} (not $\mu(300)$, which a synchronized start biases upward). Window (0.0715, 0.1431) sessions/s = (1.0, 2.0) λ_{ref}, upper edge from E0b. CAPPED drain must fit within 80% of the 7200 s available after the burst (5760 s), which holds for λ up to 2.64 λ_{ref}; predicted drain 1571 s at 1.35 and 1889 s at 1.65.
- **Rates.** 1.35 and 1.65 λ_{ref} by the one-third/two-thirds rule, rounded to 0.05, inside the margin band (1.1, 1.8); seeds 0, 1, 2. On 2026-10-22 the runbook proposed the rate rule and a cap of 20, with keeping 24 as the alternative (`288b801`, 13:13 CDT); both rules were adopted in the project status notes at 13:50 (`4435c51`) and reached GitHub at 17:45, before E0b was submitted on 2026-10-23; the registration's own text dates them 2026-10-23.
- **Predictions.** Inside the window: BURST trapped, NOBURST good through the run horizon, CAPPED recovered if the drain fits. Below λ_{ref}: BURST not trapped. Above the window: NOBURST may escape, so no such rate was registered.
- **Kill conditions.** NOBURST not good in at least one third of the seeds at a registered rate; BURST not trapped in two thirds of the seeds at every registered rate; CAPPED not recovering while BURST traps and the drain fits; a window that is empty or narrower than the noise.
- **Warnings recorded at filling.** $|\mu(300)/\lambda_{\text{miss}} - 1| = 0.507$ (a check only), and the ambiguous cap.

The verdict is in `results/agentload_analysis_burst2/20261025-144012/` inside the E2 data directory: the registered analysis passes at 1.35 and 1.65 λ_{ref} (the tool's labels, assigned by commit order, are pre-registered and decisive; by the definition of §4.3 the test is pre-registered). The three grid cells have their own derived registrations, `configs/agentload_prereg_2026-10-24_grid_<cell>.yaml` for the cells `sglang-fcfs-r0`, `sglang-lpm-r1` and `sglang-lpm-r0`, with status EXPLORATORY; the analysis never calls them decisive.

The native registration. `configs/agentload_prereg_2026-10-25_native.yaml`, status FILLED, committed in `b8c19ec` at 17:31:16 CDT on 2026-10-25 and pushed to GitHub at 17:31:56, 10 minutes before the native E2 jobs were submitted (17:42:29), SHA-256 `21bf7c03e0630b65b80d086058815e12 c5ba9fbe41e9864d3dd0183b9bf776c1`; the same template, formulas, trap rule, kill conditions and protocol as above. Cell `sglang-fcfs-r1-native`; inputs under `prereg/inputs/2026-10-25-native/`. Measured: $\mu(n)$ 0.2650 ($n = 24$), 0.4311 (48), 0.4789 (64), 0.4772 (80), 0.1228 (96), 0.0667 (128), 0.1006 (300); $\mu^* = 0.4789$; $n_{\text{cap}} = 64$ by the smallest-within-5% rule (also the argmax; flagged ambiguous, $n = 80$ within 0.35%); $\lambda_{\text{ref}} = 0.07180$ (turns-based); all-miss rate 11,282 tokens/s; native E0b largest all-stable multiple 5.5, first unstable 6.5, monotone. Window (0.0718, 0.3949) sessions/s = (1, 5.5) λ_{ref}, upper edge from native E0b, below $\mu^*/\lambda_{\text{ref}} = 6.67$. Rates 2.5 and 4.0 λ_{ref} (0.1795 and 0.2872 sessions/s) by the same rule, inside the margin band (1.1, 4.95); the rule was recorded in the project status notes (`6cb43e9`, pushed 15:08 CDT) before native E0b was read. Predicted drains 668 s and 1043 s; the drain fits up to 6.19 λ_{ref}. Warnings recorded at filling: $|\mu(300)/\lambda_{\text{miss}} - 1| = 0.401$, the ambiguous cap, and an incomplete grid (no $n = 16$, 20 or 32). The verdict is `results/agentload_analysis_burst2/20261026-100653/` inside the native E2 data directory: the registered analysis passes at 2.5 and 4.0 λ_{ref} (with the same tool labels; pre-registered by the definition of §4.3).

The exploratory vLLM registration. `configs/agentload_prereg_2026-10-24_grid_vllm-fcfs.yaml`, status EXPLORATORY, derived from the E2 registration, committed in `a78ce07` at 18:06:18 CDT on 2026-10-25 and pushed to GitHub at 18:06:33, 46 s before the vLLM BURST jobs were submitted, SHA-256 `c5037cac6003c4e98da89f88dae953bf e68bb21a27a4e3e5e3dab3c8891aeab5`. Cell `vllm-fcfs`, BURST arm only; the window, multiples, protocol and trap rule are inherited from `sglang-fcfs-r1`, and the reference rates come from `prereg/inputs/2026-10-25-vllm/calibration.json`, a byte-identical copy of the vLLM E0a calibration. Its predicted-boundary block is copied unchanged from the E2 registration (window 0.0715–0.1431 sessions/s, all-miss rate 11,243 tokens/s), but the runs used vLLM's references, 0.06566 sessions/s and 10,349 tokens/s (`lambda_ref_resolved` and

uncached_tokens_ref_resolved in each run's meta.json), so their rates were 1.35 and 1.65 times vLLM's λ_{ref}. The verdict, results/agentload_analysis_burst2/20261026-100701/ inside the native E2 and vLLM BURST data directory, reports per-run states and no registered verdict.

The vLLM registration. configs/agentload_prereg_2026-10-30_vllm.yaml, status FILLED, SHA-256 8f2d5b44d6365795fedde8bf673d51e54bf8a685a14d563c86de44492bb95f84; the same template, formulas, trap rule, kill conditions and protocol as the E2 registration. Cell vllm-fcfs; inputs: the vLLM Stage C curve and E0b bracket under prereg/inputs/2026-10-30-vllm/, byte-identical to the analyses in the vLLM Stage C and E0b data directories, and the vLLM E0a calibration above. Measured: $\mu(n)$ 0.1750 ($n = 16$), 0.2194 (20), 0.2183 (24), 0.1189 (28), 0.0433 (32), 0.0411 (40), 0.0439 (48), 0.0572 (96), 0.1050 (300); $\mu^* = 0.2194$; $n_{\text{cap}} = 20$ by the smallest-within-5% rule (also the argmax; flagged ambiguous, $n = 24$ within 0.5%); $\lambda_{\text{ref}} = 0.06566$ (turns-based); all-miss rate 10,349 tokens/s; vLLM E0b largest all-stable multiple 2.0, first unstable 2.5, monotone. Window (0.0657, 0.1313) sessions/s $= (1, 2.0)\,\lambda_{\text{ref}}$, upper edge from vLLM E0b, below $\mu^*/\lambda_{\text{ref}} = 3.34$. Rates 1.35 and 1.65 λ_{ref} (0.0886 and 0.1083 sessions/s) by the same rule, inside the margin band (1.1, 1.8); this rule and the cap rule were recorded in the project status notes (836063a, pushed 12:16 CDT on 2026-10-30) after the vLLM Stage C and E0b jobs were submitted and before any of them had ended. Predicted drains 1529 s and 1800 s; the drain fits up to 2.81 λ_{ref}. Warnings recorded at filling: $|\mu(300)/\lambda_{\text{miss}} - 1| = 0.599$ and the ambiguous cap. The file was filled from five of the six vLLM E0b runs while the 3.0 λ_{ref} run was still running; with 2.5 λ_{ref} unstable that run could not change the edge, and it was unstable too, so the six-run bracket equals the registered one. The file was committed in 5a6fbcd at 16:23:59 CDT, 37 s before the 18 jobs of the paired test (17166121–38) were submitted at 16:24:36 (the research log gives 16:30), and the jobs run from the cluster's copy of the repository at that commit; Slurm's record of the submission names the file but not its hash. It reached GitHub at 16:26:35, 1 min 59 s after the submission, when two of the jobs had started (at 16:25:42 and 16:26:17) and none could have ended. This registration therefore does not meet the definition of §4.3: it was committed before the jobs were submitted, but pushed after the first two had started, so we call this test registered, not pre-registered. As for E2, the job script writes the file's SHA-256 into every run's meta.json.
The verdict, results/agentload_analysis_burst2/20261031-020837/ in the paired test's data directory (results/nova/e2_vllm_1716612x_pulled_20261031-020956/), reports a pass at both rates; all 18 runs' meta.json files carry the registration's SHA-256. The tool also prints the labels pre-registered and decisive, which it assigns by commit order; by the definition of §4.3 the test is registered, not pre-registered. The frozen mechanism predictions for this test (configs/r11_mechanism_predict.yaml, notes/PREDICTIONS_r11_mechanism.md; commit f1338e3, pushed at 18:20:13 CDT on 2026-10-30, when three of the 18 jobs were running and none had ended; the first ended at 18:55:07) were evaluated with the frozen scripts, run unchanged. Byte copies of the frozen files and their outputs are under r11_mechanism/ in the same directory, and the scored decision is derived/r11_mechanism_decision.json, which records two detector false alarms in CAPPED runs, one more than the frozen rule allows (§5.2).

The rescue registrations. configs/agentload_prereg_2026-10-31_rescue.yaml (R1, cell sglang-fcfs-r1-rescue) and configs/agentload_prereg_2026-10-31_rescue_native.yaml (R4, cell sglang-fcfs-r1-rescue-native), status FILLED, SHA-256 502dee9e3107952a2f968c1b59b8ee3d4063870d72f0dfac1261e2b2e2316691 and ddd776f822755ba50472668b4af7dba3538354d98feb2579b8cca422b5215bb1, were committed in f0bc3ad at 22:10:47 CDT on 2026-10-30 (tag r11-rescue-reg) and reached GitHub at 22:39:02, 22 minutes before the ten jobs (a smoke job and the nine registered runs, 17168965–74) were submitted at 23:01:12–13; by the definition of §4.3 both tests are pre-registered. They reuse the inputs, rates and caps of the E2 and native E2 registrations, under their own cell labels, and add the rescue protocol, the per-run rule, the decision rule and the refutation conditions of Appendix A, with the rule's constants frozen in analysis/rescue.py at the tag. Three changes were accepted after a review of the simulator's predictions and before any rescue run: a refutation takes precedence over a pass, the no-shed part of a pass is withheld if any counted run has failed sessions, and R4, like R1, requires stuck same-seed BURST controls. R4's engage time of 2100 s replaced 2400 s after the simulator had predicted 13 of 20 rescued at 2400 s (notes/PREDICTIONS_r11_rescue.md). The verdicts, results/agentload_analysis_rescue_r1/20261031-071720/ and results/agentload_analysis_rescue_r4/20261031-071748/ in the data directory results/nova/rescue_1716896x_pulled_20261031-070502/, were computed on a clean tree with the registrations and the simulator's predictions as inputs: each reports a pass, with no rejected run and no pairing, protocol or registration problem, and every run's meta.json carries its registration's SHA-256. The tool labels each verdict decisive, a label it assigns by commit order.

Ledger of registrations. Table 9 lists every registration file we filled, with its status. Commit times are the committer's; push times come from GitHub's push log (provenance/push_times_2026-10.tsv, or the archived

log itself for the grid cells; for the registrations of 10-30 evening and 10-31, the live activity log as read on 10-31, which the archived copy does not yet include). Each is the first branch update that contains the commit; the BASIN-1 push also carried `9b67c26`, the commit of tag `r11-basin-reg`.

Superseded before registration: a native rescue engaged at 2400 s, whose simulator prediction (13 of 20 rescued) led to the engage time of 2100 s that R4 registers (`notes/PREDICTIONS_r11_rescue.md`). The SHA-256 of every file is in its tag message (`r11-basin-reg`, `r11-lpminf-pred`, `r11-rescue-reg`, `r14-basin2-reg`, `r14-extra-reg`) and in `provenance/registration-proofs/`.

C Prediction rounds and timeline

The calibrated simulator in detail. Our discrete-event simulator models a continuous-batching engine: iterations of at most 8192 prefill tokens and 256 sequences; admission reserves a request's end-of-turn KV; idle KV of sessions in a tool call or waiting is evicted whole, least recently used first; waiting requests pin nothing; a finished session's KV is freed at once, an oracle, since a real engine keeps it as evictable cache. An iteration costs a fixed time plus a per-token term plus attention terms. The five cost parameters were fitted only to SGLang's per-batch timing lines in the Stage 0 and Stage C server logs (decode $R^2 = 0.992$, median error 0.75%; prefill $R^2 = 0.998$, 1.3%), never to $\mu(n)$. It runs mixed prefill–decode iterations or, like SGLang, separate iterations with prefill priority; the primary (mixed) mode was the one closer to the real Stage C $\mu(n)$ under a pre-declared metric (mean relative error 0.104 against 0.127), a margin carried by one cell, and both modes' predictions are reported. It does not model partial prefixes, the waiting-prefix refresh or the overlap scheduler; reserving whole turns at admission is an upper bound on what paged engines hold (in round 5 of the earlier simulator, on-demand allocation with preemption and SGLang-like ratio reservation kept the curve and the trap).

Validation. The declared level test, simulated μ within 15% of the real value in each of 8 Stage C cells, failed in both modes: mixed batching missed one cell, FCFS $n = 48$, by +42% (0.070 against 0.049 sessions/s), separate batching two. The shape was reproduced: the hit fraction crosses 0.5 at $n = 32.5$ against 32.2, the plateau ($n = 16$–24) agrees within 9.2% and $n = 300$ within 2%, and $\mu(300)/\mu^*$ is 0.50 against 0.46. The single real run at $n = 48$ dipped below the simulator's all-miss level; its turns-based estimate, 0.063 sessions/s, is within 15%. Keeping a finished session's KV as evictable cache, a labelled sensitivity arm, left mixed batching the closer mode (0.118 against 0.146) but lowered the plateau (−15.5% at $n = 24$; 5 of 8 cells within 15%). In the earlier simulator the same change moved the orderings and pins of §I.2 by at most 0.025 λ_{miss} but stalled the CONCUR surrogate at its minimum cap in 12 of 18 cells; the static caps were not rerun.

Prediction rounds. The calibrated simulator then predicted, in both modes (mixed primary), B1: λ_{miss} and the all-miss token rate; B2: the fraction of from-empty runs that stay stable at each rate; B3: the E2 outcome at each rate from 0.09 to 0.16 sessions/s; B4: the closed curve with 600k- and 900k-token pools. Two later rounds repeated these predictions for the native pool (B1n–B4n) and for vLLM (B1v–B4v, with the simulator's token costs scaled to vLLM's measured all-miss rate, a fit, not a prediction), and three more, for the tests of §7, were made by the same simulator (rounds 11 and 12) and by the first-principles model of §6 (round 13, in two sets: tag `r13-theory-pred`, and `r13-theory-pred2` for BASIN-2, TOOLSLOW, SHED and the H200 600k cell). LLM coding agents made the simulator predictions, each in its own copy of the repository and instructed not to read the results they predicted; every round was committed and pushed under a tag before its results were read by its authors. What that means differs by round: B3, B4 and B3n are blind in time; B2 was pushed after E0b had finished on the cluster but before its results were pulled, and B2n, B2v and B4v after their jobs were submitted but before any had finished; B1, B1n and B4n are not blind, and B1 is close to the calibration data; B3v predates the vLLM registration, but its BURST part is blind only on the predicting agent's side. An independent verifier reran the first round. Appendix C gives the details and the time record of every round.

Commit and tag times are set by the committer. The external record is GitHub's activity log for the development repository, archived in `provenance/github_activity_2026-10.json`, together with the cluster's accounting records of the cited jobs (`provenance/sacct_cited_jobs_2026-10-30.txt`). The development repository is private; the archived log is an unsigned export made by the first author with GitHub's API, and the hash chains in `provenance/registration-proofs/` show that the files are unchanged, not when they were pushed; `provenance/push_times_2026-10.tsv` lists, for each registration and prediction commit, its first arrival on GitHub and the first job it concerns. GitHub records no push time for tags, so the push times given here are those of the first branch update that contains each commit. The branch holding B1–B4 (`64bc4e3`) and the verification reached GitHub at 14:53:07 CDT on 10-24, and the E2 registration (`d279b7b`) at 15:09:54, before the E2 jobs were submitted at 15:13:39. On 10-25, B1n–B4n (`e3ea48f`) arrived at 15:08:28, the native registration (`b8c19ec`) at 17:31:56 and the exploratory vLLM registration (`a78ce07`) at 18:06:33. Pull times are from our local records only.

B3 (E2) and B4 (native pool) reached GitHub before their experiments were submitted, so they are blind in time. B2 (E0b) was committed after E0b had finished on the cluster but before its results were pulled. The LLM agents that made the predictions worked in separate copies of the repository at a commit without the E0a or E0b results and were instructed not to contact the cluster and not to open the E0a or E0b result directories or the primary copy of the repository; they could technically have done so. Their tool-call transcripts, which are

Table 9: Every registration, in order. FILLED registrations bind a test's cell, rates, cap, references, hardware and protocol, and the job script refuses to run a test without one; EXPLORATORY ones are derived from a FILLED parent and are never called decisive. Times CDT.

File (`configs/agentload_prereg_*`)	status	committed	on GitHub	runs and outcome
`2026-10-24.yaml` (E2)	FILLED	`d279b7b` 10-24 15:09:12	15:09:54	18 runs; pass (§5.2)
`2026-10-24_grid_*.yaml` (3 grid cells)	EXPLORATORY	`499c0df` 10-24 23:30:55	23:31:32	18 BURST runs (§5.6)
`2026-10-25_native.yaml` (native E2)	FILLED	`b8c19ec` 10-25 17:31:16	17:31:56	18 runs; pass (§5.2)
`2026-10-24_grid_vllm-fcfs.yaml`	EXPLORATORY	`a78ce07` 10-25 18:06:18	18:06:33	6 BURST runs (§5.2)
`2026-10-30_vllm.yaml` (vLLM E2)	FILLED	`5a6fbcd` 10-30 16:23:59	16:26:35, after two jobs started	18 runs; pass; registered, not pre-registered
`2026-10-30_b100.yaml`, `_b25.yaml` (BASIN-1)	FILLED	`9128995` 10-30 19:51:20	20:04:30	6 BURST runs; pass (Table 17)
`2026-10-30_grid_sglang-lpminf.yaml`	EXPLORATORY	`5064580` 10-30 21:02:19	21:02:46	3 BURST runs; no collapse on the two informative seeds, no recovery (Table 18)
`2026-10-31_rescue.yaml` (R1), `_rescue_native.yaml` (R4), `_rescue_smoke.yaml`	FILLED	`f0bc3ad` 10-30 22:10:47	22:39:02	9 runs, submitted 23:01; R1 and R4 pass (§5.3); the smoke job is never decisive
`2026-10-31_b40.yaml`, `_b50.yaml`, `_b60.yaml` (BASIN-2)	FILLED	`621d88f` 10-31 01:58:38	02:10:48	9 runs, submitted 04:21; [pending: BASIN-2]
`2026-10-31_toolslow_d10.yaml`, `_d30.yaml`	FILLED	`b006291` 10-31 02:27:40	02:28:09	6 runs, submitted 04:21; [pending: TOOLSLOW]
`2026-10-31_shed_t600.yaml`, `_t3600.yaml`	FILLED	`b006291` 10-31 02:27:40	02:28:09	4 runs, submitted 04:21; [pending: SHED]
H200 cell	none			window $(1,\ 1.3)\ \lambda_{\text{ref}}$ too narrow: the rule's rates 1.10 and 1.20 fall outside its margin band $(1.1,\ 1.17)$, and the fill returns INCOMPLETE; no test
`2026-10-30_llama31.yaml` (Llama-3.1-8B E2)	FILLED	`1559f8e` 10-31 07:49:13	07:49:15	18 runs, submitted 07:50:02 (17170409–26); window $(1,\ 2.0)\ \lambda_{\text{ref}}$, 1.35 and 1.65 λ_{ref}, cap 20; [pending: Llama-3.1-8B E2]

Table 10: Order of events, 2026-10-21 to 10-26 (times CDT). Commits are those of the development repository (availability statement); commit and tag times are recorded by the committer, push times by GitHub's activity log, job times by Slurm.

When	Event
10-21	Stage 0 passes (17060165); Stage C submitted 15:28 (17062103–05)
10-22 12:54	Stage C results pulled
10-22 13:13	runbook proposes the rate rule and a cap of 20 (`288b801`)
10-22 13:14	E0a submitted (17072634–37); the last job ends 16:34
10-22 13:50	rate rule and cap adopted in the status notes; simulator cost model fitted to Stage 0/C server logs (`4435c51`)
10-22 17:03	Stage 0 and C data committed (`d0235a7`)
10-22 17:45	first push to GitHub, with the three commits above
10-23 14:32	E0a results committed (`a76f5e5`)
10-23 14:42	E0b submitted (17084750–55); jobs end by 16:09
10-24 03:13	validation of the calibrated simulator committed (`813cfc1`)
10-24 03:23	prediction spec and code committed before any prediction run (`630ab9e`); on GitHub 03:24
10-24 03:49	predictions B1–B4 committed (`64bc4e3`)
10-24 04:05	independent verification recorded (`877b936`)
10-24 14:52	tag `r6-predictions-frozen` (tagger time)
10-24 14:53	B1–B4 and the verification reach GitHub
10-24 15:06	E0b results pulled
10-24 15:09	E0b data and registration committed in the same second (`cfbc702`, `d279b7b`); registration on GitHub 15:09:54
10-24 15:13	E2 submitted (17093387–404)
10-24 15:38 to 10-25 02:36	E2 jobs run
10-25 02:28	native-pool Stage C and E0a start (17096856–58)
10-25 14:40	registered E2 verdict computed (data `49fbc7b`)
10-25 14:42	native Stage C curve analysed on the cluster; native curve and λ_{miss} in the status notes at 14:47 (`6793fb7`)
10-25 14:45	native E0b (17100938–39) and vLLM E0a (17100940) start
10-25 14:49	native prediction spec committed (`29728e3`)
10-25 14:52	native Stage C and E0a data committed (`fa265c2`)
10-25 14:54	B1n–B4n run on the clean code commit `0336c91`
10-25 15:06	B1n–B4n committed (`e3ea48f`); on GitHub 15:08:28
10-25 15:08	tag `r8-native-predictions-frozen` (tagger time); native rate rule in the status notes (`6cb43e9`)
10-25 17:26	native E0b jobs end; pulled 17:30
10-25 17:31	native E0b data and native registration committed (`f3f55ab`, `b8c19ec`); registration on GitHub 17:31:56
10-25 17:42 to 10-26 01:11	native E2 jobs (17102173–90)
10-25 18:06	vLLM calibration and exploratory registration committed (`65b65e2`, `a78ce07`), on GitHub 18:06:33; vLLM BURST submitted 18:07 (17102318–23)
10-25 22:42 to 10-26 03:24	vLLM BURST jobs
10-26 10:07	native E2 and vLLM BURST verdicts computed (data `f116cf3`)

Table 11: Order of the vLLM events on 2026-10-30 (times CDT, recorded as in Table 10).

When	Event
12:10	vLLM Stage C (17162959–61) and E0b (17162962–63) submitted; 17162959–60 start 12:11
12:16	rule for the vLLM cap and rates in the status notes, on GitHub (`836063a`)
12:31	B1v–B4v committed (`cc01dbf`); tag `r9-vllm-predictions-frozen` 12:32 (tagger time)
12:32	B1v–B4v reach GitHub (12:32:44)
14:36	first vLLM jobs end (Stage C, 17162959–60)
15:35	vLLM Stage C curve analysed on the cluster
16:23	bracket of the five finished vLLM E0b runs (16:23:19); vLLM registration committed (`5a6fbcd`, 16:23:59)
16:24	paired test submitted (17166121–38, 16:24:36); first job starts 16:25:42, second 16:26:17
16:26	vLLM registration and Stage C data (`276c5f0`) reach GitHub (16:26:35)
17:17	vLLM E0b job 17162963, with the 3.0 λ_{ref} run, ends
17:28	vLLM E0b data committed and pushed (`582c2a6`)

not in the repository, were checked afterwards by the orchestrating Claude Code session, which found no cluster command and no access to E0b before the predictions reached GitHub. For B2 this instruction-level control, checked after the fact, is the only safeguard. A scan of every line added between the commit the agents started from and the predictions found no use of withheld data, but a text scan cannot prove that nobody read it. B1 (E0a) rests on the same instruction only: E0a was committed before the predictions. The independent verifier, a separate OpenAI Codex session, had read the project status notes, which mention E0a, so it is independent but not blind; it also reused the prediction agents' copy of the repository instead of a fresh one, as its record states. Its evidence is in `provenance/codex_prediction_verification/`. Two conditions of the verification failed and are disclosed: the strict containment of B3 passes within the B2 all-stable range, and the platform-specific test failures of the full suite.

In the second round, B2n (native E0b) and B3n (native E2) reached GitHub while native E0b was still running, before any of its runs had finished, and before the native registration or any native E2 job existed. B3n is therefore blind in time. B2n reached GitHub 23 min after the two native E0b jobs were submitted, when both were running, so it is blind in the sense that no result existed, not in submission order. B1n and B4n are not: native Stage C and E0a had finished, and their headline numbers were in the project status notes before the round's specification was written. That specification is shared by all four, so B2n and B3n are blind to the native E0b and E2 outcomes but were specified after the native curve was known, and B3n's cap of 64 is B4n's output. The LLM agent that ran the round worked in a separate copy of the repository at a commit that predates the native data (`fb3510e`) and, by its own record, read nothing under the real-engine results

directories; that record has not been checked independently. No independent verifier reran the second round.

In the third round (`notes/PREDICTIONS_vllm.md`), an LLM agent in a separate copy of the repository at `a4bab1d`, which holds the exploratory vLLM BURST data but no vLLM closed curve or edge, ran the calibrated simulator with its token-proportional costs scaled by one factor per batching mode (1.0882 mixed, 1.0806 separate), chosen so that its all-miss token rate equals vLLM's measured 10,349 tokens/s; since the factor is fitted, B1v is not a prediction. By its own record, which has not been checked independently, it read nothing under the real-engine results directories; the two real vLLM numbers it used, λ_{miss} and the all-miss token rate, were given in its task. The predictions (`cc01dbf`, committed 12:31:58) reached GitHub at 12:32:44, 21 min 48 s after the five vLLM Stage C and E0b jobs were submitted (Table 11); two of these jobs had been running since 12:11 and none ended before 14:36:50. So B2v and B4v are blind in the sense that no result existed when they were pushed, not in submission order; B3v predates the vLLM registration and the paired test. The orchestrating Claude Code session that specified the round knew the outcome of the exploratory vLLM BURST runs, which were in the repository by then, and chose the candidate rates, so the BURST-arm predictions at 1.35 and 1.65 λ_{ref} are blind at most on the predicting agent's side. The notes' remark that their branch was never pushed predates the push. No independent verifier reran the third round. The vLLM registration's own timing is in Appendix B.

Rounds 11 to 14. Table 12 gives the time record of the later rounds. Round 11 froze the mechanism checks for the registered vLLM test (§5.2) and the simulator's predictions for the basin, fresh-seed, LPM-INF and rescue tests; round 12 the simulator's predictions for the Llama-3.1-8B cell, with its token costs scaled to the cell's measured all-miss rate (a fit); round 13 the first-principles model of §6; round 14 registered the BASIN-2, TOOLSLOW and SHED tests. LLM agents ran the simulator rounds in separate copies of the repository. The round-11 rescue predictions were revised after two adversarial reviews and before any rescue run, and every registered quantity was bit-identical before and after the revision (`notes/PREDICTIONS_r11_rescue.md`). The agents that derived the first-principles model read every committed result of the three measured cells and the registered protocols of BASIN-1, R1 and R4, and no H200, Llama, pool-sweep, basin, fresh-seed, LPM-INF or rescue result (`notes/PREDICTIONS_r13_theory.md`, "Blindness scope"). When they froze the basin and rescue predictions they also knew the simulator's burst-size sweep of §I.1 (bursts of 25 never trapped, 50 about half the time and 100 always) and the registered simulator rescue prediction (R1 rescued in 20 of 20 seeds at both rates). Their only H200 inputs were its E0a measurements. The agent that operated the H200 cell had shown its closed-curve values for $n \leq 32$ to the first author before the freeze; the agents that derived the model did not see them.

Table 12 also gives what each prediction is blind to. The simulator's BASIN-1 and fresh-seed predictions reached GitHub before those jobs were submitted, and its LPM-INF predictions before the LPM-INF jobs, so all three are blind in time. For the theory, no BASIN-1 run is blind in time. The three 25-session runs had started 1.7 to 2.2 h before the push (6230–7807 s of 8700 s elapsed, past the burst and into the window [5700, 8700) s), and the three 100-session runs had ended 0.2 to 1.6 h before it, so for all six the prediction is blind only by access: no one is known to have read them before the push, and the first recorded reading of any BASIN-1 or fresh-seed outcome was at about 04:07, but no file-access record can show this, because the cluster's class storage is mounted without access times. The same holds for the H200 cell's closed runs at n = 40, 48, 96 and 300 and its original E0b runs. They had ended before the freeze, and they were first read after it, when the curve and the bracket were combined at 02:30. The runs at n = 36, 44 and 52 were submitted after the freeze and are blind in time. No blind simulator prediction exists for the H200 cell. The second theory set (pred2) reached GitHub at 04:20:20, before the BASIN-2, TOOLSLOW and SHED jobs were submitted (04:21:40 to 04:21:43), so its predictions for them are blind in time. The 600k H200 jobs had been submitted at 02:50 but could not start before 05:00 and first started at 05:27, so its 600k predictions are blind in the sense that no result existed. Its table for the fresh seeds is not blind in time: it was committed at 04:20:05, after the last fresh-seed run had ended (03:57:56), although by its record its agents read no result. The simulator's rescue predictions (pushed at 21:30:40 on 10-30) and the model's (02:05:30 on 10-31) reached GitHub before the first rescue run started (02:23:44 on 10-31), so all nine rescue runs are blind in time to both; the smoke run ended before the model's push, but the model made no prediction for it, and by their record the agents that derived the model had no Nova access and opened no rescue output before the push.

C.1 Blind predictions against outcomes

B1 matched within 3.1% (Table 13), which mostly checks the simulator's bookkeeping (§4.4). B2's edge, 0.14–0.15 sessions/s, is consistent with the nominal seed-0 bracket [0.143, 0.179); near 0.179 the model keeps 2 of 3 runs stable (15 of 20 with more seeds), so one unstable run there is no contradiction, and the engine was stable there on the three fresh seeds of §5.1 (3 of 4 seeds stable in all). B3 predicted a pass at every tested rate in both batching modes. At the simulated rates that bracket each registered rate, the primary (mixed) mode's trapped token ratios, 1.003–1.027 at 0.09–0.10 sessions/s and 1.025–1.043 at 0.11–0.12, contain the observed 1.009–1.025 at 0.0966 and 1.025–1.029 at 0.1180; but at 0.12, the grid point nearest the higher rate, the prediction was 1.034–1.043, above what the engine did. The observed good-arm hit fractions (0.921–0.934) are close to, and at the low end partly below, the predicted 0.929–0.937. B3 flagged both ends of its grid as fragile. At the low end the slope margin $0.5(\lambda - \lambda_{\text{miss}})$ is small: with its registered three seeds it trapped 3 of 3 at both 0.09 and 0.10, but with 20

Table 12: Order of the round-11 to round-14 events, 2026-10-30 and 10-31 (times CDT; commit and tag times by the committer; submission, start and end times of jobs by Slurm's accounting, run end times by each run's `meta.json`). Push times marked * were read from GitHub's activity log, at the time (recorded in `notes/PREDICTIONS_r11_mechanism.md`, `notes/PREDICTIONS_r13_theory.md`, and `notes/PREDICTIONS_r13_pred2.md`) or on 10-31; they are not yet in the archived copy of the log in `provenance/`.

When	Event
10-30 18:20	r11 mechanism predictions for the vLLM test committed (`f1338e3`, 18:20:11) and pushed (18:20:13*); three of its 18 jobs running, none ended
10-30 18:55	first vLLM paired-test job ends (17166121, 18:55:07)
10-30 19:50	BASIN-1 and fresh-seed predictions (`c600fd7`); BASIN-1 registrations (`9128995`, 19:51); tag `r11-basin-reg` 20:04; all pushed 20:04:30*
10-30 21:01	LPM-INF predictions (`e2e8474`) and registration (`5064580`, 21:02); tag `r11-lpminf-pred` 21:02; pushed 21:02:46*
10-30 21:12	BASIN-1 and fresh-seed jobs submitted (17168500–07, 21:12:45–46)
10-30 21:28	rescue predictions (`72501f0`); tag `r11-rescue-pred` 21:30; pushed 21:30:40*
10-30 22:07	LPM-INF jobs submitted (17168743–45, 22:07:40)
10-30 22:10	R1 and R4 registrations (`f0bc3ad`); tag `r11-rescue-reg` 22:38; pushed 22:39:02*
10-30 22:28	the H200 operator reads the closed-curve values for $n \leq 32$ (17166818–22) and shows them to the first author
10-30 22:38 to 23:52	H200 closed runs at $n = 40$, 48, 96 and 300 end (17166828–34; not read until 02:30)
10-30 23:01	rescue jobs submitted (17168965–74, 23:01:12–13); the smoke job runs 23:24 to 00:08
10-31 00:14 to 01:08	H200 E0b jobs end (17168377–82; not read until 02:30)
10-31 00:20	last vLLM paired-test job ends
10-31 00:28 to 01:51	the three 100-session BASIN-1 runs end (00:27:56, 00:29:27, 01:50:54)
10-31 01:58	BASIN-2 registrations (`621d88f`); tag `r14-basin2-reg` 02:10; pushed 02:10:48*
10-31 02:05	theory frozen (`997d2c4`, 02:04:55) and pushed (02:05:30*); tag `r13-theory-pred`
10-31 02:05 to 03:58	the six fresh-seed runs end (the first at 02:04:59, 31 s before the theory push)
10-31 02:08	registered vLLM verdict computed on the cluster; pulled 02:09:56, committed 02:22 (`cacf833`)
10-31 02:14	Llama simulator predictions (`a62aea9`) pushed (02:14:06*); tag `r12-llama31-predictions-frozen`
10-31 02:20 to 02:47	the three 25-session BASIN-1 runs end
10-31 02:23 to 06:57	the nine registered rescue runs (17168966–74; the first starts at 02:23:44, 18 minutes after the theory push; the last ends at 06:57:06)
10-31 02:27	TOOLSLOW and SHED registrations (`b006291`); tag `r14-extra-reg` 02:27:59; pushed 02:28:09*
10-31 02:30	H200 nine-point curve and E0b bracket combined on the cluster: first reading of $n \geq 40$ and of the edge
10-31 02:33	H200 closed runs at $n = 36$, 44 and 52 submitted (17169612–14, 02:33:06); they end 03:25 to 04:01
10-31 02:50	H200 E0b reseeds (17169668–71, 02:50:20) and 600k jobs (17169672–80, 17169682–87, 02:50:32 to 02:51:20, not eligible to start before 05:00) submitted
10-31 03:49 to 04:21	the three LPM-INF runs end
10-31 04:07	first recorded reading of the BASIN-1 and fresh-seed outcomes (about 04:07)
10-31 04:20	second theory set for BASIN-2, TOOLSLOW, SHED and the H200 600k cell committed (`cfa7712`, 04:20:05) and pushed (04:20:20*); tag `r13-theory-pred2`
10-31 04:21	BASIN-2, TOOLSLOW and SHED jobs submitted (17169893–911, 04:21:40 to 04:21:43) from clones at the registration tags, by scripts that require the pred2 commit
10-31 07:05	rescue runs pulled (committed in `c6ea41e`); registered verdicts computed at 07:17 (`be50593`, `454ff7a`)
10-31 07:49	Llama-3.1-8B registration (`1559f8e`, 07:49:13) pushed (07:49:15*); its 18 E2 jobs submitted at 07:50:02 (17170409–26)

Table 13: Blind predictions (calibrated simulator, mixed batching unless noted) against outcomes; rates in sessions/s; native and vLLM rates in multiples of λ_{miss} (for the simulator its own, for the engine that cell's λ_{ref}). Percentages in the last column are predicted/real −1. B1 is not blind in time and is close to the calibration data (§4.4, Appendix C). B3 and B3n used a per-arm pass rule (each arm in 2 of 3, resp. 7 of 10, seeds); the registered rule asks for all three arms in the same seed, which B3n's runs also meet (9 of 10 seeds at its low mixed point, 10 of 10 elsewhere). B3's token-ratio and backlog ranges are over the two simulated rates that bracket each registered rate. B4 is the registered seed 0 on the grid n = 48, 64, 96 (supplementary 5-seed means: peak 0.514, $\mu(96)$ = 0.085, $\mu(300)$ = 0.108). B2n and B3n: 10 seeds ("sep.": separate batching); B3n's low and high points are 2.85 and 4.65 (sep. 2.65 and 4.35). B4v and B2v: vLLM, variant V1 (simulator token costs scaled to vLLM's all-miss rate), 10 seeds, turns-based rates. B3n is blind in time; B2n, B4v and B2v were pushed after their jobs were submitted, before any run had finished (§5.2); B4n (3-seed mean) is not blind. B3v: vLLM paired test, all four configurations (V0, V1; mixed, separate), 10 seeds; blind in time, its BURST part blind only on the predicting agent's side (§5.2).

	predicted	real
B1	λ_{miss} 0.0722 (turns)	0.0715 (+0.9%)
	11,220 tokens/s	11,243 (−0.2%)
	λ_{miss} 0.0694 (sessions)	0.0717 (−3.1%)
B2	all-stable edge 0.14 (3 seeds), 0.15 (20)	in [0.143, 0.179) (seed 0)
	at 0.18: 2/3 stable (15/20)	0.179: 3/4 seeds stable
B3	PASS at all rates 0.09–0.16	PASS at 0.0966, 0.1180
	20 seeds: sep. fails at 0.16	(0.16 not run)
	token ratio 1.003–1.027 (0.09–0.10)	1.009–1.025 (0.0966)
	1.025–1.043 (0.11–0.12)	1.025–1.029 (0.1180)
	backlog 171–234 (0.09–0.10)	194–226 requests
	253–313 requests (0.11–0.12)	277–325
	good-arm hit 0.929–0.937	0.921–0.934
	trapped 3/3 at 0.09 and 0.10;	trapped 2/3 at 0.0966
	20 seeds: 14/20, 18/20	
B4	peak n = 64, μ 0.523 (900k)	n = 64, 0.479 (+9%)
	0.5-hit crossing ≈80	≈93 (≈91 on B4's grid)
	$\mu(96)$ = 0.079, $\mu(300)$ = 0.109	0.123, 0.101
B2n	all-stable edge 6.5; sep. 6.0	in [5.5, 6.5)
	at 6.5: 10/10 stable; sep. 8/10	6.5 unstable
	at 1.5–5.5: 10/10 stable, both	stable
B3n	PASS at 2.85, 4.65; sep. 2.65, 4.35	PASS at 2.5, 4.0
	trapped token ratio 1.086–1.109 (low)	1.074–1.078 (2.5)
	1.119–1.140 (high)	1.106–1.124 (4.0)
	trapped backlog 460–643 (low)	549–594 requests
	1067–1333 requests (high)	1089–1130
	good-arm hit 0.928–0.936	0.929–0.935
	BURST trapped 9/10 to 10/10	3/3 at both
B4n	peak n = 72, μ 0.532	64–80, 0.479 (+11%)
	0.5-hit crossing 94; hit 0.42 at 96	≈93; 0.42
	cap 64, 0.35% inside the rule	64
B4v	peak n = 24, μ 0.215; cap 20	n = 20, 0.211 (+2%); 20
	hit 0.74 at n = 28, 0.46 at 32	0.66, 0.08
	0.5-hit crossing 31.4	29.1
B2v	all-stable edge 2.0; at 2.5: 7/10	in [2.0, 2.5); 2.5 unstable
B3v	PASS at 1.35, 1.65 (all four)	PASS at both
	BURST trapped 7–9/10	2/3 at each
	NOBURST, CAPPED(20) 10/10	3/3 and 3/3 at each
	trapped token ratio 0.99–1.07	0.948–1.018

seeds 14 and 18, so 10–30% of runs there are stuck without meeting the slope test; one indeterminate real run in three at 0.0966 is consistent with that. At 0.16 the 20-seed supplement of the secondary (separate) mode failed the pass rule, with NOBURST good in 13 of 20 seeds, one short; the engine was not run there. (Backlogs compare only roughly: the simulator counts waiting requests, the engine runs outstanding ones.) B4 got the peak's position, within its coarse grid, and the outward shift of the cliff; the predicted peak was 9% above the real one and the cliff early: on B4's grid of 64 and 96 the real hit fraction crosses 0.5 near 91, against B4's 80, and at n = 96 the engine still hit 42% of turns, against B4's 5%. The early cliff is mostly the pool: B4n, the same simulator at the real pool (not blind), hit 0.42 at n = 96 and crossed 0.5 near 94. The peak error is not: at n = 64, where nothing is evicted, seed 0 gives the same 0.523 in both pools. Two qualifications: the verifier's strict check that every B3 pass lies inside the B2 all-stable range failed (B3 asks for 2 of 3 seeds over a longer run), and B2 is blind only by instruction (checked afterwards from the agents' transcripts) and by commit order, not by event order: E0b had finished on the cluster before the predictions were committed, and was pulled 13 minutes after their commit reached GitHub.

The second round, for the native pool, held in part: B3n's pass held at the registered rates, which lie 6–16% below its own test points, with trapped token rates, backlogs and good-arm hit fractions consistent with the lower rates; the primary mode of B2n put the from-empty edge at least one grid step too high; and B1n and B4n are not blind (Appendix G). For vLLM, B4v and B2v got the peak level, the cap and the edge right but predicted a later and gentler cliff, and B3v's pass held (§5.2).

Slips in the registration record. The vLLM registration reached GitHub after the first two of its jobs had started (§4.3), and the memory request of its 15 pending jobs was lowered after submission (§5.2). B1n and B4n were specified after the native curve was known, and B2n, B2v and B4v were pushed after their jobs were submitted (Appendix C). The agent that operated the H200 cell saw its closed-curve values for $n \leq 32$, which include the predicted peak, before the freeze and showed them to the first author; the agents that derived and froze the model did not see them, so the H200 peak-ratio rule and hit(32) are not blind to the first author, while the runs at n = 36, 44 and 52 are blind in time and the other runs at $n \geq 40$ and the original from-empty runs, which had ended before the freeze, are blind by access only; four H200 E0b reseeds were added after the first H200 E0b had been read (§7). No blind simulator prediction was made for the H200 cell. The 100-session runs of BASIN-1 had ended 0.2–1.6 h before the model's predictions were pushed, and the 25-session runs were then 6230–7807 s into their 8700 s; no one is known to have read them before then, but the cluster's file system records no access times, so all six predictions are blind by access only. The H200 runs at n = 36, 44 and 52 and the fresh-seed from-empty runs ended before pred2 was pushed, so pred2's tables for them are not blind in time (pred2's freeze

record, notes/PREDICTIONS_r13_pred2.md, wrongly says it was pushed before the first job of the H200 runs at $n = 36$, 44 and 52: those jobs were submitted at 02:33 and had ended before the push); the BASIN-2, TOOLSLOW and SHED jobs were submitted after it (pushed at 04:20:20 CDT; jobs submitted 04:21:40–43). The agents that froze the basin and rescue predictions of the model knew the simulator's burst-size sweep and its registered rescue prediction. The engage time of the native rescue test was chosen after a simulator prediction for an earlier engage time had been seen, before any rescue run. The bands of the frozen vLLM mechanism checks were chosen on SGLang runs and on exploratory vLLM runs whose arrivals the registered BURST arm repeats, so only its no-burst and capped arms are out of sample for those checks. No registration or prediction is in a third-party registry, and none has a timestamp other than GitHub's push log of the development repository, which is private, so readers see that log only in our export.

D Provenance

Every number in the paper is listed with its source file and commit or job in paper/NUMBERS.md. Table 14 gives the data directory and commit behind each result. The commit hashes cited are those of the development repository; the public repository's history is a redacted copy of it, and provenance/registration-proofs/ maps each cited registration and prediction commit to its public counterpart (availability statement). The 103 commits made before 17:45 CDT on 2026-10-22, including 288b801, 4435c51 and d0235a7 cited here, predate a history rewrite on that day that removed the AI co-author trailers from their commit messages and changed nothing else: trees, authors, dates and parent relations are unchanged. provenance/commit-map.tsv maps their old hashes, which notes and run records written before the rewrite cite, to the current ones; the repository the old hashes belong to is not on GitHub, so they can be checked only against this map. Their earliest server record is the repository's first push to GitHub, at 17:45:50 CDT on 2026-10-22, with which 97 of them arrived, these three among them, before E0b was submitted on 2026-10-23; one more (fc870c2) arrived 2 s later, and five are reachable only from archive tags, which have no push record.

Repository, redactions and timing evidence. Its history is a redacted copy of the authors' development repository: passages unrelated to the paper were removed from four notes and agent-instruction files and one clause from each of two commit messages (all listed in provenance/REDACTIONS.md), and no code, configuration, registration, result or paper file changed in it. A final release commit adds licences and README text and replaces allocation names in some notes, job-script comments and tests (provenance/). Its commit hashes therefore differ from those cited in this paper, which are the development repository's. The directory provenance/ maps every cited hash to its public counterpart and holds the timing evidence: GitHub's push log of the development repository (github_activity_2026-10.json), the external record of when each registration and prediction reached GitHub (§4.3, Appendices B and C); for every registration and frozen prediction, the original commit, tree and tag objects, the push-log event that first carried them, and a script that recomputes the hash chains and checks that each registration file in the public repository is byte-identical to the registered one (provenance/registration-proofs/; [pending: the registrations and predictions pushed after 2026-10-30 are added at the final build]); the cluster's accounting records of the cited jobs ([pending: refreshed accounting export for the jobs after the vLLM paired test]); and the evidence of the independent verifier of the first prediction round. Simulator outputs are not stored, apart from their summaries; each can be regenerated from the command, commit and configuration recorded with the run (meta.json, config.yaml). The derived tables in paper/derived/ name each engine run by its path on the cluster, results/agentload_*/<timestamp>; the run is stored under the pull directory of its jobs in results/nova/, with the same final path components. The agents' tool-call transcripts (§4.4, Appendix C) are not in the repository.

Derived numbers. Quantities that no committed analysis reports directly are computed from committed run data by figures/paper/derived_numbers.py and written to paper/derived/: the CAPPED drain times; the BURST backlog after the burst and over the window; the E0b hit fractions and realized arrival rates; the arm counts and the post-hoc sign test that counts each seed once; per BURST run, the hit fraction before the burst and the first 100 s window from which no turn hit; and, per run of every arm, the client-side cost (gate waits, turn latency, unfinished sessions) and the realized arrival and completion rates over the window. They cover E0b, E2, native E0b, native E2, vLLM E0b and the exploratory vLLM BURST runs. Each derived file names a run both by its path on the cluster and by its path from the repository root.

Notes that support the text. The math check of the model, notes/MATHCHECK_thrashing_curve.md, was done by an LLM agent on 2026-10-21; it predates the present assumptions (A1)–(A4) and Lemma 1, and where it differs from §3 and Appendix E, those hold; the verification of Stage C (three-source hit check, eviction survival times, prompt continuity) is notes/STAGEC_VERIFICATION.md; the audit of the PEEK, Continuum and CONCUR rules against released code is notes/BASELINES_AUDIT.md; an audit of the citations as of 2026-10-25, the scoop check and a first reading of the SGLang and vLLM sources are in notes/lit/CITATION_AUDIT_2026-10-25.md (that reading went twice through a page summariser, a web tool that returns a language model's summary of each file, without verbatim code; the line references of §2 and Appendix E were later read verbatim in the files at the tags), and that audit predates the citations added since, among them Yuan et al. [102], which it lists as not cited; the trace statistics are in notes/lit/AGENT_TRACES.md.

Table 14: Data behind each result (paths relative to the repository root at tag `arxiv-v1`).

Result	Directory (commit)
Stage 0, C	`results/nova/s0_smoke_17060165_pulled_20261021-152347/`, `results/nova/stageC_1706210x_pulled_20261022-125454/` (`d0235a7`)
E0a	`results/nova/e0a_1707263x_pulled_20261023-142037/` (`a76f5e5`)
E0b	`results/nova/e0b_1708475x_pulled_20261024-150639/` (`cfbc702`)
E2	`results/nova/e2_170933xx_pulled_20261025-144558/` (`49fbc7b`)
native, grid, vLLM	`results/nova/r7_native_grid_vllm_pulled_20261025-145126/` (`fa265c2`)
native E0b	`results/nova/e0b_native_1710093x_pulled_20261025-173038/` (`f3f55ab`)
vLLM E0a	`results/nova/e0a_vllm_17100940_pulled_20261025-180814/` (`b273241`)
native E2, vLLM BURST	`results/nova/e2_native_vllm_burst_pulled_20261026-101256/` (`f116cf3`)
vLLM C	`results/nova/stageC_vllm_1716295x_pulled_20261030-153621/` (`276c5f0`)
vLLM E0b	`results/nova/e0b_vllm_1716296x_pulled_20261030-172810/` (`582c2a6`)
validation	`results/r6_validate_stagec/20261024-025222/` (`813cfc1`)
B1–B4	`results/r6_predict_report/20261024-034707/`, `notes/PREDICTIONS.md` (`64bc4e3`)
B1n–B4n	`results/r8_native_report/20261025-150146/`, `notes/PREDICTIONS_native.md` (`e3ea48f`)
B1v–B4v	`results/r9_vllm_report/20261030-122723/`, `notes/PREDICTIONS_vllm.md` (`cc01dbf`)
vLLM E2	`results/nova/e2_vllm_1716612x_pulled_20261031-020956/` (`cacf833`); mechanism checks in its `r11_mechanism/` and `derived/`
theory	`results/theory/merged/`, `notes/PREDICTIONS_r13_theory.md`, `notes/THEORY_r13_scoring.md`, `configs/r13_theory_predict.yaml` (`997d2c4`, tag `r13-theory-pred`); `results/theory/paper/`, `figures/theory/` (`5271ff6`)
H200	`results/nova/h200_wave1_pulled_20261031-023416/` and `prereg/inputs/2026-10-31-h200/` (`d108056`); `results/nova/h200_stageC_extra_1716961x_pulled_20261031-040933/` (`748900b`); `results/nova/h200_e0b_reseeds_1716966x_pulled_20261031-050308/` (`4e27364`)
BASIN-1, E0F	`results/nova/b1_e0f_1716850x_pulled_20261031-040517/` (`4dd21e0`)
LPM-INF	`results/nova/lpminf_1716874x_pulled_20261031-042548/`, `results/r11_lpminf_eval/20261031-eval/` (`fc8b028`)
rescue R1, R4	`results/nova/rescue_1716896x_pulled_20261031-070502/` (`c6ea41e`); verdicts `results/agentload_analysis_rescue_r1/20261031-071720/` (`be50593`) and `results/agentload_analysis_rescue_r4/20261031-071748/` (`454ff7a`) in it; descriptive tables `results/rescue_paper_data/20261031-071938/` in it (`51c803d`); registrations `configs/agentload_prereg_2026-10-31_rescue*.yaml` (`f0bc3ad`); predictions `results/r11_predict_rescue/20261030-195231/` (`72501f0`)
Proposition 4 pred2	`results/theory/pred2/prop_e_cor_c.json` (`cfa7712`, tag `r13-theory-pred2`)
rounds 11, 12	`results/r11_predict_basin/20261030-194539/` (`c600fd7`), `results/r11_predict_lpminf/20261030-194911/` (`e2e8474`), `results/r11_predict_rescue/20261030-195231/` (`72501f0`), `results/r12_llama31_report/20261031-021057/` (`a62aea9`)
gap/R_M	`paper/derived/loopvars_*.csv` (frozen script of `f1338e3`)
robustness	`results/r7_report/20261024-233554/` (`c653498`); `results/r7_traces_sim/r7_report/20261025-012451/`, `results/r7_traces_sim_adj/r7_report/20261025-012453/` (`b530e55`)
mechanisms	`results/prefix_meta_baselines_audit_summary/20261025-001850/` (`dc76032`); `results/prefix_meta_r4_curve/20261021-050856/`, `results/prefix_meta_r4/20261021-051550/` (`a1d485b`); `results/prefix_meta_mech/20261021-025902/` (`18ff187`)
push log, jobs	`provenance/`
figures	`figures/paper/`, `figures/rescue_paper_data.py`, `paper/figs/README.md`
derived	`paper/derived/`, `figures/paper/derived_numbers.py`

The theory's derivations and their review are `notes/THEORY_r13_merged.md`, `notes/THEORY_r13_fluid_rigorous.md` and `notes/THEORY_r13_che_endogenous.md`; where they differ from §6 and Appendix F, those hold. Where the research log and a committed CSV differ in the last digit (three hit fractions and one backlog of the policy grid), the paper uses the CSV.

E Proofs of the quasi-static results

The congestion bound. Section 3.4 states the following proposition in brief.

Proposition 6 (Request-level FCFS, any online eviction rule)**.** *Consider the closed system with n sessions under any online eviction rule, one that does not foresee tool times, and assume that its long-run rates exist and are deterministic (Appendix E) and that:*

(A1) sessions share no cached tokens, each has at most one outstanding request, and evicted KV is discarded, not offloaded to another memory tier; tool times are i.i.d. with finite mean, independent of all other randomness, and unknown to the engine until they end;

(A2) admission is request-level FCFS: no request is admitted for the first time before a request that was already waiting when it joined the queue. More generally, it suffices that the order meets the condition of Appendix E for some Q_0 and S_0 independent of n, which allows reordering while at most Q_0 requests wait and lets a follow-up pass at most S_0 requests that were waiting when it joined. At most B requests run at once, and at most $\alpha t + \beta$ admissions, re-admissions after a preemption included, occur in any interval of length t, with B, α and β independent of n;

(A3) in congestion the engine computes prompt tokens no faster than with caching disabled: $\limsup_{n\to\infty} u_n \le C$*, where u_n is the long-run rate of prompt tokens that are computed rather than served from cache when their request is first admitted (each prompt token is counted once, so tokens recomputed after a preemption are not), and C is the limit of that rate as $n \to \infty$ for the closed system with caching disabled.*

Then the long-run rates of cache-served prompt tokens and of hit turns tend to 0 as $n \to \infty$, so does the hit fraction if $\liminf_{n\to\infty} \mu(n) > 0$*, and* $\limsup_{n\to\infty} \mu(n) \le \lambda_{\text{miss}}$*. If also (A4) $u_n \to C$, that is, the congested engine stays saturated and computes prompt tokens as fast as with caching disabled, then $\mu(n) \to \lambda_{\text{miss}}$, so the hit fraction tends to 0.*

The proof below rests on Lemma 1. The idea is that a follow-up can reuse a cached token only if the token stays in the pool while the follow-up waits, which bounds reuse by M over the wait whatever the eviction rule, and that request-level FCFS makes short waits rare as n grows, because the engine cannot foresee tool times.

Remark (resident-first orders; a conjecture). Under pure LPM (without SGLang's 128-request fallback) in congestion we expect the follow-up hit fraction to tend to $P(Z < R_\infty)$, with R_∞ the congested lifetime of idle KV, so μ_∞^{LPM} exceeds λ_{miss} through tool calls shorter than R_∞; the simulator agrees (§I.2), but we have no proof. That service in arrival order sustains a metastable state is known from other systems [10, 36], and serving the newest requests first once a queue forms is deployed practice [68]; favouring a job whose cache footprint is still resident is classical affinity scheduling [85]; FCFS and LPM under prefix reuse are compared analytically by Dexter et al. [30]; and PEEK documents the 128-request fallback [95]. The exploratory LPM-INF test of §7 disables that fallback on the engine.

Model and conventions. In the closed system of §3.1, n sessions are present and a finished session is replaced at once by a fresh one, whose first request joins the queue. At time 0 every session is fresh, so no tool call is in progress, as in the synchronized start of our closed runs. A request joins the queue when its session sends it, waits, and is admitted; it leaves the queue only by admission. An admitted request runs until it completes or, if the engine preempts it, rejoins the queue. At a common instant joins are processed before admissions, as when an engine receives requests before each scheduling pass, and requests that join at the same instant enter in some order; the requests waiting when request i joins are those in the queue just before it joins and those that join at the same instant ahead of it. When a request that is not its session's last completes, the session starts a tool call, and when the call ends the session's next request, a follow-up, joins the queue. By (A1) evicted KV is discarded, not offloaded. A follow-up's wait runs from its joining the queue to its first admission, and its reuse, and whether it hits, are counted at that admission. SGLang logged no retraction in any of the 73 server logs of our SGLang runs. When a running request cannot get a block for its next tokens, vLLM preempts the most recently admitted running request and returns it to the head of its queue (*Other orders* below); its closed runs logged 12 to 72 preemptions per run from $n = 32$ on (72 at $n = 300$, in a run that completed 5,395 requests), and its cache-disabled reference 9 or 10 per 32-session run. Since u_n and C count each prompt token once, at its request's first admission, recomputation after a preemption enters neither, and the proofs below go through with re-admissions counted among the admissions that (A2) bounds. By (A1) each session is at every moment in exactly one of three states: its request waits, its request runs, or it is in a tool call. Tool times have law F_Z and mean $\bar{z} < \infty$. A long-run rate is $\lim_{t\to\infty} A(t)/t$ for a count or sum $A(t)$ over $[0, t]$; we assume that these limits exist almost surely and are deterministic, as for a positive Harris recurrent Markov process from any initial state, and that λ_{miss} and C, the limits of $\mu(n)$ and u_n as $n \to \infty$ with caching disabled, exist. Every $A(t)/t$ used below is bounded for fixed n and $t \ge 1$, since at most $\alpha t + \beta$ requests are admitted by time t, so by bounded convergence each rate also equals $\lim_{t\to\infty} E[A(t)]/t$, and bounds on expectations carry over to rates.

Lemma 1 (short waits are rare)**.** *Assume (A1) and (A2), with*

Q_0 and S_0 as in (A2) (both 0 for request-level FCFS). Let $\gamma_n(K)$ be the long-run rate of follow-ups that wait less than K, and let $\kappa = \lfloor \alpha + \beta \rfloor + B$, $y = \kappa(1 + \bar{z})$ and $m = n - B - \alpha K - \beta - Q_0 - S_0$. If $m > \kappa$, then

$$\gamma_n(K) \le \frac{y + y^2 + \kappa y}{m - \kappa},$$

so $\gamma_n(K) = O(1/n)$ for every fixed $K > 0$.

The general ordering condition of (A2), the only property of the order that the proof uses, is this: if more than Q_0 requests wait at every moment from a follow-up i's joining the queue to its first admission, counting at a moment with admissions the requests that wait just before them, then all but at most S_0 of the requests that were waiting when i joined are admitted, for the first time or again, no later than that admission. Request-level FCFS meets it with $Q_0 = S_0 = 0$, since i itself waits.

Proof. For a follow-up i write q_i for the time it joins the queue, D_i for its wait and Q_i for the number of requests waiting when it joins, and let $X(t)$ be the number of sessions in tool calls at time t.

A short wait needs a short queue. Let $D_i < K$. At most $\alpha K + \beta$ admissions occur in $[q_i, q_i + D_i]$, i's own included, so fewer than $\alpha K + \beta$ occur before i's. If $Q_i \ge \alpha K + \beta + Q_0 + S_0$, then at every moment of $[q_i, q_i + D_i]$, counted as above, more than Q_0 of the Q_i requests still wait, because requests leave the queue only by admission; so at least $Q_i - S_0$ of them are admitted no later than i, and $Q_i - S_0 + 1 \le \alpha K + \beta$, a contradiction. Hence $Q_i < \alpha K + \beta + Q_0 + S_0$.

A short queue needs almost every session in a tool call. Just before q_i each of the other $n - 1$ sessions is waiting (at most Q_i of them), running or in a tool call, and i's own session is in a tool call or, if that call has length 0, running; at most B sessions run. So $X(q_i-) \ge n - Q_i - B > m$ whenever $D_i < K$.

From the instants at which tool calls end to fixed times. Follow-ups join the queue when tool calls end, which happens more often when many sessions are in tool calls, so an average over these instants is not a time average; we avoid it as follows. A request that completes in an interval $(s, s+1]$ was admitted, for the last time, in that interval or was running at time s, so at most κ requests complete, and at most κ tool calls start, in any such interval. Fix an integer $j \ge 0$. A follow-up that joins the queue in $(j, j+1]$ ends a tool call that was in progress at j or started in the interval, so at most $X(j) + \kappa$ follow-ups join in it, and $X(q-) \le X(j) + \kappa$ for every $q \in (j, j+1]$. By the previous step, the number of follow-ups that join in $(j, j+1]$ and wait less than K is at most $g(X(j))$, where $g(x) = (x + \kappa)\,\mathbf{1}\{x > m - \kappa\}$.

A bound on $X(t)$ that does not depend on n. Fix t and split $(-\infty, t]$ into the intervals $I_k = (t - k - 1, t - k]$, $k \ge 0$; no tool call starts before time 0, and at most κ start in each I_k. Let $(Z_{kl})_{k \ge 0,\, 1 \le l \le \kappa}$ be i.i.d. with law F_Z, independent of everything else. Give the l-th call that starts in I_k the duration Z_{kl}, and calls that start after t fresh durations. This does not change the law of the system: by (A1) each call's duration is independent of all other randomness, and the system's evolution up to any time depends on a call's duration only through whether the call has ended by then; the index (k, l) of a call is fixed when the call starts, so the variable it receives has not been used and is independent of the past at that moment, and the two constructions agree in law by induction over successive events. A call that started in I_k and is still in progress at t has a duration greater than k, so in this construction

$$X(t) \le Y = \sum_{k \ge 0} \sum_{l=1}^{\kappa} \mathbf{1}\{Z_{kl} > k\},$$

whose law depends on neither t nor n. Y is a sum of independent indicators with $E[Y] = \kappa \sum_{k \ge 0} P(Z > k) \le \kappa(1 + \bar{z}) = y$, so $\operatorname{Var} Y \le E[Y]$ and $E[Y^2] \le y + y^2$.

Conclusion. Since g is non-decreasing and $\mathbf{1}\{Y > m - \kappa\} \le Y/(m - \kappa)$,

$$E[g(X(j))] \le E[g(Y)] \le \frac{E[Y^2] + \kappa E[Y]}{m - \kappa} \le \frac{y + y^2 + \kappa y}{m - \kappa}.$$

Summing over the unit intervals of $(0, t]$, for integer t, bounds the expected number of follow-ups that join by time t and wait less than K by t times this quantity; dividing by t and letting $t \to \infty$ gives the claim. □

The bound is of order $1/n$, but its constants grow with the admission rate and the mean tool time, so it is a statement about the limit, not about the populations we ran.

Proof idea. By (A1), the r_i prompt tokens that follow-up i reuses at its first admission are its session's own and stay in the pool from the end of the session's previous turn until that admission, an interval no shorter than its queue wait D_i, and the token-time that these intervals claim does not overlap. Resident token-time accrues at rate at most M, so follow-ups that wait at least K reuse tokens at rate at most M/K, whatever the eviction rule. By (A2) a follow-up that waits less than K joined behind fewer than $\alpha K + \beta + Q_0 + S_0$ requests, so just before it joined more than $n - B - \alpha K - \beta - Q_0 - S_0$ sessions were in tool calls; Lemma 1 (Appendix E), which uses that the engine does not see tool times, shows that such follow-ups occur at a rate of order $1/n$, and each reuses at most $L_{\max} = 40{,}350$ tokens. Letting $n \to \infty$ and then $K \to \infty$, the cache-served token rate tends to 0, and since a hit reuses more than $L_0/2$ tokens, so does the rate of hits. Counted once, at its request's first admission, each of a session's W prompt tokens is either served from cache or computed, so u_n is $\mu(n)E[W]$ less the cache-served token rate, and with caching disabled the same count gives $C = \lambda_{\text{miss}} E[W]$; (A3) gives the bound and (A4) the limit. The same count gives turns at the rate $\mu(n)E[T]$, so the hit fraction tends to 0 whenever $\mu(n)$ stays bounded away from 0.

Proof of Proposition 6. Reuse needs waiting token-time. Let follow-up i of session s join the queue at q_i, wait D_i, and reuse r_i prompt tokens from cache at its first admission. By (A1) these tokens are s's own, computed by its earlier requests, and evicted KV is discarded, so each of them was in the pool throughout $[q_i, q_i + D_i]$, during which s has no running

request to recompute it. These pairs of token and time are distinct for different requests: sessions share no tokens, and the waits of one session's requests do not overlap. At most M tokens are resident at any time, so for every t, $\sum_i r_i D_i \le Mt$ over the follow-ups first admitted by t. This step uses neither the order nor the eviction rule, so it holds for clairvoyant rules too.

Reuse vanishes. Fix $K > 0$. By time t the follow-ups that waited at least K reused at most Mt/K tokens, and each of the others at most $L_{\max} = 40{,}350$, the longest prompt. So the long-run rate ρ_n of cache-served prompt tokens satisfies $\rho_n \le M/K + L_{\max}\gamma_n(K)$, and Lemma 1, with the order's Q_0 and S_0, gives $\limsup_{n\to\infty} \rho_n \le M/K$ for every K, so $\rho_n \to 0$. A hit reuses more than half its prompt, hence more than $L_0/2$ tokens, so hits occur at a rate of at most $2\rho_n/L_0 \to 0$.

Work. A session with T turns sends $W(T)$ prompt tokens (§3.1), and each, counted once at its request's first admission, is either served from cache or computed. The closed system has n slots, each holding one session at a time and a fresh one after it finishes, so each slot holds a sequence of sessions with i.i.d. turn counts, and by the strong law of large numbers completed sessions bring prompt tokens at the long-run rate $\mu(n)E[W]$ and turns at the rate $\mu(n)E[T]$; sessions in progress hold at most $nW(60)$ tokens and $60n$ turns and do not affect rates. Hence $u_n = \mu(n)E[W] - \rho_n$. With caching disabled nothing is reused, so the same count gives $C = \lambda_{\text{miss}}E[W]$. So $\mu(n) = (u_n + \rho_n)/E[W]$, and (A3) gives $\limsup_{n\to\infty} \mu(n) \le C/E[W] = \lambda_{\text{miss}}$. The hit fraction is the rate of hits over the rate of turns, so it is at most $2\rho_n/(L_0\,\mu(n)E[T])$ and tends to 0 whenever $\liminf_{n\to\infty} \mu(n) > 0$. Under (A4), $\mu(n) \to \lambda_{\text{miss}} > 0$. □

Scope of Proposition 6. The proof's last step, from vanishing reuse to λ_{miss}, is the bound that MISA-T notes at prefill saturation, a request rate of at most the prefill throughput over the mean uncached prompt tokens per request [45]; what the proposition adds is that request-level FCFS makes reuse vanish in congestion whatever the online eviction rule. (A2) describes an admission pass that stops at the first request that does not fit, as in our simulators and, on our reading of their sources, in SGLang and vLLM under their default FCFS policies (Appendix E); vLLM also preempts running requests and returns them to the head of its queue, which (A2) allows, because its ordering clause concerns first admissions; with asynchronous scheduling it also lets a follow-up pass a bounded number of just-preempted requests, so it meets the general condition of (A2) with $Q_0 = 0$ and a bounded S_0 (Appendix E). A diverging mean cycle time would not be enough: Lemma 1 uses bounded admissions and i.i.d. tool times to bound the tail of the number of sessions in tool calls independently of n, which makes it rare that almost all sessions are in tool calls when one returns. The lemma's bound is of order $1/n$ but has large constants, so the proposition describes the limit, not the populations we ran. The lemma needs FCFS order only while more than Q_0 requests wait, so, on our reading of its source, the proposition also covers SGLang's stock LPM, which stops re-sorting its queue whenever more than 128 requests wait, so that new requests queue behind all waiting ones ($Q_0 = 128$). It does not cover program-level FCFS, which puts an old session's follow-up ahead of newer sessions' requests however long the queue: in our earlier simulator its hit fraction stays near 0.03 from $n = 64$ to 300, while FCFS has none. A pure resident-first order, such as LPM without the fallback, violates (A2) outright: a returning session whose KV is still resident is admitted ahead of the queue.

(A1) holds for our random-token sessions, apart from rare one-token coincidences (5 of the 19,924 first turns in E2 had one cached token), but not for agents that share a system prompt: one resident copy then serves every session. We expect the argument to apply to the private suffixes, with λ_{miss} and C measured with the shared prefix cached, but have not shown this. (A1) also rules out offloading evicted KV to host memory, which would let a follow-up reuse tokens that held no pool space while it waited. (A3) and (A4) hold on our engines only approximately: with zero hits, closed runs and the registered BURST runs computed prompt tokens at 0.95–1.12 times the cache-disabled rate C, and at 1.03 times it near the stationary turn mix, so under (A4) the proposition puts μ_∞ about 2–5% above λ_{ref}, within the standard error of λ_{ref} (Appendix E gives the measurements).

The bound holds for every online eviction rule, including protection of queued prefixes and TTL pins that keep request-level FCFS order, where (A4) can fail; in our earlier simulator both still hit 10–23% of turns at $n = 300$, so a finite-n hit fraction does not by itself separate covered orders from uncovered ones (Appendix E). Belady's offline rule [7] foresees tool times and is outside (A1): the token-time step of the proof covers it, Lemma 1 does not.

Checks of (A3) and (A4) on our engines. (A3) and (A4) treat the engine as a server of prompt tokens, as E0a suggests (§5.1). They concern long-run rates of the closed system, in which every order serves turns in the stationary mix, since each completed session contributes all its turns; our measurement windows did not have that mix, and the token rate varied between them, so we can check (A3) and (A4) only approximately. We estimate C by E0a's uncached token rate at 32 sessions, 11,243 tokens/s. With zero hits, closed runs computed prompt tokens at 1.03 C at $n = 96$, whose served prompts were within 3% of the stationary mean, on SGLang and on vLLM (against vLLM's own C), at 0.99 C at $n = 48$ (prompts 12–14% longer) and at 1.10–1.11 C at $n = 300$ (28–30% shorter); the registered SGLang BURST runs computed 1.01–1.03 C at 300k (prompts 9–20% shorter) and 1.07–1.12 times the 32-session all-miss rate of the native pool (prompts 29–41% shorter). So on our engine (A3) and (A4) hold only approximately: to within a few percent at the stationary mix, and to within 12% over the windows we measured. We estimate λ_{miss} by λ_{ref}, E0a's turns-based rate at 32 sessions (0.0715 sessions/s); λ_{ref} and $C/E[W]$ estimate the same cache-disabled limit. $C/E[W]$ is 0.0728 sessions/s, 1.7% above λ_{ref}, because the turns in E0a's window were longer than the stationary mix (10,648 against 10,469 prompt tokens on average). E0a's 64-session check computed 1.7%

more tokens per second than the 32-session runs and 3.2% more than the same seed at 32 sessions, although its window held longer turns (11,456 prompt tokens on average), so the 32-session reference falls short of saturation by about 2–3%, perhaps more. With the 64-session rate in place of C, $C/E[W]$ is 1.034 λ_{ref}, and with the zero-hit rate at $n = 96$ (11,602 tokens/s) it is 1.050 λ_{ref}. Under (A4) the proposition therefore puts μ_∞ about 2–5% above λ_{ref}, an offset within the standard error of λ_{ref} over seeds (0.0038 sessions/s, 5%); on vLLM the three estimates are 1.020, 1.038 and 1.050 λ_{ref} ($\lambda_{\text{ref}} = 0.0657$ sessions/s).

Covered orders at finite n. In our earlier simulator, under FCFS order, a hard pin of queued prefixes gives 0.919 λ_{miss} at $n = 300$ (§I.2) but 1.04–1.15 λ_{miss} at $n = 48$–150 (3 seeds), as a limit statement allows, and TTL pins of 5–20 s, which that simulator keeps past expiry while the session's next request waits, give 0.10–0.29 λ_{miss} at $n = 300$ with the earlier release rule of §I.2, with the GPU busy only 8–33% of the time (3 seeds). Both still hit 13% and 10–23% of turns at $n = 300$, so a finite-n hit fraction does not by itself separate covered orders from uncovered ones. The proposition makes their hit rate vanish as n grows, and their hit fraction too unless their throughput tends to 0, as the TTL pins' may; the hard pin keeps the GPU busy, and its hit fraction falls from 0.23 at $n = 48$ to 0.13 at $n = 300$. The TTL rows of §I.2 that exceed λ_{miss} also serve pinned programs first, an order that (A2) excludes.

Other orders. The proof uses the order only through Lemma 1, so it holds for any order that meets the condition above for some Q_0 and S_0. In SGLang 0.5.22 the admission loop walks the waiting queue in order and stops at the first request that cannot be added or when the batch is full (`python/sglang/srt/managers/scheduler.py`, lines 4080–4176 at tag v0.5.22); it skips a request only for LoRA, pooling-mode or storage-prefetch conditions that our runs do not use. New requests join the queue at its tail (line 3366), as would a retracted request (lines 4369–4416). FCFS sorts the queue only under priority scheduling, which none of our runs enabled (`python/sglang/srt/managers/schedule_policy.py`, lines 227–231), and LPM sorts it only while at most 128 requests wait (lines 133 and 300–311); with more waiting, LPM takes the FCFS branch, which leaves the queue's order unchanged (lines 260–261), so new requests queue behind all waiting ones. So FCFS meets the condition with $Q_0 = S_0 = 0$ and stock LPM with $Q_0 = 128$ and $S_0 = 0$. vLLM 0.32.0 admits from the head of its FCFS queue and stops at the first request whose allocation fails (`vllm/v1/core/sched/scheduler.py`, lines 1275–1282 at tag v0.32.0), and new requests join at the tail (`vllm/v1/core/sched/request_queue.py`, lines 75–94). It also preempts the most recently admitted running request when a running request cannot get a block (`vllm/v1/core/sched/scheduler.py`, lines 794–844), frees its blocks and returns it to the head of the queue (lines 1571–1613). The condition concerns first admissions, so this re-admission ahead of earlier requests does not violate it, and the re-admissions count among the admissions that (A2) bounds. With asynchronous scheduling, vLLM also skips a preempted request until its output still in flight drains, within the pipeline depth (lines 933–939), and a follow-up admitted meanwhile passes it; only requests preempted in the last few steps are skipped this way, at most a fixed multiple of the running limit, so S_0 stays bounded. vLLM also skips requests that are blocked, for example waiting for a KV transfer or over a LoRA limit, which our runs do not involve. Its FCFS thus meets the condition with $Q_0 = 0$ and a bounded S_0. The line numbers refer to the files at these tags, read verbatim, not through the page summariser of Appendix D. Program-level FCFS meets the condition for no Q_0 and S_0: a follow-up of an old session is admitted before requests of newer sessions that were waiting when it joined, however long the queue.

Engine sources of §2. The SGLang rules of §2 (default policy and eviction rule, the 128-request LPM fallback, the unlocked prefix of a waiting request, and the refresh of waiting prefixes) are in these files under `python/sglang/srt/` at tag v0.5.22: `arg_groups/fields/schedule.py`, lines 82–98; `arg_groups/fields/memory.py`, lines 25–42; `managers/schedule_policy.py`, lines 133–135, 278–311 and 1591; `mem_cache/radix_cache.py`, lines 552–580; and `environ.py`, line 594. vLLM 0.32.0 schedules FCFS, its default, or by priority (`vllm/config/scheduler.py`, lines 21 and 159), and a freed cached block keeps its hash until it is reallocated (`vllm/v1/core/block_pool.py`, lines 666–690 and 774–803; at tag v0.32.0). We first read these sources through a web tool that returns a language model's summary of each file (Appendix D); the line references here and above are to the files at the tags, read verbatim.

The quasi-static model in congestion. Eq. (1) assumes that the internal state (turn-index mix, cache contents, queue order) relaxes quickly compared with changes in n. An automated check of the model by an LLM agent (Appendix D) found this reasonable at fixed points and for the sign of the drift in deep congestion, where the stationary μ is flat, but not near the cliff. The model is also not quantitative in congestion, and not only transiently: over the evaluation window the registered SGLang BURST runs still completed 1.13–1.35 λ_{ref} at 300k and 1.44–2.03 times the native λ_{ref}, and their fitted backlog slopes were only 18–69% and 68–78% of the rate $\lambda - \lambda_{\text{miss}}$ that Eq. (1) gives with μ at its congested value λ_{miss}, taking λ as the arrival rate each run received and λ_{miss} as λ_{ref}, or 21–75% and 70–80% with λ_{miss} 3.4% above λ_{ref} (§5.2, §5.2 and the checks of (A3) and (A4) above). Under FCFS each waiting session advances one turn per cycle of the queue while new sessions keep arriving, so the backlog stays young and its turns are short (in the BURST windows served prompts were 9–20% and 29–41% below the stationary mean; checks of (A3) and (A4) above). At a token rate near the all-miss rate the engine then serves more turns per second than in the stationary mix, and since a session ends after each turn with probability 1/15 until its 60th, it also completes sessions faster than λ_{miss}, for as long as the backlog grows.

A work balance, which we do not make rigorous for the open system, suggests that the recovery threshold depends on the token rate in congestion, not on the departure rate. If hits vanish in congestion (as in every registered SGLang BURST run) and the engine computes prompt tokens at about $C = \lambda_{\text{miss}} E[W]$ (Proposition 6) while they arrive at $\lambda E[W]$, then for $\lambda > \lambda_{\text{miss}}$ the uncomputed prompt work of the sessions in the system grows, and eventually so does their number, since each holds at most $W(60)$ tokens of it, while for $\lambda < \lambda_{\text{miss}}$ that work shrinks as long as the engine computes at C. If the zero-hit token rate is lower at some population that the system passes through on the way down, the threshold is correspondingly lower; in the fluid model it is $\inf_{n \geq n^*} \mu(n)$ rather than μ_∞. On our engine the closed runs' zero-hit token rate was 4% lower at $n = 48$ than at $n = 96$, with older turns (checks of (A3) and (A4) above). We therefore use Eq. (1) for the window, as a long-run statement for the recovery threshold, and for the capped system's drain time (§3.4), not for burst sizes or backlog growth in congestion: in an earlier simulator with assumed costs (§4.4) the fluid critical burst $n_2 - n_1$ at $2\lambda_{\text{miss}}$ was 20.5 sessions against an empirical 40, because burst sessions are fresh and small. The heuristic F3 of §6 corrects for this by counting a burst session at its peak footprint: at 1.65 λ_{ref} on the 300k pool its critical fresh burst is 53.4 sessions, against 21.3 for the stationary-composition estimate $n_2 - n_1$, and the registered basin tests of §7 test it.

A non-monotone tail, and Bronson et al.'s terms. Without the monotone tail, the recovery threshold from deep congestion is $\inf_{n \geq n^*} \mu(n)$ instead, and congested fixed points can exist. Our measured curve is not monotone beyond n^*: in completed sessions it falls to 0.68 λ_{ref} at $n = 48$ and rises to 1.51 λ_{ref} at $n = 300$ (turns-based, 0.88 and 1.56 λ_{ref}; §5.1). Taken at face value, either estimate would put the recovery threshold from $n = 300$ below λ_{miss} and give a stable congested fixed point at 1.35 λ_{ref}. We attribute both features to the synchronized start, which no longer run has tested, and take μ_∞ from Proposition 6. In the terms of Bronson et al. [10], μ_∞ is the hidden capacity, the load up to which the system heals itself after any trigger, and $\min(\mu^*, \lambda_e)$ the advertised capacity; at loads in $\mathcal{W}$ the good state is their vulnerable state, a burst is a trigger whose intensity matters, the congested regime is the metastable failure state, and the feedback of §2, in which misses lengthen the queue and the gap between a session's turns, is its sustaining effect.

The fluid statements of §3.3. With μ continuous and piecewise linear, Eq. (1) has a unique solution from every $x_0 \geq 0$; it is monotone in time, and it converges to a fixed point or diverges. Let $0 < \lambda < \mu^*$. Then $\mu(n) = \lambda$ has exactly one solution n_1 in $(0, n^*)$, because μ increases strictly from 0 to μ^* there, and, since $\lambda \neq \mu_\infty$, at most one in (n^*, ∞), because there μ is non-increasing and strictly decreasing wherever $\mu > \mu_\infty$. If $\lambda < \mu_\infty$, then $\mu \geq \mu_\infty > \lambda$ beyond n^*, so $\mu < \lambda$ on $[0, n_1)$ and $\mu > \lambda$ on (n_1, ∞), and every solution converges to n_1. If $\mu_\infty < \lambda$, the second solution n_2 exists, because $\mu(n^*) > \lambda$ and $\mu \to \mu_\infty < \lambda$; then $\mu > \lambda$ on (n_1, n_2) and $\mu < \lambda$ on (n_2, ∞), so n_2 is unstable, and for $x_0 > n_2$, $\dot{n} \geq \lambda - \mu(x_0) > 0$ along the solution and $n \to \infty$. If $\lambda > \mu^*$, then $\mu < \lambda$ everywhere and every solution diverges. A solution from $x_0 > n_1$ converges to n_1 if and only if $\mu > \lambda$ on all of $(n_1, x_0]$: if so, it decreases and stays in $(n_1, x_0]$, where n_1 is the only possible limit; if $\mu(x) \leq \lambda$ for some $x \in (n_1, x_0]$, it never falls below x, where the drift is non-negative. For $x_0 > n^*$, $\mu > \lambda$ on $(n_1, n^*]$ and the minimum of μ on $[n^*, x_0]$ is $\mu(x_0)$, so the condition is $\lambda < \mu(x_0)$, and $\mu(x_0)$ does not increase and tends to μ_∞ as $x_0 \to \infty$. Without the monotone tail the same argument gives the condition $\lambda < \min_{[n^*, x_0]} \mu$, which tends to $\inf_{n \geq n^*} \mu(n)$, and a point beyond n^* where μ crosses λ from below is a stable congested fixed point. Under the assumptions of Proposition 6 this infimum is at most $\liminf_{n \to \infty} \mu(n) \leq \lambda_{\text{miss}}$, and it equals λ_{miss} if, in addition, $\mu(n) \geq \lambda_{\text{miss}}$ for every $n \geq n^*$.

Proof of Corollary 1. While $N \geq c$ the admitted sessions form a closed system of c sessions (Proposition 1), so $\dot{N} = \lambda - \mu(c)$ and the gate, $N - c$, shrinks if and only if $\mu(c) > \lambda$. By unimodality $\mu > \lambda$ exactly on (n_1, n_2), so the gate empties, from any initial population $N_0 > c$, if and only if $n_1 < c < n_2$; if $c \leq n_1$ or $c \geq n_2$, then $\mu(c) \leq \lambda$ and it never empties. Once $N < c$ the cap does not bind, $\dot{N} = \lambda - \mu(N)$ with $N < c < n_2$, and by the statements above N stays in $[0, n_2)$ and converges to $n_1 < c$, so the cap never binds again and removing it changes nothing. This is the standard closed-system equivalence of a system with a bounded multiprogramming level (§2). If the closed curve itself folds (Proposition 7), μ is multivalued on a band of populations and we take n_2 on the good branch; a cap $c \geq n_b$ engaged inside the trap keeps the all-miss state, which exists at every $n \geq n_b$, so $n_1 < c < \min(n_2, n_b)$ is necessary, and we have not shown that it is sufficient. The stochastic counterpart (positive recurrence for $\lambda < \mu(c)$) should follow from standard arguments; we have not written them out.

F First-principles model: details and proofs

Cost model and server. The prefill GPU time of x new tokens on a cached prefix of y tokens is $t_P(x, y) = a \max(1, x/B_c) + \max(f_0, bx) + c_p\, x\,(y + (x+1)/2)$, with $a = 9.24$ ms, $f_0 = 1.26$ ms, $b = 62.6\,\mu$s/token, $c_p = 2.24$ ns per causal pair and chunk $B_c = 8192$ (`calibration/round6_server_log_fit/calibration.yaml`). For a miss, $t_k^m = t_P(L_k, 0)$; for a hit, $t_k^h = t_P(551, L_k - 551)$; turn 1 always misses. The mean prefill time per turn is $\bar{t}_P(h) = \sum_k w_k t_k(h)$, so $p = \nu \bar{t}_P(h)$ and $\partial_h \bar{t}_P = -\beta_P$. A decode iteration over R_D requests takes $\tau_D(R_D) = a + \max(f_0, bR_D) + b_{\text{att}} R_D$, with $b_{\text{att}} = c_d \bar{y}_D / 8192$, $c_d = 0.31$ ms and $\bar{y}_D$ the mean decode context. We assume: (S1) a turn's prefill latency is its own prefill GPU time, with no queueing behind other prefills; (S2) the GPU is busy whenever a request runs; (S3) a running request reserves E_k from admission to completion, with no retraction; (S4) admission is first-come-first-served, work-conserving and memory-gated: a request waits only if admitting it would make the running reservations exceed M

(idle KV is always evictable), and the engine's limit of 256 running requests and, on SGLang, the prefill chunk budget never bind. For vLLM, prefill and decode share iterations: $S_D = (d-1)/r_{\text{it}}$, with iteration rate $r_{\text{it}} = (1-\nu c_{\text{turn}})/a$, where $c_{\text{turn}} = b\bar{x} + c_p\bar{\pi} + (b + b_{\text{att}})(d-1)$ is the token-proportional GPU time of a turn ($\bar{x}$ and $\bar{\pi}$ are the mean prefilled tokens and causal pairs per turn), as long as an iteration's tokens fit its chunk budget. Lemmas 2 and 3, and the results that use them, Proposition 2(a) and (c), Proposition 3(b)–(c) and Proposition 7, are for the separate-iteration (SGLang) server; for vLLM, n_b follows from the identity of Proposition 2(b) whenever the mixed server's ν_b exists, that is, while the chunk budget does not bind. Proposition 2(a) also uses (S4).

Lemma 2 (fluid server)**.** *Fix h, and let $\nu_{\text{cap}}(h) = 1/(\bar{t}_P(h) + (d-1)(b + b_{\text{att}}))$. For $\nu < \nu_{\text{cap}}(h)$ the decode equation has exactly one positive root; for $\nu \geq \nu_{\text{cap}}(h)$ it has none. S_D is continuous, strictly increasing in ν, non-increasing in h, and $S_D \to \infty$ as $\nu \uparrow \nu_{\text{cap}}(h)$. $K_{\text{run}}(\nu, h)$ increases strictly in ν from 0 to ∞ and decreases strictly in h.*

Proof. $\psi(R) = R/\tau_D(R)$ is continuous and strictly increasing, since τ_D is affine with positive intercept on each side of the kink f_0/b. Moreover $\psi(0) = 0$ and $\psi(\infty) = 1/(b + b_{\text{att}})$. The decode equation reads $\psi(R_D) = \nu(d-1)/(1-p)$, which has a unique root if and only if the right side is below $1/(b + b_{\text{att}})$, that is, $\nu < \nu_{\text{cap}}(h)$. The right side increases in ν and does not increase in h, because $t_k^h \leq t_k^m$; so does R_D, which diverges at ν_{cap}. Then $S_D = R_D/\nu = (d-1)\tau_D(R_D)/(1-p)$ is a product of positive factors with the same monotonicity. $K_{\text{run}} = \nu \sum_k w_k E_k(t_k(h) + S_D)$ inherits it; strict decrease in h follows from $\partial_h K_{\text{run}} = -\nu(\beta_E + \bar{E}\chi\xi) < 0$ (Lemma 3). □

Lemma 3 (decode sensitivity)**.** *Away from the kink, $-\partial_h S_D = \chi\, \nu\partial_\nu S_D$, with $\chi = \beta_P/(\bar{t}_P(h) + (d-1)\tau_D'(R_D))$ and $\nu\partial_\nu S_D = R_D(\bar{t}_P + (d-1)\tau_D')/J > 0$, where $J = (1-p) - \nu(d-1)\tau_D' > 0$.*

Proof. Write $G(R_D, \nu, h) = R_D(1 - \nu\bar{t}_P(h)) - \nu(d-1)\tau_D(R_D)$. Then $\partial_{R_D} G = J$; at the root, $J = (1-p)\alpha_D/\tau_D > 0$, with $\alpha_D > 0$ the intercept of the affine piece of τ_D. Next, $\partial_\nu G = -R_D\bar{t}_P - (d-1)\tau_D$ and $\partial_h G = \nu R_D\beta_P$, since $\partial_h \bar{t}_P = -\beta_P$. Hence $\partial_h S_D = \partial_h R_D/\nu = -R_D\beta_P/J$ and $\nu\partial_\nu S_D = \partial_\nu R_D - R_D/\nu = [R_D\bar{t}_P + (d-1)\tau_D - JR_D/\nu]/J$. Substituting $JR_D/\nu = R_D/\nu - R_D\bar{t}_P - R_D(d-1)\tau_D'$ and $R_D/\nu = (d-1)\tau_D/(1-p)$ gives $(d-1)\tau_D - R_D/\nu = -pR_D/\nu = -\bar{t}_P R_D$, so the numerator equals $R_D(\bar{t}_P + (d-1)\tau_D')$. □

Proof of Proposition 2. (a) Fix $R > 0$ and $h = F_Z(R)$. The function $\nu \mapsto K_{\text{run}}(\nu, h) + \nu\mathcal{D}(R)/E[T] - M$ is continuous and strictly increasing (Lemma 2). It equals $-M$ at $\nu = 0$ and is unbounded at ν_{cap}, so it has exactly one root $\nu(R)$, which is continuous in R. By (S4), a state with a queue has $K_{\text{run}} = M$, so $\mu\mathcal{D} = 0$, hence $R = 0$, $h = 0$ and $\nu = \nu_b$. From $\nu\mathcal{D}(R)/E[T] \leq M$ and $\mathcal{D}(R) \geq E[E_T]R$ we get $\nu(R) \to 0$ as $R \to \infty$. As $R \downarrow 0$, $\nu(R) \to \nu_b$ by continuity, since $\nu_{\text{cap}}(0)$ is excluded. So $n(R)$ is bounded on $(0, \infty)$, and beyond its supremum and n_b only the all-miss state remains. (b) $K_{\text{run}}(\nu_b, 0) = \nu_b \sum_k w_k E_k S_k = M$ gives $\nu_b\bar{S} = M/E_S$; Little's law with no queue gives $n_b = \nu_b\bar{S} + \nu_b\bar{z}'$. For $n > n_b$ the all-miss state carries a queue of $(n - n_b)/\nu_b$ seconds. (c) Divide every cost by s and time by s: every service time scales by $1/s$ and every rate by s, and the decode equation and p are invariant. So (ν, h) solves $K_{\text{run}} = M$ for costs C if and only if $(s\nu, h)$ does for C/s. Hence $\nu_b(s) = s\nu_b(1)$; E_S, a ratio of S-weighted sums, is invariant; substitute into (b). □

Proof of Proposition 3. (a) Rescaling time by s maps a stationary state (n, ν, h, R) of $(C/s, \bar{z})$ to $(n, \nu/s, h, sR)$ of $(C, s\bar{z})$. Each equation of Eq. (3) is invariant: Little's law and K_{run} are rate×time, $E[\min(\bar{z}Y, R)] = s^{-1}E[\min(s\bar{z}Y, sR)]$, and h compares times. Both μ^* and μ_b scale by s, so their ratio depends on $s\bar{z}$ only. (b) Parametrize the queue-free states by $R \in (0, \infty)$, with $h = F_Y(R/\bar{z})$. The balance reads $K_{\text{run}}(\nu, F_Y(R/\bar{z})) + \nu(A_1 E[\min(\bar{z}Y, R)] + E[E_T]R)/E[T] = M$. Its left side increases strictly in ν (Lemma 2) and does not decrease in $\bar{z}$, since h falls, K_{run} decreases in h and $E[\min(\bar{z}Y, R)]$ rises; where $h > 0$ it increases strictly in $\bar{z}$, because then $E[\min(\bar{z}Y, R)]$ does. So $\nu(R; \bar{z})$ does not increase in $\bar{z}$, while ν_b (the equation at $h = 0$) does not involve $\bar{z}$, and taking the supremum over R, ω_b is non-increasing in $\bar{z}$. For strictness, let $\bar{z}_1 < \bar{z}_2$ and $\omega_b > 1$ at $\bar{z}_2$. Since Z has a density, $\nu(R; \bar{z})$ is continuous in R, and by the proof of Proposition 2(a) it tends to ν_b as $R \downarrow 0$ and to 0 as $R \to \infty$. So $\sup_R \nu(R; \bar{z}_2) > \nu_b$ is attained at some $R_2 \in (0, \infty)$, where $h > 0$, because a state with $h = 0$ and $R > 0$ has $K_{\text{run}}(\nu, 0) < M$ and so $\nu < \nu_b$. Hence $\sup_R \nu(R; \bar{z}_2) = \nu(R_2; \bar{z}_2) < \nu(R_2; \bar{z}_1) \leq \sup_R \nu(R; \bar{z}_1)$. This holds for any Y with a density, bounded or not and whether or not F_Y has flat parts. Apply (a) for s. (c) Write $r(h) = F_Y^{-1}(h)$ and $m(h) = E[\min(Y, r(h))]$, and index the queue-free states by h. For $h \geq h_0 > 0$, $\nu(h; \bar{z}) \leq ME[T]/(\bar{z}(A_1 m(h_0) + E[E_T]r(h_0))) \to 0$. For $h < h_0$, $\nu(h; \bar{z})$ is below the root of $K_{\text{run}}(\nu, h) = M$, which tends to ν_b as $h_0 \to 0$. □

The feedback map and the fold identity. For a closed population n define the *feedback map* Φ_n. At hit probability h, Little's law with no queue, $n = \nu(\bar{S}(\nu, h) + \bar{z}')$, gives a unique turn rate $\nu(h)$; then $\Phi_n(h) = 0$ if $K_{\text{run}}(\nu(h), h) \geq M$, and otherwise $\Phi_n(h) = F_Z(R)$, with R the solution of $(\nu(h)/E[T])\,\mathcal{D}(R) = M - K_{\text{run}}(\nu(h), h)$. The fixed points of Φ_n are the stationary states at population n; we call one *locally stable* if it is stable under the relaxation $\dot{h} = \eta(\Phi_n(h) - h)$, $\eta > 0$, at fixed n.

Proposition 7 (fold identity; proof sketch)**.** *On the queue-free branch parametrized by R, away from the R at which R_D crosses the kink f_0/b of the decode cost τ_D (see the cost model above; there the identity holds with one-sided values), with $\xi = \nu\,\partial_\nu S_D \geq 0$,*

$$\frac{dn}{dR} = \frac{\nu}{\Gamma_\nu}\Big[f(R)\,I(R) - \frac{n_\nu\,\mathcal{D}'(R)}{E[T]}\Big],$$

where $\Gamma_\nu = S_E + \bar{E}\xi + \mathcal{D}/E[T] > 0$, $n_\nu = \bar{S} + \bar{z}' + \xi$, $S_E = \sum_k w_k E_k S_k$, and $I = \xi[C_E + \chi(\bar{E}(\bar{S} + \bar{z}') - S_E -$

$\mathcal{D}/E[T])] + \beta_E(\bar{S} + \bar{z}') - \beta_P(S_E + \mathcal{D}/E[T])$, *with* $\beta_P = \sum_{k\geq 2} w_k(t_k^m - t_k^h)$, $\beta_E = \sum_{k\geq 2} w_k E_k(t_k^m - t_k^h)$, $C_E = \beta_E - \bar{E}\beta_P$ *and* $\chi = \beta_P/(\bar{t}_P(h) + (d-1)\tau_D'(R_D))$. *Consequently: (a) if* $f(0^+)I(0^+) > n_\nu \bar{E}$ *at the all-miss end, then* $n(R) > n_b$ *for small* R*, and the closed system has at least three stationary states for* $n \in (n_b, n_{\text{fold}})$*,* $n_{\text{fold}} = \max_R n(R)$*: the all-miss state, which is locally stable under this relaxation, and two queue-free states. (b) If the bracket is negative for every* R*, on both sides of the kink, then* $n(R)$ *is strictly decreasing, every* n *has exactly one stationary state, and* $\mu(n)$ *is continuous.*

Proof sketch of Proposition 7. We work away from the R at which R_D crosses the kink of τ_D, where Lemma 3 applies; at that R the same computation holds with one-sided derivatives. On the branch, $\Gamma(\nu, R) = \nu S_E(\nu, h) + \nu\mathcal{D}(R)/E[T] - M = 0$ and $n = \nu(\bar{S} + \bar{z}')$, with $h = F_Z(R)$ and $dh/dR = f(R)$. From $S_k = t_k(h) + S_D(\nu, h)$ we have $\partial_h \bar{S} = -\beta_P + \partial_h S_D$, $\partial_h S_E = -\beta_E + \bar{E}\partial_h S_D$, $\partial_\nu \bar{S} = \partial_\nu S_D$ and $\partial_\nu S_E = \bar{E}\partial_\nu S_D$. By Lemma 3, $\partial_h S_D = -\chi\xi$ with $\xi = \nu\partial_\nu S_D$. Then $\Gamma_\nu = \partial_\nu\Gamma = S_E + \bar{E}\xi + \mathcal{D}/E[T] > 0$ and $\partial_R\Gamma = -\nu f(\beta_E + \bar{E}\chi\xi) + \nu\mathcal{D}'/E[T]$, so

$$\frac{d\nu}{dR} = \frac{\nu}{\Gamma_\nu}\left[f(\beta_E + \bar{E}\chi\xi) - \frac{\mathcal{D}'}{E[T]}\right].$$

Next, $dn/dR = n_\nu\, d\nu/dR + \nu f\,\partial_h\bar{S} = n_\nu\, d\nu/dR - \nu f(\beta_P + \chi\xi)$, with $n_\nu = \bar{S} + \bar{z}' + \xi$. Put both over Γ_ν: the coefficient of $\nu f/\Gamma_\nu$ is $I = n_\nu(\beta_E + \bar{E}\chi\xi) - (\beta_P + \chi\xi)\Gamma_\nu$. Expanding, the terms $\bar{E}\chi\xi^2$ cancel. Collecting the terms linear in ξ gives $\xi[\beta_E - \bar{E}\beta_P + \chi(\bar{E}(\bar{S}+\bar{z}') - S_E - \mathcal{D}/E[T])]$, and the remainder is $\beta_E(\bar{S} + \bar{z}') - \beta_P(S_E + \mathcal{D}/E[T])$. This is the stated I. The term in $\mathcal{D}'$ is $-n_\nu\mathcal{D}'/E[T]$. (a) As $R \to 0^+$, $n(R) \to n_b$ (Proposition 2), $\mathcal{D} \to 0$ and $\mathcal{D}'(0^+) = A_1 + E[E_T] = E[T]\bar{E}$. If the bracket is positive at 0^+, then by continuity n increases on some $(0, \delta)$. Since $n(R) \to 0$ as $R \to \infty$, n attains a maximum $n_{\text{fold}} > n_b$, and by the intermediate value theorem every $n \in (n_b, n_{\text{fold}})$ is reached on both sides of the maximizer. Together with the all-miss state this gives at least three stationary states. For $n > n_b$, Little's law at $h = 0$ with no queue forces $\nu > \nu_b$, so $K_{\text{run}} > M$; by continuity the feedback map is memory-gated ($\Phi_n \equiv 0$) near $h = 0$, and the all-miss state is locally stable under the relaxation defined before Proposition 7. (b) A strictly decreasing continuous $n(R)$ is a bijection onto $(0, n_b)$. *Window opening.* At $R \to 0^+$ the formula for $d\nu/dR$ has the sign of $f(0^+)\Delta_S - 1$, with $\Delta_S = (\beta_E + \bar{E}\chi\xi)/\bar{E}$. If it is positive, $\mu = \nu/E[T]$ rises above μ_b as R leaves 0. □

This argument is a proof sketch: the algebra of the identity is given in full, but the local stability of the all-miss state rests on a continuity step that we have not written out, and the stability of the queue-free states is argued, not proved (below). The identity is also checked in magnitude: its closed form agrees with central finite differences of $n(R)$ within a relative tolerance of 2×10^{-3} at five values of R, for the exponential law and for a folding mixture (`tests/test_theory_paper.py`). That the registered cells are fold-free is a numerical statement: the ratio $fI/(n_\nu\mathcal{D}'/E[T])$ was evaluated on 400 values of R per cell, with complete sign agreement against finite differences of n (`analysis/theory/merged/fold.py`). The stability of the queue-free states is a further statement: under relaxation $\dot{h} = \eta(\Phi_n(h) - h)$, $\eta > 0$, and the condition $\partial_n\Phi_n < 0$, which we have checked only numerically and outside the committed code, a queue-free state is stable if and only if $dn/dR < 0$ there, because along the curve $n'(R) = f(R)(1 - \partial_h\Phi_n)/\partial_n\Phi_n$.

The fold criterion on the predicted cells. For the registered exponential tool times the ratio $fI/(n_\nu\mathcal{D}'/E[T])$ peaks at the all-miss end at 0.805 (300k), 0.826 (native), 0.784 (H200), 0.781 (Llama), 0.761 (150k) and 0.821 (600k), checked on 400 values of R per cell; over the other four H200 cost variants it is 0.67–0.78. Every separate-iteration cell is therefore fold-free by this criterion, with margins of 17–24% for the primary cost models and up to 33% over the H200 variants; the vLLM curve, to which the proposition does not apply, is fold-free on a numerical scan of $n(R)$. On every predicted cell, metastability is therefore a property of the open system. The same identity at $R \to 0^+$ gives a sufficient condition for the window to open: $\mu^* > \mu_b$ if $f(0^+)\,\Delta_S > 1$ with $\Delta_S = (\beta_E + \bar{E}\chi\xi)/\bar{E}$ (595 s at 300k, against $1/f(0^+) = \bar{z} = 5$ s). Dispersed tool times can reverse the conclusion. With the same 5 s mean, a mixture of exponentials with means 1 s and 9 s in equal proportions (coefficient of variation 1.5) folds the 300k curve up to $n_{\text{fold}} = 34.2 > n_b = 32.7$, where the all-miss state and a good state ($h = 0.69$) are both stable (Fig. 5, Appendix F); mixtures with coefficients of variation 2.5 and 5 fold up to 38.2 and 44.6. Measured agent tool times are also dispersed (the SWE-Bench tool times Continuum reports have a coefficient of variation of about 3.8 [58]). Part (a) is a sufficient condition at the all-miss end, set by the density near 0. A law can also fold at an interior R, where the bracket changes sign; the theory notes report this, outside the committed code, for lognormal laws with a coefficient of variation of 2 or more, which have $f(0^+) = 0$. So neither $f(0^+)$ nor the coefficient of variation alone decides it, and we have not evaluated the criterion for any measured law. When the fold is at the all-miss end, n_b is the lower edge of a bistable band of the closed system, not the collapse point. At an interior fold the bracket is negative near $R = 0$, so $n(R)$ first falls below n_b, and the band in which several stationary states coexist starts at a local minimum of $n(R)$ below n_b. None of our engine runs used such a law.

Proof of Proposition 5. If each present session completes one turn per cycle of length t_{cyc}, the gap between a session's consecutive turns is t_{cyc}. The churn is $\sum_i(L_{k_i} + d)/t_{\text{cyc}}$, so $R_M = M/\text{churn} = Mt_{\text{cyc}}/\sum_i(L_{k_i} + d)$ and $t_{\text{cyc}}/R_M = \sum_i(L_{k_i} + d)/M$; the cost model cancels. LRU evicts an idle item before M newer tokens exist, so its eviction age is at most R_M: the trap is self-consistent when the next admission comes later than that. □

Proposition 4 on the measured cells. Table 15 evaluates three exogenous-gap closures at every closed run past the measured peak of its cell (the largest $\mu(n)$ in completed sessions: $n = 24$ at 300k, 64 with the native pool, 20 on vLLM), with nothing fitted: Yao et al.'s characteristic-time equation [97]

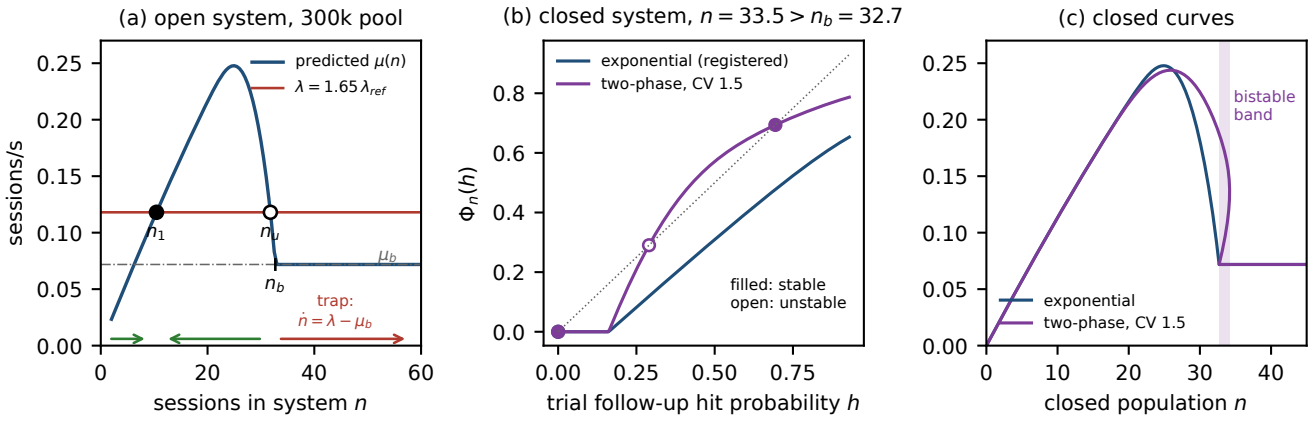


Figure 5: Fixed points of the first-principles model. (a) Open system, predicted 300k curve at 1.65 λ_{ref}: stable good state $n_1 = 10.4$, unstable state $n_2 = 31.8$ (marked n_u), all-miss onset $n_b = 32.7$, and the trap, where Eq. (1) makes N grow at $\lambda - \mu_b$ (the round-robin layer RR of §6.3, which the model uses for the trap, predicts slower growth). (b) Closed system at $n = 33.5 > n_b$: the feedback map $\Phi_n(h)$, the hit probability produced at hit probability h. With the registered exponential tool times the all-miss state is the only fixed point. With a two-phase law of the same 5 s mean (coefficient of variation 1.5), the all-miss state and a good state ($h = 0.69$) are both stable, and an unstable state ($h = 0.29$) separates them. (c) The two closed curves: the dispersed law folds above n_b (shaded band, 32.7 to 34.2), as Proposition 7 predicts.

at the measured throughput (turns-based); the same equation with its rate set by Little's law with exogenous gaps and zero service time, $\lambda_0 = n/((E[T] - 1)\bar{z})$, which does not use the measured throughput and exceeds it past the peak (0.70 against 0.063 sessions/s at 300k, $n = 48$); and Che's approximation with the tool time as the gap, sessions as objects of the mean footprint $\bar{E}$ and no dead content, whose follow-up hit $\min(1, M/(n\bar{E}))$ involves no rate at all. The proposition's bound with $K_{\text{run}} = 0$ is evaluated at the measured throughput, so it bounds only the first closure; the Little and Che values can fall below it. Zero service time is the choice least favourable to hits, since it gives the largest rate, and omitting dead content is favourable to them. Every value counts first turns, so it is at most 0.932. The closures decay smoothly in n (Che's as $1/n$ beyond $M/\bar{E}$) and never reach the measured zeros; zero hits require running requests to hold the whole pool. The evaluation is retrospective: it was added to the second prediction set (tag `r13-theory-pred2`) with the measured closed curves as inputs, and it predicts nothing.

The open system on the predicted curve. For $\mu_b < \lambda < \mu^*$, under Proposition 7(b) and unimodality in the sense of §3.3, $g(n) = \lambda - \mu(n)$ is positive at 0, crosses zero downward once on $[0, n^*]$ (at n_1) and upward once on $[n^*, n_b]$ (at n_2), and equals $\lambda - \mu_b > 0$ on $[n_b, \infty)$, so the fluid statements of §3.3 hold with $\mu_\infty = \mu_b$ and $n_2 \le n_b$, and from $n \ge n_b$ the solution of Eq. (1) grows linearly at $\lambda - \mu_b$; the model's trapped-state predictions do not use this statement; they come from the round-robin layer RR (§6.3). With the capped system, Corollary 1 applies; the closed system of c sessions has a unique stationary state, which is good if and only if $c < n_b$. With a fold, μ is multivalued on the band where several

Table 15: Proposition 4 on the measured closed curves (`results/theory/pred2/prop_e_cor_c.json`), at every closed run past the measured peak of its cell: the hit fraction measured, given by three exogenous-gap closures (Yao at the measured throughput; Little's-law and Che closures, whose rates do not use it), and the proposition's lower bound with $K_{\text{run}} = 0$ at the measured throughput, which applies to the Yao column only; the share of the pool that running requests would have to hold to give the measured hit fraction; and the frozen first-principles model (fluid; F1 in parentheses where it differs by more than 0.05).

cell, n	measured	Yao	Little	Che	bound	K_{run}/M	model
300k, 32	0.505	0.932	0.723	0.827	≥ 0.899	0.832	0.371
300k, 48	0.000	0.932	0.499	0.551	≥ 0.930	1.000	0.000
300k, 96	0.000	0.932	0.254	0.276	≥ 0.927	1.000	0.000
300k, 300	0.000	0.932	0.082	0.088	≥ 0.903	1.000	0.000
native, 80	0.922	0.932	0.868	0.932	≥ 0.866	0.531	0.920
native, 96	0.419	0.932	0.761	0.878	≥ 0.932	0.956	0.000 (0.355)
native, 128	0.007	0.932	0.590	0.659	≥ 0.932	0.9995	0.000
native, 300	0.000	0.932	0.259	0.281	≥ 0.932	1.000	0.000
vLLM, 24	0.879	0.932	0.889	0.932	≥ 0.783	0.412	0.910
vLLM, 28	0.658	0.932	0.806	0.932	≥ 0.892	0.763	0.812 (0.717)
vLLM, 32	0.083	0.932	0.723	0.827	≥ 0.931	0.987	0.441 (0.381)
vLLM, 40	0.007	0.932	0.592	0.662	≥ 0.931	0.999	0.000
vLLM, 48	0.000	0.932	0.499	0.551	≥ 0.931	1.000	0.000
vLLM, 96	0.000	0.932	0.254	0.276	≥ 0.929	1.000	0.000
vLLM, 300	0.000	0.932	0.082	0.088	≥ 0.909	1.000	0.000

stationary states coexist (on (n_b, n_{fold}) for a fold at the all-miss end) and n_2 is taken on the good branch; a cap $c \geq n_b$ engaged in the trap keeps the all-miss state, so $c < n_b$ is a further necessary condition, whose sufficiency we have not proved.

Heuristic layers. These are heuristics, not consequences of the propositions. *RR*, the trapped layer, replaces Eq. (1) in the trap by a fluid FCFS round robin. Every follow-up misses, and every present session completes one turn per cycle $\theta(t) = \sum_i [t^m_{k_i} + (1 - p_b)/\nu_b]$, the sum over the sessions present of the miss prefill time of the next turn and the per-turn decode share of the all-miss state (p_b is p at $(\nu_b, 0)$). Each cohort advances one turn per cycle and survives to turn k with probability $(14/15)^{k-1}$, up to turn 60. The layer starts at the burst from the good-state population n_1 with the stationary turn mix w_k, and adds the burst and arrivals at rate λ. The trapped rows of Table 3, the model column of Table 4 (Proposition 5 applied to its turn mix) and F4's starting population and first cycle come from it. Because the backlog stays young, its cycle is short at first: on the 300k pool, at 1.35 and 1.65 λ_{ref}, N falls after the burst to 153 and 174 sessions and then grows, over the evaluation window, at 0.016 and 0.029 sessions/s, 64% and 63% of $\lambda - \mu_b$. *F1* averages the curve over the KV-demand fluctuation of n sessions: relative variance $0.570/n$, from the turn mix. *F2* predicts the from-empty edge: demand follows a one-dimensional diffusion with drift $\lambda - \mu(x)$ and noise intensity $1.567(\lambda + \mu(x))$, the dispersion index of the session shot noise (Campbell), and a run counts as unstable if it reaches n_b within the 3000 s horizon. *F3* gives the probability that a burst leaves the run stuck (hit fraction below 0.2 over the evaluation window) as the splitting probability [54] of the same diffusion, started at the burst cohort's peak live demand, plus the background escape (Appendix F); burst sessions are fresh, so a burst counts by its peak footprint (4,331 tokens per session, at turn 12), not by its number. *F4* gives the release time of a cap engaged inside the trap by a two-phase drain that starts from RR's population and cycle at the engage time: paused sessions first, then fresh arrivals. In detail: *RR* is integrated with a 1 s step from the burst time: each second a cohort of mass λ (plus 200/60 during the burst) enters at turn 1, every cohort's turn counter advances by $1/\theta(t)$, and a cohort that entered with mass m holds $m(14/15)^{k-1}$ sessions at turn k; the initial n_1 sessions are spread over turns with the stationary mix (`analysis/theory/fluid_rigorous/trapped.py`, `round_robin_trap`). Backlog, slope, token rate and gap/R_M are its window means. *F1*: at population n the KV demand ratio is $\theta \sim N(1, 0.570/n)$, and $\tilde{\mu}(n) = E_\theta[\mu_{\text{fluid}}(n; M/\theta)]$. *F2*: $dX = (\lambda - \mu(X))dt + \sqrt{I_D(\lambda + \mu(X))}\, dW$, reflected at 0, with $I_D = 1.567$, the shot-noise dispersion index by Campbell's theorem. The probability that a run is unstable is $1 - e^{-t_H/\mathcal{T}}$, with $\mathcal{T}$ the mean first-passage time from 0 to n_b and $t_H = 3000$ s. *F3*: a burst of N_{b} fresh sessions starts the diffusion at $X_0 = D^{-1}_{\text{live}}(D_{\text{live}}(n_1) + 4331\, N_{\text{b}})$, where 4331 tokens is a fresh cohort's peak expected KV per session (turn 12); the probability that the run is stuck (hit fraction below 0.2 over the registered evaluation window) is $q(X_0) + (1 - q(X_0))P_{\text{esc}}$, with q the splitting probability between n_1 and n_b [54] and P_{esc} the F2 probability of an escape from the good state in the time that remains before such an escape could no longer bring the registered evaluation window's hit fraction below 0.2. *F4*: RR gives the population n_{on}, the mean turn index k_0 and the cycle θ_{on} at the engage time t_{on} (161.3 sessions and 88 s at 1800 s and 1.35 λ_{ref} on the 300k pool). Within one cycle every session reaches a tool-call boundary, so the admitted set falls to c by $t_{\text{on}} + \theta_{\text{on}}$. The gate serves the paused sessions first. Phase 1 lasts $T_1 = (n_{\text{on}} - c)/\mu_1$, with μ_1 the capped rate of paused sessions: footprint inflated by $1 + 650(k_0 - 1)/E_{\text{lb}}$ and one forced miss. Phase 2 drains the $\lambda(\theta_{\text{on}} + T_1)$ fresh arrivals collected meanwhile at $\mu(c) - \lambda$, so the gate empties at $t_{\text{on}} + \theta_{\text{on}} + T_1 + \lambda(\theta_{\text{on}} + T_1)/(\mu(c) - \lambda)$. Release comes at the first 10 s tick at which the gate has been empty for 300 s, and not before $t_{\text{on}} + 600$ s.

Relation to prior models in detail. Yao et al. [97] give a mean-field (Che-type) LRU hit ratio for multi-turn sessions with exogenous gaps between turns, a rate scaled with memory, and prefill/decode disaggregation; their characteristic-time equation already contains the term for content that is never reused and has a unique fixed point, and they note that end-to-end latency may change the observed gaps without modelling it; EfficientAgent measures that waiting behind other agents stretches the gap between an agent's turns, and its backlogged-pool working-set scale is the cycle argument of Proposition 5 [83]. Eq. (3) reduces to theirs when nothing runs and nothing queues. The feedback here runs through service time and reserved KV: misses starve decode under prefill priority, which swells running reservations and squeezes idle KV. That turns their unique fixed point into a curve with a rise, a peak, a cliff at Eq. (4) and an all-miss branch, on which the queue sets the reuse gap (Proposition 5). Yuan et al. measure this decode slow-down under "context thrashing" as agent concurrency rises [102]. MISA-T describes the eviction–cold-prefill–queue loop in words, measures the collapse, derives a session cap of the same capacity-over-session-size form as the pool part of Eq. (4) (a reserved block budget over a conservative session length) and contracts it on a hit-rate decline [45]; neither gives equilibria, a rescue condition or a fold criterion. The Che approximation and its analyses [17, 34, 38, 40, 67] take request processes as exogenous, with a unique fixed point; here it enters as Assumption MF, at the edge of its validity. CacheRoute states the Che criterion, revisit time longer than eviction time, for LLM prefix caches whose revisit times are set by routing, and reports that a single characteristic-time model misses a 70B engine's hit rate by 14.3 points [19]. The closest classical couplings of cache loss and queueing, processor-cache affinity scheduling [85] and buffer hits under data contention [25], stop short of a threshold. The closed curve is a Denning thrashing curve [26], and Corollary 1 is the classical decomposition argument [23] applied to a predicted $\mu(n)$. Metastable-failure models [2, 10, 35, 42, 48] study retry- and timeout-sustained metastability with Markov chains, simulation or stress tests; here the sustaining effect is

cache loss, and the thresholds are predicted from independent inputs (van Rooyen [89] also predicts a collapse threshold from a measured service law, for feedback from output to load). What we believe is new: a predicted thrashing curve for prefix-cached agent serving, the onset identity and its speed corollary, the $s\bar{z}$ law for the peak ratio, a fold criterion (proof sketch) under which, numerically, dispersed tool times make the closed curve fold, and the exact time-domain form of the trapped identity. Closed-system bistability in thrashing multiprogramming models is classical [23, 72]; the criterion in terms of the tool-time density is ours. Our literature search was targeted, not exhaustive; in particular we have not read the multiprogramming literature after Courtois for a fold criterion analogous to Proposition 7.

Not modelled. SGLang's retraction of running requests under memory pressure, which the theory notes list as a candidate cause of the 1–3 point deficit in the trapped token rate, although none of our 73 SGLang server logs records one (Appendix E); prefill queueing (S1); the limit of 256 running requests (Lemma 2 needs $M < 256E_S \approx 2.8$ million tokens, which every cell satisfies); and the reset of the release hold when sessions arrive just after the gate empties.

G Supplementary results

G.1 Closed curves and from-empty runs

With the 300k pool (Fig. 2, left), $\mu(n)$ rises from 0.176 sessions/s at $n = 16$ to 0.224 at 20 and 0.233 at 24, with hit fractions 0.93, 0.93 and 0.89. The run at $n = 24$, which sets μ^*, contains a degraded episode inside its window (hit fraction 0.66 over [900, 1200) s, 0.46 in its worst 100 s); over [1200, 2700) s alone $\mu(24) = 0.257$ against $\mu(20) = 0.231$, so the near-tie that sets the cap of 20 owes part of its closeness to this episode. The run at $n = 32$ mixes two regimes: degraded from about 450 to 2400 s, then recovered to 0.80, for a hit fraction of 0.51. At $n = 48$ none of the 20.0M prompt tokens in the window was served from cache: evictable tokens survive about 6.3 s before eviction (estimated as mean evictable KV over the eviction rate) while a session returns after 30.6 s on average (2.9 s against 66 s at $n = 96$). The hit fraction crosses 0.5 at $n = 32.2$, about 3.7 sessions above the capacity ratio $N_{\mathrm{lb}} \approx 28.5$ and just below the all-miss onset $n_b = 32.7$ of the first-principles model, which predicts this curve without using its points (§6, Table 3). Token-level hit rates have been reported to collapse as concurrent agents grow on SGLang [18] and on vLLM [45, 102]; what we add is $\mu(n)$ in completed sessions per second, the quantity the open-system model uses, at two pool sizes and, in §5.2, on a second engine. Beyond the peak the curve is not monotone: it falls to $\mu(48) = 0.049$ sessions/s and rises again, $\mu(96) = 0.061 < \mu(300) = 0.108$ sessions/s. We attribute both to the synchronized start, which leaves old, long turns in the window at $n = 48$ (served prompts 14% above the stationary mean) and young sessions with short prompts at $n = 300$ (71% of the stationary length in flight); no longer run has tested this; the all-miss level implied by token throughput is roughly flat at 0.072–0.081 sessions/s, so the collapse from the peak is about threefold. Stock LPM keeps 39% of turns hitting at $n = 48$ (rising from 0.28 to 0.44 through the window); at $n = 300$ its queue of 269 exceeds the 128-request threshold above which, by the source, SGLang's LPM stops re-sorting its queue, and no turn hits; whether the fallback causes the zero was not tested (at $n = 300$ the pool may be oversubscribed under any ordering).

With the native pool (Fig. 2, middle), SGLang sized the KV pool at 955,757 tokens, 3.19 times the 300k pool. μ peaks at 0.479 sessions/s at $n = 64$ (0.477 at 80) with 92–93% of turns hitting, then falls to 0.123 at $n = 96$ (hit fraction 0.42), 0.067 at 128 (0.007) and 0.101 at 300. The hit fraction crosses 0.5 between $n = 80$ (0.92) and 96 (0.42), near 93 by linear interpolation, against just above 32 at 300k; the peak is at 64–80 against 20–24. With one run per n, the cliff moves out roughly threefold. The all-miss capacity is unchanged within its standard error: 0.0718 ± 0.0037 sessions/s turns-based against 0.0715 ± 0.0038, and $11{,}282 \pm 51$ against $11{,}243 \pm 84$ all-miss tokens/s. The 300k-token pool limit did not create the cliff; it moved it. The ratio $\mu^*/\lambda_{\mathrm{ref}}$ is 6.67 with the native pool and 3.26 at 300k.[1]

E0a gives $\lambda_{\mathrm{ref}} = 0.0715 \pm 0.0038$ sessions/s turns-based (standard error over three seeds), our estimate of λ_{miss}, 0.0717 ± 0.0076 in completed sessions, and $11{,}243 \pm 84$ all-miss prefill tokens/s. The token rate varies by ±1.2% across seeds and the session rate by ±17%: the engine is bound by prefill tokens, and the sessions completed in 1800 s are a small sample of a heavy-tailed quantity. The token rate divided by $E[W]$ gives 0.0728 sessions/s, within 2% of the turns-based value.

From an empty cache and queue (Fig. 6, top) the engine stayed stable at 1.25, 1.5, 1.75 and 2.0 λ_{ref}, ending each run with an empty queue and with 91–93% of turns hitting over the run, and was unstable at 2.5 λ_{ref} (final queue 127, rising by 146 over the last 1500 s) and 3.0 (289 and 196). On this grid, which used the single arrival seed 0, the edge lies between 2.0 and 2.5 λ_{ref} in nominal rates, well below $\mu^*/\lambda_{\mathrm{ref}} = 3.26$, so the from-empty edge, not the peak, bounds the window. The registered window was $(1, 2.0)\,\lambda_{\mathrm{ref}} = (0.0715, 0.143)$ sessions/s. Each rate is one 3000 s run; a longer horizon could find a lower edge, and at 2.0 λ_{ref} the hit fraction fell from 0.934 in [300, 1500) s to 0.903 in the second half of the run. All six runs used seed 0, so they share one random-number stream: their arrival sequences are nested, and each arrival count lies about 2.1 standard deviations above its Poisson mean, 8–13% above the nominal rate (302, 359, 416, 472, 585 and 697 sessions; the runs labelled 2.0 and 2.5 λ_{ref} received 2.20 and 2.73 λ_{ref}). This makes the stable side of the bracket conservative and its unstable side weaker than the labels suggest; in realized arrivals the bracket is still below $\mu^*/\lambda_{\mathrm{ref}}$.

An exploratory replication with fresh seeds 3, 4 and 5 at 2.0 and 2.5 λ_{ref} (jobs 17168506–07; §7) found all six runs stable,

[1] These are the registered values: the peak in completed sessions over the turns-based λ_{ref}. Turns-based over turns-based gives 6.48 and 3.15.

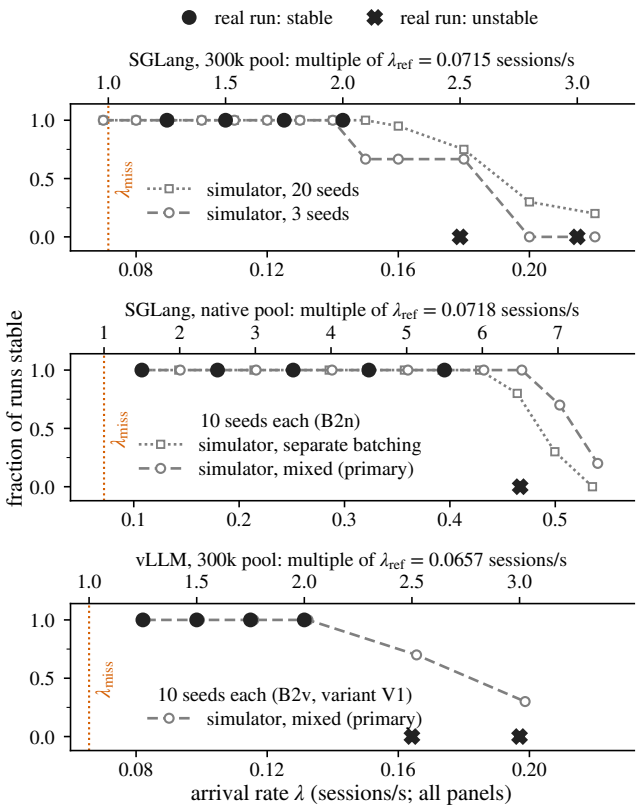


Figure 6: From-empty stability. Real engine: one 3000 s run per rate from an empty cache and queue (1 = stable, 0 = unstable), plotted at its nominal rate; all runs used arrival seed 0, whose realized rates were 6–13% higher (the fresh-seed runs of §5.1 are not plotted). Gray: blind simulator predictions, the fraction of simulated runs stable at multiples of the simulator's own λ_{miss}. Top, SGLang with the 300k pool: B2 (3 seeds dashed, 20 dotted). Middle, SGLang with the native pool: B2n (10 seeds; mixed batching, the primary mode, dashed; separate, dotted). Bottom, vLLM 0.32.0 with the 300k pool: B2v (variant V1, costs scaled to vLLM; mixed batching, 10 seeds). Top axes: multiples of each cell's λ_{ref} (the simulators' own λ_{miss} are 0.0722 at 300k, 0.0720 and 0.0714 native, and 0.0662 for B2v).

each ending with an empty queue and with 92–93% of turns hitting. Its frozen rule asked for 2.0 λ_{ref} to be stable in all three seeds and 2.5 λ_{ref} unstable in at least two, so the seed-0 bracket is not replicated: near 2.5 λ_{ref} the nominal edge depends on the seed (2.5 λ_{ref} was stable in 3 of 4 seeds). The rule's fallback restates the bracket in realized arrival rates. Over the twelve from-empty runs at this pool (seeds 0 and 3–5), every run that received at most 2.665 λ_{ref} (0.191 sessions/s) was stable and the two that received 2.726 and 3.248 λ_{ref} were unstable, so stability is monotone in the realized rate and the edge lies between 2.665 and 2.726 λ_{ref}, still below μ^*/λ_{ref}. The registered window was derived from the nominal seed-0 bracket and stands as frozen; the registered points, 1.35 and 1.65 λ_{ref}, lie inside the from-empty stable range on either reading. The calibrated simulator had predicted all six fresh-seed runs stable, blind in time. The heuristic F2 of §6 was pessimistic: it puts the per-seed probability of instability at 0.15 at 2.0 and 0.69 at 2.5 λ_{ref}, so three stable runs at 2.5 had probability 0.03 under it (§7 states which of its values are blind).

With the native pool (Fig. 6, middle; native E0b, rates in multiples of the native λ_{ref} = 0.0718 sessions/s) the engine stayed stable at 1.5, 2.5, 3.5, 4.5 and 5.5 λ_{ref}, ending each run with an empty queue and with 93% of turns hitting over the run, and was unstable at 6.5 λ_{ref}: its final queue was 505, rising by 477 over the last 1500 s, and its hit fraction fell from 0.84 in [300, 1500) s to 0.00 in the second half. In nominal rates the edge lies between 5.5 and 6.5 λ_{ref} (0.395 and 0.467 sessions/s), so the native window is (1, 5.5) λ_{ref}. Unlike at 300k, this does not show the edge below the peak (μ^*/λ_{ref} = 6.67, or 6.48 turns-based): these runs also used seed 0 (361 to 1478 sessions, 5.6–11.7% above nominal), and the 6.5 run received 0.493 sessions/s (6.86 λ_{ref}), above the measured peak, so overload alone can explain its instability. The stable 5.5 run received 0.419 sessions/s (5.84 λ_{ref}). Its upper edge is 2.75 times higher than at 300k in units of λ_{ref} (5.5 against 2.0), and the window is 4.5 λ_{ref} wide against 1.0. Again each rate is a single 3000 s run.

G.2 The registered paired tests in detail

The registered analysis, run on all 18 runs with the code frozen at the registration commit, passes at both rates (Table 1, top); by the definition of §4.3 the test is pre-registered. Every BURST run had a hit fraction of 0.000 over the evaluation window, an uncached prefill rate of 1.009–1.029 times the all-miss rate and a mean backlog of 200–332 sessions. One (1.35 λ_{ref}, seed 1) is stuck in the same way, but its backlog was flat over the window (207 sessions at its start, 206 at its end; slope 0.0036 sessions/s, below the 0.0125 threshold), so the rule calls it indeterminate: its seed delivered 0.0910 arrivals/s in the window, 5.8% below the nominal 0.0966 (seeds 0 and 2: 0.1037 and 0.1017), and the trapped engine completed 0.0913 sessions/s. The pass at 1.35 λ_{ref} therefore rests on 2 of 3 seeds, the minimum the rule allows. The other runs' base arrivals over the window came to 1.42 and 1.45 λ_{ref} at 1.35 and 1.61–1.75 λ_{ref} at 1.65 (Table 1). Every NOBURST and CAPPED run is good (hit fractions 0.921–0.934, backlogs of 8–12 sessions, most asleep in tool calls); in the window CAPPED cannot be told apart from NOBURST, because the gate had drained the burst. The counts are small, and pooling the two rates does not make them independent: both rates use the same three seeds, with the same burst and nested arrivals (§4.1). BURST's hit fraction is below NOBURST's in all six seed-and-rate pairs; counting each seed once, a post-hoc one-sided sign test gives $p = 1/8$, the smallest value three seeds allow.

In Fig. 1, every BURST run is at zero hits from the window starting at 1300 s to the end. Its backlog first fell, to 135–172 sessions 300–1300 s after the burst, as short burst sessions finished, then rose; over the last 300 s it was 206–369 sessions,

still growing in five runs and flat in the sixth. The five trapped runs still completed 0.081–0.097 sessions/s (1.13–1.35 λ_{ref}), above λ_{miss}, because the backlog's sessions are young and their turns short, so that at about the all-miss token rate more turns, and more sessions, complete (§3.3), but below the arrival rate, and the sixth completed sessions about as fast as they arrived (1.28 and 1.27 λ_{ref}); only the uncached prefill rate sits at the all-miss level, which is why the registered rule uses tokens. The CAPPED gate's queue emptied 1547–1904 s after the burst at 1.35 λ_{ref} and 2021–2693 s at 1.65 λ_{ref}, 3–52% longer than the fluid estimate of §3.4 from the population present at the burst (1453–1508 s and 1757–1804 s; the registration's cruder estimate was 1571 s and 1889 s), and well before the evaluation window. Outside the registered criteria, the CAPPED arm's hit fraction dips while the gate drains the burst: its lowest 100 s window is 0.48–0.77 in four of six runs, against 0.82–0.89 for NOBURST, which plausibly slowed the drain. The cap has a cost that the registered criteria do not measure: it turns the burst into admission delay at the client. In each CAPPED run 349–543 sessions waited at the gate, 45–66% of them ordinary arrivals queued behind the burst, for 405–471 s on average and at most 853–985 s; outside the drain the gate bound only briefly, when arrivals clustered (at most 40 sessions in a run, each for less than 50 s). Without the cap, turns completed in the BURST arm's evaluation window took a mean of 163–237 s from send to completion (the client does not stream), against 1.4–1.7 s in NOBURST and CAPPED and 1.4–1.5 s in every arm before the burst; at 8700 s, 19–29% of BURST sessions were unfinished. So at loads that the engine carried from empty for the full 8700 s run (E0b carried the bracketing rates, 1.25–1.75 λ_{ref}, for 3000 s), 200 sessions arriving within a minute, an order of magnitude more than the 8–12 present in the good state, moved it into an all-miss state that it did not leave within 7500 s, and a cap of 20 sessions, within 5% of the measured peak of $\mu(n)$ (argmax $n = 24$), prevented that at the price of that admission delay. The registered basin test BASIN-1 (§7) varied the burst at 1.65 λ_{ref} with the same seeds: 100 sessions trapped the engine in 3 of 3 seeds (hit fraction 0.000; trapped by the rule) and 25 left it good in 3 of 3 (hit fraction 0.923–0.934, within 0.003 of the no-burst run of the same seed), so in each seed the boundary lies between 25 and 100 sessions. The heuristic F2 of §6 puts the probability that a run at these rates leaves the good state on its own within 8700 s at about 0.5% at 1.35 and 5% at 1.65 λ_{ref}, so good NOBURST arms over 8700 s are the expected outcome, not evidence that the good state never ends; on the fresh-seed from-empty runs F2 overstated the chance of instability (§5.1).

A second registration was filled by the same formulas from the native curve, the native E0a calibration and the native E0b bracket, and committed before the first native E2 job (Appendix B). Its window is (1, 5.5) λ_{ref}; the one-third/two-thirds rule, written down before native E0b was read, gives 2.5 and 4.0 λ_{ref} (0.1795 and 0.2872 sessions/s); the cap rule gives $n_{\text{cap}} = 64$, the peak (flagged ambiguous: $\mu(80)$ is within 0.35% of $\mu(64)$). The protocol, the burst of 200 sessions and the trap rule are E2's, with the native reference rates.

The registered analysis passes at both rates, and every run is in the predicted state (Table 1, bottom; Fig. 7); by the definition of §4.3 this test, too, is pre-registered. All six BURST runs were trapped: hit fraction 0.000 over the evaluation window, an uncached prefill rate of 1.074–1.078 (2.5 λ_{ref}) and 1.106–1.124 (4.0 λ_{ref}) times the native all-miss rate of 11,282 tokens/s, and mean backlogs of 557–601 and 1098–1139 sessions with slopes above the threshold. All six NOBURST runs were good and all six CAPPED runs recovered (hit fractions 0.929–0.935, backlogs of 16–34 sessions, most asleep in tool calls). Over the window the base arrivals were 2.48–2.73 λ_{ref} at 2.5 and 3.89–4.28 at 4.0. Counting each seed once, the post-hoc sign test again gives $p = 1/8$. Pooled post hoc over the two pre-registered SGLang tests (a descriptive count, not a registered quantity: the tests differ in pool, cap and rates, and reuse the same three seeds, so the 12 runs of each arm share three seeds' session sequences), NOBURST was good and CAPPED recovered in 12 of 12 runs each, and BURST was at zero hits in 12 of 12 and trapped by the rule in 11 of 12.

With the larger pool the collapse took longer and the backlog never drained (Fig. 7). In the 100 s window that contains the burst, the native BURST runs still hit with 0.99–1.00 of their follow-up turns (all but 4 of 3,326), against 0.43–0.59 at 300k; counting the burst's own first turns, which always miss, they hit with 0.63–0.76 of all turns (0.21–0.33 at 300k). Every BURST run was at zero hits from 300–600 s after the burst to the end of the run. In 50 s bins its backlog fell after the burst only at 2.5 λ_{ref}, to 155–168 sessions 150–200 s after it; at 4.0 λ_{ref} it grew from the first bin after the burst (201–231 sessions). It then grew to 658–1379 sessions over the last 300 s. The trapped engine completed 0.103–0.146 sessions/s (1.44–2.03 λ_{ref}), above λ_{miss}, because its backlog's sessions are young and their turns short (§3.3), and well below the arrival rates; it prefilled 1.074–1.124 times the 32-session all-miss token rate, as the simulator predicted (§C.1). The CAPPED gate's queue emptied 512–583 s after the burst at 2.5 λ_{ref} and 837–937 s at 4.0, between 4% shorter and 13% longer than the fluid estimate from the population present at the burst (491–518 s and 840–876 s), and 10–23% shorter than the registration's cruder 668 s and 1043 s. Here 234–448 sessions waited at the gate, 128–180 s on average and at most 247–361 s; outside the drain the gate bound for at most 6 sessions in a run, each for at most 7.1 s. BURST turns completed in the evaluation window took 321–511 s on average, against 1.6–2.5 s in NOBURST and CAPPED, and 38–50% of BURST sessions were unfinished at 8700 s. Here 200 sessions were 6 to 13 times the 16–34 present in the good state. B3n had predicted a pass at somewhat higher rates (§C.1).

vLLM 0.32.0 passed its smoke test on the same GPU type (token ids and cached-token counts returned, flush verified) and ran with the SGLang cell's settings where they exist: FCFS, prefix caching on, chunked prefill of 8192 tokens, at most 256 running requests and a 300,000-token pool (18,750 blocks of 16 tokens). With prefix caching off, its all-miss capacity (vLLM E0a, three seeds) is 0.0657 ± 0.0030 sessions/s turns-based and $10{,}349 \pm 81$ uncached prefill tokens/s, 8%

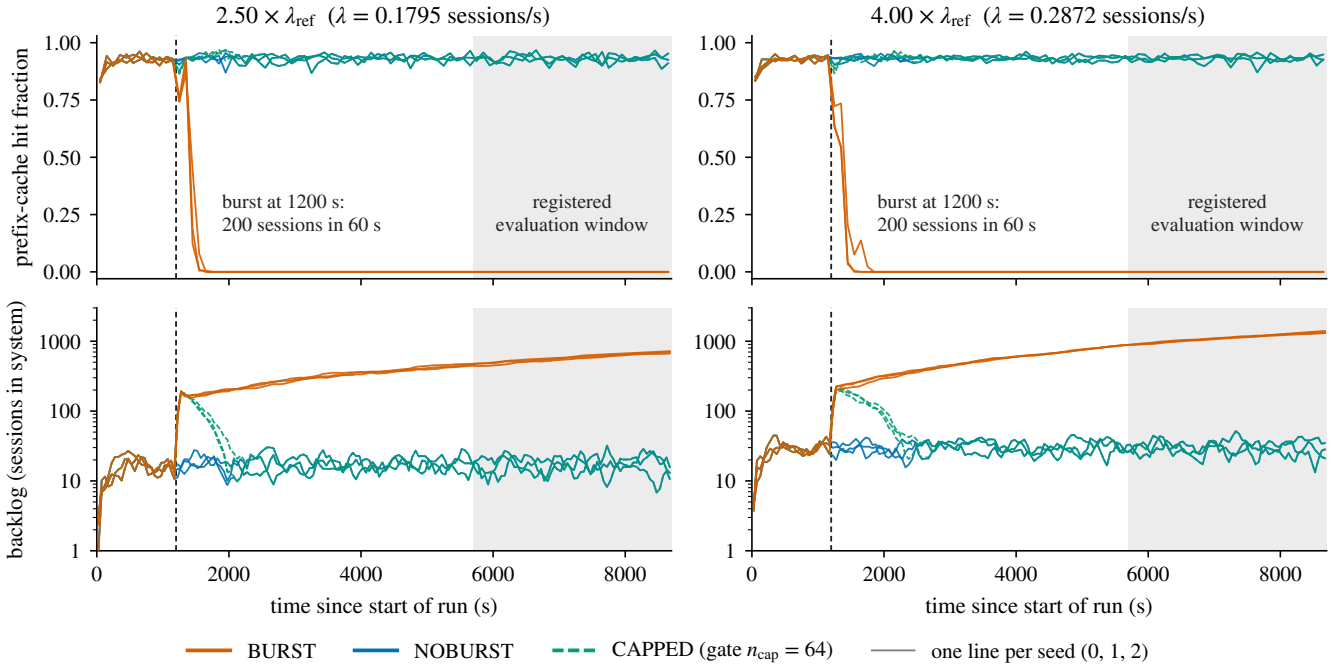


Figure 7: The registered test with the native pool (native E2) at 2.5 λ_{ref} (left) and 4.0 λ_{ref} (right), $\lambda_{\text{ref}} = 0.0718$ sessions/s; layout as in Fig. 1, with the CAPPED gate at $n_{\text{cap}} = 64$ and the backlog axis extended to 3000.

below SGLang's 0.0715 and 11,243 on the same model, GPU and workload.

Six exploratory BURST runs came first, under a registration derived from E2's: the multiples 1.35 and 1.65 of vLLM's own λ_{ref} (0.0886 and 0.1083 sessions/s), seeds 0, 1, 2, and E2's protocol and trap rule with vLLM's reference rates. Their window and multiples were inherited from the SGLang cell, not measured: these runs preceded vLLM's own closed curve and from-empty edge, and no NOBURST or CAPPED arm ran with them. Before the burst all six runs were in the good state (hit fraction 0.93–0.94 from 300 s to the burst; a post-hoc reading). After it, all six were at zero hits from the 100 s window starting 100 s after the burst to the end of the run (also a post-hoc reading), with an uncached prefill rate of 0.997–1.031 times vLLM's all-miss rate and mean backlogs of 203–227 sessions at 1.35 and 277–309 at 1.65 λ_{ref} (Fig. 8, bottom row). The registered rule calls four of them trapped. The other two are indeterminate by the slope test: one has a flat backlog near 203 sessions (slope 0.0002 against a threshold of 0.0115; 1.35 λ_{ref}, seed 1), and in the other the backlog grew from 269 to 294 sessions over the window, at 0.0175 against 0.0213 (1.65 λ_{ref}, seed 1). Both are seed 1, as are the indeterminate SGLang FCFS runs at 1.35 λ_{ref} with and without the refresh. A vLLM run's arrivals are a subset of the SGLang run's at the same seed and multiple, and seed 1 delivered 5.5–7.9% fewer arrivals than nominal in these windows, so the labels on the two engines agree on the same workloads rather than on independent ones. So the same all-miss state appears on a second engine after the same burst. Without control arms these runs are not a registered paired test: they do not show that vLLM's good state would have lasted the run without the burst, or that a cap would have recovered it.

vLLM's own closed curve (Fig. 2, right; one run per n) has the shape of SGLang's at the same pool. $\mu(n)$ peaks at 0.219 sessions/s at $n = 20$, with $\mu(24)$ 0.5% lower, so the peak is a tie at one run per n; $\mu^*/\lambda_{\text{ref}} = 3.34$ (3.21 turns-based). The hit fraction is 0.93 at $n \le 20$, 0.88 at 24, 0.66 at 28, 0.08 at 32 and 0.0075 at 40; it crosses 0.5 near $n = 29.1$, close to N_{lb}, and μ falls 80% from $n = 24$ to 32. The closed curve had no cache hits from $n = 48$ on, as with SGLang's FCFS; Proposition 6, a limit statement, is consistent with this but does not predict it. Like SGLang's, the curve is not monotone beyond its peak: in completed sessions it falls to 0.63–0.67 λ_{ref} at $n = 32$–48 (0.83–0.90 turns-based), whose windows held old, long turns (served prompts 12–17% above the stationary mean), and rises to 1.60 λ_{ref} at $n = 300$ (1.59 turns-based). vLLM preempted running requests in every closed run from $n = 24$ on (12 to 72 per run from $n = 32$ on), which the proposition allows on our reading of vLLM's source (Appendix E).

From an empty cache and queue (Fig. 6, bottom; one 3000 s run per rate) vLLM stayed stable at 1.25, 1.5, 1.75 and 2.0 λ_{ref} (0.082–0.131 sessions/s), ending each run with an empty queue and with 92–93% of turns hitting, and was unstable at 2.5 λ_{ref} (final queue 82, rising by 94 over the last 1500 s) and 3.0 (247 and 175). The edge lies between 2.0 and 2.5 λ_{ref}, as in SGLang's seed-0 grid with the same pool, so vLLM's window is (1, 2.0) λ_{ref}. These runs also used seed 0 and received 8.6–13.3% more arrivals than nominal (279 to 642 sessions; the runs labelled 2.0 and 2.5 λ_{ref} received 2.21 and 2.74 λ_{ref}), but the unstable 2.5 run's realized 0.180 sessions/s

is below the measured peak (0.219, or 0.211 turns-based), so here, as at 300k on SGLang and unlike with the native pool, the edge lies below the peak.

Blind predictions for this cell, B4v for the curve and B2v for the edge, were made with the calibrated simulator in two variants, unchanged and (V1) with its token costs scaled so that its all-miss token rate matches vLLM's measured one (a fit to vLLM's E0a, not a prediction), each in both batching modes, and are frozen in `notes/PREDICTIONS_vllm.md`. They were committed and pushed to GitHub before any of the curve and edge runs had finished, but 22 minutes after the jobs were submitted, when two of them had been running for almost as long; they are blind in that no result existed, not in submission order. The peak level and the cap held: B4v (V1, mixed batching, 10 seeds) put the peak at $n = 24$ with 0.215 sessions/s turns-based, against the real 0.211 at $n = 20$ (0.210 at 24), and the cap at 20. The real cliff is steeper and earlier than predicted: hit fractions of 0.66 against 0.74 at $n = 28$ and 0.08 against 0.46 at $n = 32$ (0.46–0.53 over the four simulator configurations), and a 0.5 crossing near 29.1 against 31.4–32.6. B2v put the edge at 2.0 λ_{ref} in all four configurations (V1 with mixed batching: 10 of 10 runs stable through 2.0, 7 of 10 at 2.5 and 3 of 10 at 3.0), as measured. The simulator does not model vLLM's block-level eviction or its scheduler and token budget; like vLLM, and unlike SGLang, it has no waiting-prefix refresh.

A registration for vLLM was then filled by the formulas used for SGLang (Appendix B) from this curve, vLLM's E0a and five of the six E0b runs (the 3.0 λ_{ref} run was still running; it came out unstable, which leaves the bracket unchanged). It gives the window (1, 2.0) λ_{ref} with $\lambda_{\text{ref}} = 0.06566$ sessions/s, the points 1.35 and 1.65 λ_{ref} by the one-third/two-thirds rule, which was written in our status notes before any vLLM curve or edge existed, and $n_{\text{cap}} = 20$, flagged ambiguous because $\mu(24)$ lies within 0.5% of the peak. The protocol, the burst and the trap rule are E2's. The points equal the multiples of the exploratory runs; the rule that selected them was adopted for vLLM (`836063a`) after their outcome was known, and the BURST arm reuses their seeds 0–2 and so their arrivals: it is not an independent replication of them. The registration was committed 37 s before the 18 jobs of the paired test were submitted, but reached GitHub 2 min after the submission and 53 s after the first job had started, before any job had finished; by our definition (§4.3) this test is therefore registered but not pre-registered. At 18:21 CDT the memory request of the 15 jobs still pending (17166124–38) was lowered from 128G to 48G so that they could be scheduled; no code, argument or registration changed, and no job had ended. The third prediction round, frozen before the registration existed, also predicted this test (B3v): a pass at both points in all four simulator configurations, with every simulated BURST run at zero hits (7 to 9 of 10 per point trapped by the rule, the rest indeterminate), NOBURST good in 10 of 10 and CAPPED at a cap of 20 recovered in 10 of 10. Because the orchestrating agent that specified that round knew the exploratory BURST outcomes, the BURST-arm part of B3v is blind at most on the predicting agent's side, and there only by instruction and by the agent's own unchecked record: its copy of the repository held those runs (Appendix C).

The registered analysis passes at both rates (Table 1, bottom): at each rate, in two of three seeds BURST was trapped, NOBURST good and CAPPED recovered. Its tool also labels the test pre-registered and decisive, labels it assigns by commit order (as it did for the two SGLang tests); by the definition of §4.3 this test is registered, not pre-registered. Every BURST run had zero hits over the evaluation window and prefilled 0.948–1.018 times vLLM's all-miss token rate. The two runs the rule does not call trapped are both seed 1, the seed with the fewest arrivals in the window (1.24 and 1.56 λ_{ref} at the nominal 1.35 and 1.65). At 1.35 its backlog stayed flat at about 200 sessions with an uncached prefill rate of 1.007 times the all-miss rate, and the rule labels it draining because its fitted slope was -5×10^{-5} sessions/s: it was stuck, not draining. At 1.65 it was indeterminate by the slope test (0.0183 against 0.0213). The pattern repeats the exploratory runs, whose arrivals these BURST runs share. Over the last 300 s the BURST backlog was 196–250 sessions at 1.35 and 299–383 at 1.65 λ_{ref}; turns completed in the window took 180–261 s on average, against 1.4–1.6 s in the other arms, and 20–31% of BURST sessions were unfinished at 8700 s. NOBURST stayed good and CAPPED recovered in all six runs each (hit fractions 0.930–0.933). In each CAPPED run 322–499 sessions waited at the gate, 43–63% of them ordinary arrivals, for 413–526 s on average and at most 1053 s, and the gate's queue first emptied 1521–2735 s after the burst, 1.05–1.63 times the fluid estimate from the population present at the burst. B3v had predicted a pass at both rates; the trapped runs' token rates lay at or below its range (0.948–1.018 against 0.99–1.07). The first-principles model, pushed at 02:05:30 CDT, three minutes before the registered verdict was computed and, by the log of the agent that operated the registered vLLM test (`notes/LOG.md`), before any of these run directories was opened, predicted good-state hit fractions of 0.932 and gate drains of 1449 and 1725 s, short of the measured 1521–2027 and 1848–2735 s, as in three of the four SGLang cells (Table 3); the theory's own blindness declaration does not cover these runs. Across the three registered tests, BURST was at zero hits in 18 of 18 runs and trapped by the rule in 15, NOBURST was good in 18 of 18 and CAPPED recovered in 18 of 18; the tests share seeds, and vLLM's BURST arm shares arrivals with earlier runs, so these counts are descriptive.

Mechanism predictions for this test were frozen and pushed at 18:20:13 CDT on 2026-10-30, before any of its 18 runs had ended (`configs/r11_mechanism_predict.yaml` and `notes/PREDICTIONS_r11_mechanism.md`, commit `f1338e3`; byte copies under `r11_mechanism/frozen/` in the test's data directory). They were: (a) in NOBURST and CAPPED, a median gap between a session's turns below 0.05 times the pool's turnover time $R_M = M/\text{churn}$ (§6.4) and a follow-up hit fraction above 0.95, and in BURST a ratio above 2 and a hit fraction below 0.05; (b) in BURST, a mean KV usage of at least 0.90 and an uncached prompt-token rate within 15% of the all-miss rate, that is, a full pool rewritten at the all-miss rate; (c) an online detector, a follow-up hit fraction

below 0.2 over 60 s, alarming within 300 s of the burst in every BURST run and never after 600 s in a good arm; (d) an engine rule, a mean time to first token above 30 s over 100 s, behaving the same way. The frozen decision rule required (b) and (c) in every BURST run and (a) and (c) in all good-arm runs but at most one. (a), (b) and (d) held in all 18 runs: the gap ratio was 6.44–8.78 in BURST and 0.014–0.022 elsewhere; in BURST the KV usage was 0.920–0.937 and the uncached rate 0.871–0.955 times the all-miss rate; and the engine rule alarmed 170–210 s after the burst in BURST and never in a good arm. (c) failed. The detector alarmed 100–120 s after the burst in every BURST run, but also in two CAPPED runs, seed 2 at both rates, 880 and 820 s after the burst, while the gate was draining with 20 sessions admitted and 123–143 waiting; the follow-up hit fraction dipped to 0.06–0.17 for 20–40 s and was above 0.8 again 110–160 s later. The frozen decision rule is therefore not met. The detector uses CONCUR's hit-rate threshold (0.2) without its KV-usage condition [18]; we do not claim it, and a deployed alarm on this rule would have fired twice without a trap. The bands of (a)–(d) were chosen on the SGLang runs and on the exploratory vLLM runs, whose arrivals the BURST arm repeats, so only the NOBURST and CAPPED checks are out of sample.

G.3 The rescue tests in detail

Table 16 gives the registered runs one by one. In every run the cap was engaged within 0.003 s of t_{on}, every pause was matched by a resume, no session was paused or gated at the end, and the in-job mechanical check passed; the smoke job ended with 142 sessions gated, 20 in the engine and none failed. Hits came back within three minutes of the engage and draining the backlog took the rest of the capped phase: over 300 s bins the follow-up hit fraction was 0.56–1.00 during R1's capped phase, and in R4 it dipped to 0.47–0.57 in the 300 s bin from 3000 s (0.56–0.78 in the bin before), while the native backlog in the engine, at the gate and paused peaked near 430, before it reached 1.00 once the queue drained. The registered RECOVERED test, evaluated every 10 s, first held 540–700 s (R1) and 300–350 s (R4) before the release, while the cap was still on, and the cap last held a session back 301–310 s before each release, the hold the release rule requires. Each paused session waited 386–569 s on average and the longest total pause of one session was 817–1240 s; averaged over all sessions of a run, pauses took 30–63 s. Every session of the cohort that arrived in [1200, 3000) s completed in every rescue run, with a median duration of 630–909 s (300k) and 1138–1146 s (native), against 62–73 s without the burst and 793–1099 s in the uncapped controls, where 2–24% of that cohort had not finished at 8700 s. After release the uncached token rate was 0.08–0.12 (300k) and 0.23–0.27 (native) times the all-miss rate, and every 300 s window had a follow-up hit fraction of at least 0.938. Against the simulator's exact-plan predictions, the engine paused 0–13 more sessions, its client queue held up to 1.5 times as many sessions with the native pool (387–397 against 258–342), its delay there was 1.5–3 times as large, and its completed shares equal the predicted ones to four decimals in 6 of 9 runs. The persistence, cost and trace tables behind §5.3 and Fig. 3 are in `results/rescue_paper_data/20261031-071938/` in the data directory, written after the verdicts by `figures/rescue_paper_data.py`.

Table 16: The registered rescue runs (R1: 300k pool, cap 20 from 1800 s; R4: native pool, cap 64 from 2100 s). Outstanding at t_{on}: requests in flight plus sessions held at the client. Sim.: the release time the simulator predicted on the run's exact arrival plan (tag `r11-rescue-pred`). Held: the largest number of sessions waiting at the gate or paused. After: the largest number of sessions in the system after release. Delay: session-hours of pauses and admission waits.

$\lambda/\lambda_{\text{ref}}$	seed	outst. at t_{on}	t_{rel} (s)	sim. (s)	paused	held	after	delay (h)
1.35	0	146	4550	3960	124	134	16	53.1
	1	148	3980	3470	130	146	17	38.4
	2	139	4400	3940	121	148	20	65.4
1.65	0	172	4920	4900	141	158	23	77.8
	1	171	4430	4440	143	159	21	50.1
	2	143	4570	4280	138	145	27	60.8
4.0	0	324	6000	4250	251	397	55	222.0
	1	322	5980	5180	240	397	43	229.6
	2	298	6140	4410	220	387	49	225.0

G.4 Exploratory policy grid and stock LPM

Three other engine cells ran the BURST arm at the registered rates and seeds (Fig. 8) under exploratory registrations derived from E2's, with no control arms. In these runs the refresh made no difference for FCFS: all six runs without it, which share their arrivals with the registered runs, have the registered runs' states, hit fraction 0.000 and backlogs within 1.3% of theirs (201–231 sessions at 1.35 λ_{ref}, 282–328 at 1.65), as the source predicts: the refresh moves waiting prefixes up the LRU order but does not pin them. Both rates lie inside the window; near λ_{miss}, with a cap or with the native pool, the refresh was not tested.

LPM partly helps. From the 200-session burst, at 1.35 λ_{ref} it recovered in 4 of 6 runs and was draining in a fifth (hit fractions over the evaluation window; RECOVERED is judged on the last 300 s, where every recovered run was at 0.92–0.94; with the refresh: recovered at 0.71 and 0.81, draining at 0.51; without: recovered at 0.80 and 0.93, trapped once). At 1.65 λ_{ref} it was trapped in 4 of 6, indeterminate in one (hit 0.44) and recovered in one (0.82). The paths behind these window means differ from FCFS's, which had no hits from 100 s after the burst in all twelve FCFS runs (a post-hoc reading in 100 s windows). Each LPM run that ended trapped kept some hits until 1500–1900 s after the burst (3900 s in one run) and then had none; each run that recovered first exceeded a hit fraction of 0.8 3100–6500 s after the burst and stayed above it to the end, so that had the runs ended 2000 s earlier, only two of the five would have met the RECOVERED rule. This is a finite-burst result for the BURST arm alone, not an estimate of LPM's congested service rate μ_∞; with mixed outcomes

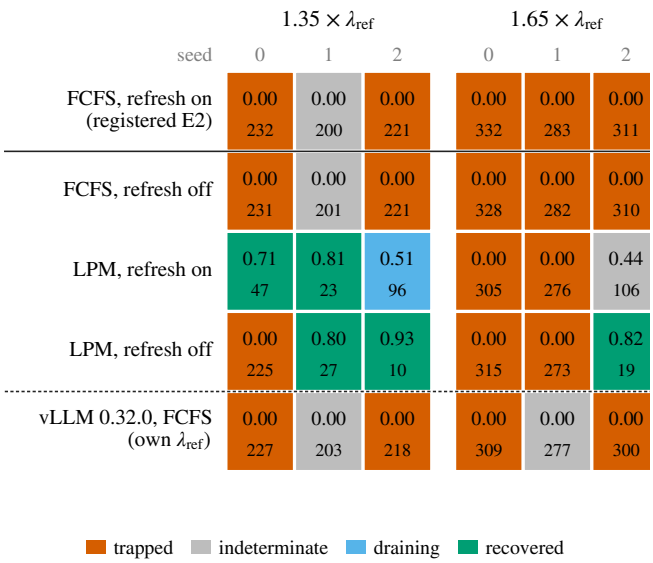


Figure 8: Exploratory policy grid, BURST arm only, 300k pool. Rows: FCFS with the waiting-prefix refresh (the registered runs), FCFS without, LPM with, LPM without (under LPM the refresh acts only during the FCFS fallback above 128 waiting requests); below the dashed line, the exploratory vLLM 0.32.0 runs with FCFS (§5.2), at multiples of its own λ_{ref} (0.0657 sessions/s). Columns: 1.35 and 1.65 λ_{ref} by seed. Colour: registered state (RECOVERED judged on the last 300 s); numbers: hit fraction and mean sessions in the system over the evaluation window.

in every cell, three seeds do not bracket a threshold. Post hoc, the timing matches SGLang's 128-request fallback: in the five runs that recovered the engine's waiting queue never exceeded 128 after the burst, and in each of the five that ended trapped the last hit fell in the 100 s window in which the queue last stood at or below 128. This is a correlation in ten runs, not an isolation; Appendix G gives the details and the other candidate causes.

Stock LPM and its fallback. In deep congestion stock LPM stops re-sorting its queue, so, on our reading of its source, Proposition 6 applies to it (Appendix E); consistent with this, Stage C's LPM run at $n = 300$ had zero hits with a queue above 128, and round 5 of our earlier simulator, which modelled the 128-request fallback, found no LPM advantage in congestion. In the same earlier, uncalibrated simulator, pure LPM without the fallback has a congested service rate of about 1.17 times that simulator's own λ_{miss} and recovered from 300 cold sessions only up to about 1.1 times it (§I.2); these are deep-congestion quantities and do not compare directly with recovery from a finite burst. We do not claim that LPM fails, and we have not isolated why it recovered. Post hoc, the timing matches the 128-request fallback: in the five runs that recovered, the engine's waiting queue (5 s samples) never exceeded 128 after the burst (at most 116–126), and in each of the five that ended trapped the last hit fell in the 100 s window in which the queue last stood at or below 128. This is a correlation in ten runs, not an isolation. An exploratory intervention followed (LPM-INF, below), which switches off LPM's ordering fallback and the LRU touch of its prefix matching together. Other candidates are partial prefix hits and LPM's own prefix matching, which moves the cached part of every waiting request's prefix to the recent end of the LRU order on each scheduling attempt; the simulator omits both. Under LPM the waiting-prefix refresh is skipped, because that matching already touches the waiting prefixes, so the two LPM configurations differ only while more than 128 requests wait and LPM has fallen back to FCFS. Seed 0 at 1.35 λ_{ref} recovered with the refresh and was trapped without it; since the refresh made no difference under FCFS, single runs near LPM's threshold appear sensitive to small differences.

LPM without its fallback (LPM-INF, exploratory). To test whether the fallback causes stock LPM's collapse, a job-local hook raised SGLang's 128-request limit to 10^9 and changed nothing else, and the BURST arm reran at 1.65 λ_{ref} on seeds 0–2 with byte-identical arrival plans (jobs 17168743–45); the exploratory registration, the endpoints with their decision rule and the simulator's predictions were frozen under tag `r11-lpminf-pred` before the jobs were submitted. The controls are the grid's stock-LPM runs of 2026-10-24 and 25, partly on other nodes, with the same arrival plans. On seeds 0 and 1, where stock LPM had collapsed in all four runs (with and without the refresh), LPM without the fallback did not collapse: its token hit fraction over the evaluation window was 0.59 and 0.42, against 0.000, and its waiting queue peaked at 165 and 184 requests, against 287–345. This is the outcome that the rule frozen for the test reads as consistent with the fallback causing stock LPM's collapse. On seed 2 the queue never exceeded 128 (at most 120), so the hook never acted and the run does not enter the rule. It is not a fix: no run recovered by the registered rule (draining, indeterminate and draining on seeds 0, 1 and 2), and on seed 1 the engine still prefilled 0.92 times the all-miss token rate with 194 sessions unfinished at 8700 s. The blind simulator predicted the direction, no collapse on any seed, but not the level (token hit fractions of 0.08–0.10) or the registered state, trapped in its prediction and in none of the three runs; it was flagged in advance as pessimistic about LPM, because it has no partial prefix hits. With the fallback off, LPM keeps both its ordering and the LRU touch that its prefix matching gives waiting requests, so the test does not say which of the two prevents the collapse. Replicate noise in this partial-hit regime is large: on seed 2, three LPM runs that were in effect one configuration, since none of their queues passed 128, gave token hit fractions of 0.44, 0.56 and 0.82. With one rate and two informative seeds this is directional evidence.

G.5 The native-pool prediction round

The native-pool round. A second round (B1n–B4n) reran the same simulator with only the pool changed to 955,757 tokens. B2n and B3n were committed and pushed before any native E0b run had finished, 23 minutes after the native E0b jobs were submitted, and before any native E2 job existed, so B3n is blind in time and B2n is blind in that no result existed, not in submission order; their specification, like that of B1n and B4n, was written after the native curve was known (below), and B3n's cap of 64 is B4n's output. B2n held in part. Both modes predicted stability at all five rates where the engine was stable (10 of 10 seeds each), but the primary (mixed) mode put the all-stable edge at 6.5 and kept 10 of 10 runs stable there, where the real run was unstable, so its edge is too high by at least one step of its 0.5 grid in nominal multiples (the real 6.5 run's arrivals came to 6.86 λ_{ref}, above the real peak, and B4n puts the simulated peak 11% above the real one); the secondary (separate) mode put the edge at 6.0, with 8 of 10 runs stable at 6.5, consistent with the real bracket. Because the real edge came in lower, the registered rates (2.5 and 4.0) lie below B3n's own test points, which are 6–16% higher in multiples of λ_{miss}, and B3n was not simulated at them; 2.5 lies below every point it ran, toward the low end that it had flagged as its weak spot (9 of 10 seeds trapped at 2.85, the tenth recovering). The pass held at both registered rates. The levels are consistent with the lower rates, not a direct match: the trapped token rate rises with the rate in both the simulator and the engine, and the real 1.074–1.078 at 2.5 λ_{ref} lies just below the 1.086–1.109 predicted at 2.65–2.85, while the real 1.106–1.124 at 4.0 overlaps the low end of the 1.119–1.140 predicted at 4.35–4.65. Outstanding backlogs of 549–594 and 1089–1130 requests lie inside the predicted 460–643 and 1067–1333 (they compare only roughly, as for B3), and good-arm hit fractions of 0.929–0.935 lie inside the predicted 0.928–0.936. B3n had also warned that at its higher point the trapped token rate sits close to the registered tolerance of 15%, so that at higher rates the rule would stop calling a stuck run trapped; the real 1.106–1.124 at 4.0 λ_{ref} is inside the tolerance, by 2.6 to 4.4 percentage points.

B1n and B4n are not blind. The native curve and λ_{miss} had been measured and summarised in our project status notes (`notes/STATE.md`) before this round's specification was written, and the specification's finer grid near the cliff (n = 72, 80, 88) may reflect that; only the LLM agent that ran the predictions was instructed not to read those data and, by its own record (not checked independently), did not. For completeness: B1n's λ_{miss}, 0.0720 sessions/s and 11,239 tokens/s, is within 0.4% of the real 0.0718 and 11,282. B4n put the peak at n = 72, which the real grid does not contain, with μ 11% above the real peak at 64–80; it put the 0.5-hit crossing at 94, against about 93, and the cap at 64, as registered. It flagged that cap as knife-edge ($\mu(64)$ only 0.35% above the 5% threshold, where a slightly lower value would have given 72) and the cliff cells n = 80–96 as cells whose outcome differed across seeds; the real cliff lies somewhere between 80 and 96.

G.6 Trace-driven tool times and mechanisms in detail

Trace-driven tool times. TraceLab's sessions come from 52 pseudonymous users (release v0.0.2, a later snapshot than the one the paper describes) [105]. From TraceLab we take the per-turn tool time (mean 8.51 s, median 0.237 s, coefficient of variation 4.65) and, as an upper bound on it, the idle gap between requests (mean 14.52 s). Both are capped at 600 s, which affects 0.18% of turns and 0.38% of gaps but removes 38% and 66% of their raw means (13.6 s and 42.9 s); the per-turn series includes sub-agent runs and can include waits at permission prompts. Only TraceLab's tool times are used: its prompts average 172k tokens, more than the testbed's 49,152-token context limit. The mini-SWE-agent law comes from 500 published bash-only trajectories, the same 100 instances for each of five models (mean 1.40 s, median 0.104 s, coefficient of variation 4.29); 0.56% of its steps hit the harness's 60 s command limit, so the law is specific to that limit. The exponential control reproduced the record workload exactly and passed at both points. A secondary workload with mini-SWE-agent's mean turn and token counts (geometric turn counts) passed at 1.65 λ_{miss} and failed at 2.35. There CAPPED (cap 12, the curve's peak) had left the trap, but at $\mu(12)-\lambda \approx 0.022$ sessions/s it needs about 9000 s to drain 200 sessions, longer than the 7500 s the run lasts after the burst; NOBURST was good in 8 of 10, and the other two collapsed without a burst. This workload's sessions outlive the declared measurement windows (λ_{miss} is 0.0120 sessions/s as declared, 0.0141 with a 20 times longer window, in which units the failed point is 1.99); a labelled post-hoc rerun with longer horizons gives, in those units, a narrower window, [1, 1.5], a pass at 1.35, and at 1.15 a failure of the slope rule (3 of 10 trapped, all 10 stuck below a hit fraction of 0.2).

Mechanisms in detail. *TTL pins.* Continuum's released preview pins for 2 s, and only when the parsed tool's historical mean is at most 2 s [58]. We did not implement the paper's estimator. The preview's release rule is: when an allocation fails and nothing runs, release the pin that expires last; under it, at τ = 20 s the system recovers from a cold start up to 4.0 λ_{miss} and from 5 of 5 bursts. The rule we read in the paper's deadlock-prevention text is: when a request does not fit, unpin the latest-arriving programs until it fits, also while work runs; under it, τ = 10 and 20 s recover from no cold start at 1.1 λ_{miss} or above and from no burst. Our earlier rule, which waited for expiry while the GPU idled, gave 1.719 at τ = 20 s. With plain FCFS order instead of Continuum's, under that earlier rule, the pins gave 0.29, 0.17 and 0.10 at τ = 5, 10 and 20 s (3 seeds), with the GPU busy 33%, 17% and 8% of the time: that simulator keeps an expired pin while the session's next request waits, which under FCFS lasts the rest of the queue wait. *Adaptive caps.* With finished sessions' KV counted as usage, the 12 of 18 cells that stall do so at our surrogate's minimum window of 4, at 0.915–0.935 λ_{miss}; of five exploratory sensitivities, one (a 5 s control interval instead of our surrogate's 10 s) raised the rate by 0.184 ± 0.167, an

unadjusted pointwise interval. Pause and resume of admitted agents is not modelled.

G.7 The pre-deployment procedure in detail

The window of Eq. (2) comes from three measurements that an operator can make before deployment, on the engine, model, pool and workload that will be served. (1) *All-miss capacity*: run a saturated closed population with prefix caching disabled and measure the uncached prefill token rate, which is stable across seeds; check saturation by measuring at two populations (for example 32 and 64 sessions) and use the larger rate, since a population that is not saturated understates λ_{miss} (our 32-session reference fell about 2–3% short, Appendix E); convert it to sessions by the mean work per session, or use turns per second over $E[T]$ measured over a window longer than a session's lifetime at that population (mini-SWE-agent's sessions outlived the declared 1800 s window, and the turn rate there gave a λ_{miss} 15% low); completed sessions over a short window are noisy. By Proposition 6 and the fluid model of §3.3, under request-level FCFS this is an upper bound on the recovery threshold from deep congestion if the congested engine computes prompt tokens no faster than the cache-disabled one, and it equals that threshold if, in addition, $\mu(n) \geq \lambda_{\text{miss}}$ at every population beyond the peak and not only in the limit (with zero hits, if the engine computes prompt tokens as fast as the cache-disabled one at every congested population); the heuristic work balance of §3.3 leads to the same conditions for the open system. With zero hits our engine computed prompt tokens at 1.03 times the all-miss rate near the stationary turn mix and at 0.95–1.12 times it over the windows we measured, so (A3) and (A4) hold on it only to within a few percent at the stationary mix and to within 12% over those windows. If sessions share a prefix (system prompt, tool definitions), measure instead with caching on and each turn's private context fresh, so that only the shared prefix can hit, the reference against which we expect Proposition 6 to hold for the private suffixes (Appendix E); disabling caching entirely understates the congested rate. (2) *Closed-population curve*: measure $\mu(n)$ on a grid that crosses the cliff, and do not read μ_∞ off the largest n of a short run with a synchronized start; here $\mu(300)$ was 0.108 sessions/s, 51% above λ_{ref}. (3) *From-empty edge*: run open arrivals from an empty engine at increasing rates over the horizon of interest, at least that of the burst test, with more than one arrival seed or in realized rates: at 300k, 2.5 λ_{ref} was unstable on the original seed (realized 2.73 λ_{ref}) and stable on three fresh ones (realized 2.47–2.67), and ordered by realized rate the 12 runs place the edge between 2.665 and 2.726 λ_{ref}; on the H200, 1.4 times its λ_{ref} was stable in 2 of 3 seeds, and in realized rates its ten runs are monotone, with the edge between 1.496 and 1.503 λ_{ref}.

If $\mu^* > \lambda_{\text{miss}}$, then in the fluid model of §3.3, under request-level FCFS and the assumptions of Proposition 6, every operating point in $(\lambda_{\text{miss}}, \min(\mu^*, \lambda_{\text{e}}))$ is one sufficiently large burst away from the congested state unless admission is bounded; a steady-state load test there passes without testing recovery. On the engine we tested two rates per pool with a burst of 200 sessions, about 20 times the 8–12 present at 300k and 6 to 13 times the 16–34 present with the native pool, which left all 12 registered SGLang BURST runs at zero hits (11 trapped by the rule); at 1.65 λ_{ref} on the 300k pool the registered test BASIN-1 found that 100 sessions also trap (3 of 3 seeds) and 25 do not (3 of 3), as in the simulator, where 25 never trapped and 50 trapped about half the time. Without a cap, Proposition 6 and the fluid model predict that recovering from deep congestion requires cutting the arrival rate below λ_{miss}, which was 31% of peak throughput at 300k tokens and 15% with the native pool; the earlier, uncalibrated simulator agrees (over 150,000 s, FCFS recovered from congestion at 0.9 times its own λ_{miss} and not at 1.0), but no completed real run has lowered the rate after a trap; the registered SHED test of §7, which stops arrivals for a fixed time, is the first, and its outcome is pending. Stock LPM, which on our reading of its source stops re-sorting its queue when more than 128 requests wait and so falls under Proposition 6 in deep congestion, recovered from the same finite burst at 1.35 λ_{ref} in 4 of 6 grid runs, and at 1.65 λ_{ref} in 1 of 6 (§5.6); with the fallback disabled it avoided the all-miss collapse at 1.65 λ_{ref} on the two arrival streams on which stock LPM had collapsed, without recovering (exploratory, §I.2); the proposition is a limit statement and does not predict recovery from a finite burst for any ordering. Corollary 1 says that a cap drains a burst, or an established trap, only if it lies inside (n_1, n_2), and the drain is fastest where $\mu(c) - \lambda$ is largest, at the peak. A cap at the capacity boundary $N_{\text{lb}} = M/E_{\text{lb}} \approx 28$, the size a capacity-ratio rule gives at 300k, lies on the steep side of the curve, within a few sessions of n_2 (31.8–32.2 at the registered rates), where the quasi-static model is least reliable; in the earlier simulator a cap of 28 held 2.39 λ_{miss} against 4.04 for a cap of 20 (§I.2). We take the smallest n within 5% of the peak (20 at 300k and on vLLM, 64 with the native pool), because the curve has one run per n, and check it with a burst test. We recommend a cap because it needs no engine change, its effect in the quasi-static model follows from Proposition 1 and Eq. (1), and it is the one mechanism we tested on the engine (all 12 registered SGLang CAPPED runs recovered), not because it dominates: in the earlier simulator, 20 s TTL pins with a pinned-first order reached a higher $\mu(300)$ than any static cap (§I.2). A cap does not make the burst free: it moves the burst's cost to the client as admission delay (per-run maxima of 853–985 s at 300k and 247–361 s with the native pool), instead of turns that take several minutes for the rest of the run (§5.2, §5.2). The delay ends only if the gate drains, which in the quasi-static model needs $\lambda < \mu(c)$ for a cap c (Proposition 1 and Eq. (1)). Whether the gate should queue or reject sessions is a deployment choice we did not evaluate.

The cap must be measured again when the workload or configuration changes. In the simulator the peak moved from 20 to 32 or 40 sessions with the TraceLab tool-time laws and to 24 with mini-SWE-agent's (where the cap stayed at 20), and fell to 12 with mini-SWE-agent's mean turn and token counts. On the real engine it moved to 64–80 with the 955,757-token pool, where the registered cap of 64 drained the burst in every run, and with vLLM 0.32.0 on the 300k pool

it stayed at 20–24. On an H200 with the 300k pool it moved to 32 (the smallest-within-5% rule gives 28), and the window narrowed from $(1, 2.0)$ to $(1, 1.3)$ times that GPU's own λ_{ref}: the all-miss rate rose 2.71-fold over the A100's, the peak only 1.63-fold (turns-based; 1.65 in completed sessions) and the from-empty edge, in nominal multiples on each GPU's seed-0 grid, 1.76-fold (§7). A faster GPU can thus leave too little room between λ_{miss} and the edge for our registration rule to place a burst test. A cap sized for one setting can fail in another in either direction. Too small, it holds the engine below the offered load: a cap of 20 under the TraceLab idle-gap law gives $\mu(20) = 1.26\,\lambda_{\text{miss}}$, below both test rates (1.35 and 1.65 λ_{miss}), so the gate would not drain. Too large, it sits on the collapsed side of the curve: with mini-SWE-agent's full workload $\mu(20)$ is 1.45 times that workload's declared λ_{miss}, again below both test rates (1.65 and 2.35). Neither mis-sized cap was run. The procedure sizes a static cap offline. An online detector is a separate question: a hit rate below 0.2, with KV usage above 0.5, is CONCUR's thrashing signal [18], MISA-T confirms a decline relative to a learned reference hit rate [45], hit rate is among Bronson et al.'s characteristic metrics [10], and learned early indicators have been proposed [66]. Our frozen version of the first, on the client's follow-up turns, alarmed 100–120 s after every vLLM burst but also twice during capped drains (§5.2); and with sessions that share a system prompt the raw hit fraction of a trapped engine would stay near the shared share, so a detector would have to count private-suffix reuse. We do not claim a detector. Here its three stages took 14–16 A100-hours per pool and engine, and a three-arm burst check about 7.5 more per rate and seed.

G.8 Threats to validity in detail

Synthetic sessions and clients. Sessions are synthetic (random tokens, fixed tool outputs and output lengths, geometric turn counts, exponential 5 s tools) and share no prefixes, which removes the case PEEK's scheduler targets. These parameters were fixed before we measured any trace, and they differ from the two trace sources of §I.1. Our mean tool time of 5 s lies between theirs (1.40 and 8.51 s), but their medians are 15–33 times shorter than ours, 63–85% of their tool times are under 1 s against 18% of ours, and their coefficients of variation are 4.3–4.7 against 1. Their outputs per call are 2.7–5.9 times our 100 tokens, and TraceLab's mean prompt, 172k tokens, exceeds our 49,152-token context limit. Apart from one secondary mini-SWE-agent workload, the trace-driven runs of §I.1 replace only the tool-time law, and only in the simulator. With no shared prefix a miss reuses nothing from other sessions. A shared 2000-token system prompt, cached once, would cut the prompt work of a session from 154,529 to 125,007 tokens when every turn misses and from about 9,600 to 7,600 when every follow-up hits, so the ratio of the two, about 16, would barely change. Production agent traces report a hit rate of about 55% across turn boundaries and user idle gaps of minutes [62], while 93% of our turns hit in the good state (not the same metric). The simulator suggests that real tool-time laws keep the trap but move the window and the cap; the engine has not tested this. Our clients never retry or abandon a request, and the 7200 s request timeout never fired, although trapped turns took 163–237 s on average from send to completion at 300k and 321–511 s with the native pool. Retries would add load in the trapped state and abandonment would remove it, so either could change whether the engine recovers; neither was tested. The follow-up hit fraction falls from 0.93 to 0 in a trap only because no prefix is shared; with a shared system prompt the hit fraction of a trapped engine would stay near the shared share, so the hit-based signals of §8 would have to count private-suffix reuse.

Few runs, finite horizons, nominal rates, uncontrolled factors. The closed curves and, apart from the repeats below, the from-empty sweeps have one run per point, so the peak's location (20 against 24 at 300k and on vLLM, 64 against 80 native) is within noise and single runs near the cliff land on either side of it. The registered tests, the grid and the vLLM runs share the same three seeds, so their runs are not independent across rates, pools or engines (§4.1). All rates in this paper are nominal. Each from-empty sweep used a single arrival seed, whose realized rates were 6–13% above nominal, and the edge it gives depends on that seed: on SGLang at 300k, three fresh seeds were stable at both 2.0 and 2.5 λ_{ref}, where the original seed had been unstable at 2.5 (realized 2.73), so the replication rule for the from-empty bracket frozen with the simulator's predictions, which needed 2.5 λ_{ref} unstable in at least two of them, is not met. The nominal bracket $(2.0, 2.5)$ is that of the single original grid; ordered by realized arrival rate, stability over the 12 runs is monotone and the edge lies between 2.665 and 2.726 λ_{ref}. The registered test points, 1.35 and 1.65 λ_{ref}, lie inside the window either way. On the H200 the edge is a stochastic escape band: 1.3 times its λ_{ref} was stable in 3 of 3 seeds and 1.4 in 2 of 3. In the evaluation windows of the pre-registered SGLang tests the runs labelled 1.35 and 1.65 λ_{ref} received 1.27–1.45 and 1.61–1.75 λ_{ref} (native, 2.5 and 4.0: 2.48–2.73 and 3.89–4.28; vLLM BURST: 1.24–1.45 and 1.56–1.76). "Does not leave" means within 7500 s on the engine and 30,000 s in the simulator; the 3000 s horizon of the from-empty runs cannot see slow escapes, so the true edges may be below 2.0 λ_{ref} (both engines) and 5.5 λ_{ref} (native). The native cap rests on that coarse curve: $\mu(80)$ was within 0.35% of $\mu(64)$, and the cliff lies somewhere between 80 and 96 with no run in between. The registered smallest-within-5% rule gives 64 whichever of 64 and 80 is the true peak (the registration still flags the cap as ambiguous), and with no grid point between 48 and 64 it could not have found a smaller cap; a rule that takes the argmax could as well have picked 80, which may sit within one grid step of the cliff; how far a cap of 64 is from the cliff was not measured. The registered rule loses margin at both ends of the window: the slope test near λ_{miss}, and the token test at high multiples, where the trapped engine's prefill rate rises above the all-miss rate (1.106–1.124 times it at 4.0 λ_{ref} with the native pool, against a tolerance of 1.15), so at somewhat higher rates the rule would stop calling a stuck run trapped. In both SGLang closed curves the cliff coincides with a change of job and node (at 300k between

$n = 32$ and 48, native between 80 and 96; Table 6), so neither curve excludes a node or job effect on where the cliff falls. The collapsed side ran on different nodes in the two curves, which argues against one slow node; on vLLM the hit fraction crossed 0.5 between two runs of one job on one node ($n = 28$ and 32); and both registered SGLang tests, which ran on all three nodes with consistent outcomes, argue against a node effect on the trap. The SGLang server seed was not pinned, and the load client runs on the server's node.

Model and simulators. Eq. (1) is used for the window and F2's from-empty edge, the recovery threshold, the capped drain time and F4, the burst sizes of F3, and the frozen SHED and TOOLSLOW predictions. The engine met the capped drain time within −4% to +63% given the population present at the burst (SGLang −4% to +52%, vLLM +5% to +63%). F2 was pessimistic at 300k: its frozen escape probability makes 2.5 λ_{ref} unstable with probability 0.69 per seed, and all three fresh seeds were stable there (probability 0.03 under F2). The freeze declared no prediction for these runs; the probability is read from its frozen files, and pred2's table for them was computed after the runs had ended. The model is not quantitative in congestion and is least reliable near the cliff (§3.3); the SHED prediction applies it in congestion, to a backlog of 161 sessions, and the TOOLSLOW prediction at the cliff, so those tests also test that use. Proposition 6 is a limit statement about the closed-population throughput $\mu(n)$ as n grows and says nothing about any finite population. It covers request-level FCFS and, on our reading of the engines' sources, vLLM's FCFS with its preemption and SGLang's stock LPM, which stops re-sorting its queue when more than 128 requests wait; it does not cover program-level FCFS, pure resident-first orders or eviction rules that foresee tool times. It assumes that long-run rates exist, private prefixes, at most one outstanding request per session, no offloading of evicted KV, i.i.d. tool times with finite mean that are independent of everything else and unknown to the engine in advance, a bounded number of running requests, a bounded admission rate, and an engine that in congestion computes prompt tokens no faster than with caching disabled; it gives equality only if the engine computes them as fast. With zero hits our engine computed them at 1.03 times the all-miss rate near the stationary turn mix and at 0.95–1.12 times it over the windows we measured, so (A3) and (A4) hold on it only to within a few percent at the stationary mix and to within 12% over those windows. The open system's recovery threshold rests on a heuristic work balance and, without a monotone tail of μ, can lie below λ_{miss}; that recovery from deep congestion without a cap needs a rate below λ_{miss} is untested on the engine (the SHED test of §7 stops arrivals for a fixed time instead); pure LPM's congested rate is a conjecture; and the capped system's stochastic stability is not proved. The calibrated simulator failed its level test at $n = 48$, reserves whole turns, omits partial prefixes and the refresh, frees a finished session's KV at once (an oracle: a real engine keeps it as evictable cache), and understated LPM, again with LPM's fallback disabled (token hit fraction 0.08–0.10 predicted, 0.42–0.59 measured; §I.2). In its primary mode it put the native from-empty edge at least one grid step too high in nominal multiples, against a single real run whose arrivals exceeded the real peak, and for vLLM it predicted a later, gentler cliff than the engine showed (hit fraction 0.46–0.53 against 0.08 at $n = 32$). The mechanism comparison uses an older simulator whose assumed costs are about 1.5 times the fitted ones, so its absolute rates and recovery thresholds do not carry over to the engine. Its pin rows are fixed, configured TTLs, not Continuum's cost model; its PEEK rows are ablations and a projection of the released scheduler; and its CONCUR row is a surrogate of the paper's control equation.

The first-principles model. Its propositions hold for a deterministic fluid model, and no limit theorem connects the engine to it. Its mean-field step, Assumption MF, is least reliable when one session is a large share of the pool: at 150k tokens and on the H200 the mean live demand of the predicted peak population is close to or above the pool. On the H200 the model was closest up to $n = 36$, but from $n = 40$ the measured hit fraction stayed above the prediction (0.46 at $n = 40$ against 0.27, then 0.32 at 44 and 0.16 at 52, where the model is all-miss from its onset at 41.3 sessions, and zero only by 96), so that onset came early. The frozen notes named the mean field's lack of concentration at this pool as the likeliest failure, but expected it to bias the peak upward, and the peak was matched; we have not traced the tail miss to a specific assumption and have no correction for it. The H200 predictions assume that a faster GPU divides every cost by one factor, which misfits the H200's ratio of 64-session to 32-session all-miss token rates (1.019 predicted, 1.046 measured). On vLLM the model predicts a later and gentler cliff than measured. Unimodality and fold-freeness are numerical; the trapped layer RR and F1–F4 are heuristics; the model and its edge criterion were selected on the three measured cells; the 300k comparison shares its data source with the cost-model fit; and Proposition 7 has only a proof sketch (§6.6).

G.9 Further related work

What is classical, and what is specific to agent serving, in full. The collapse past the working-set level, the remedy of limiting active jobs, and a bad state that outlives its trigger are classical, and so are metastability arising from a unimodal throughput curve under open load, bistability when the load comes from a finite population of sources (ALOHA terminals with a fixed think time [14]), hysteresis, and bounded admission as the cure, for random-access channels [14, 56], thrashing multiprogramming systems [23, 72, 92, 98] and service systems whose customers return [15, 32, 47]; in all of them the slowdown is a given function of occupancy. The fluid picture of §3.3, a unimodal throughput curve that gives one stable and one unstable equilibrium under open arrivals, is the classical fluid (drift) analysis of unstable random-access channels [37, 56]; network congestion collapse, a stable bad state sustained by retransmissions, is its analogue [49, 71]. Replacing an open system's departure rate by its closed-system throughput is the classical decomposition (flow-equivalent)

approximation, whose use to find unstable regimes of thrashing multiprogramming systems goes back to Courtois [23], and choosing the number of admitted jobs from the throughput curve, by paging heuristics [29] or from measured throughput [43, 82], is multiprogramming-level load control. Two features of agent serving make these apply. The reusable KV is exactly the idle KV: a session in a tool call or waiting in the queue holds no running request, so memory-gated admission reclaims its cache. And reuse makes a hit much cheaper than a miss, so losing the cache multiplies the work of every later turn. Both features are the premise of earlier agent-serving systems [1, 8, 9, 58, 94], and §9 describes those that describe, bound or avoid the resulting loop. What is new on the theory side is Proposition 6, a bound specific to prefix caching, and the first-principles curve of §6, in which the slowdown is derived from the pool, the workload and the server and running reservations depend on the hit rate and, on the all-miss branch, the queue sets the gap between a session's turns, with its closed-form all-miss onset, speed law, bound for exogenous-gap closures with little running KV, trapped identity and fold criterion (Propositions 2–5 and 7).

Other systems. CacheWise evaluates coding agents with sessions started together [88]; XPerf, a benchmark, sees hit rates fall toward saturation in fixed-rate runs at increasing rates [90]. Other systems manage idle KV across tool calls or workflow steps by offloading, prefetching or retention [1, 8, 9, 63, 74, 94, 103], schedule agent programs [61, 65], cache conversation state hierarchically [39, 79, 100] (Mooncake's scheduler also rejects requests early under overload), admit, place or reserve capacity for agent sessions by tool readiness, KV residency or likely returns [51, 60, 70], or make locality-aware scheduling fair [13]. Eviction-order work [64, 75, 76, 93] changes which idle state is evicted; RR-Evict assumes that the active trajectories fit in the cache, ScaleSim prefetches and evicts each agent's state by the estimated order of the agents' next requests, and Liu et al. replay traces without feedback from queueing to arrivals, as do Arya et al. for production Microsoft M365 agent traces, where LRU with a pooled budget of a tenth of the busiest hour's no-eviction demand makes 63% of Deep Research calls recompute [6]. Other close neighbours: Metronome calls a collapse in real-time speech-model serving metastable, in the sense that identical runs collapse or survive near a threshold, but there each session's KV stays pinned for the whole conversation until the pinned KV exhausts the pool, a different mechanism from eviction and recomputation of reusable prefixes [69]; AgentServeSim simulates agent serving with KV retained across tool gaps and successor turns released by simulated completions, so call timing depends on the policy, and validates it against vLLM in 20 paired cells, without a metastability analysis [81]; and ServeTwin simulates multi-turn agent workloads while tracking KV-cache evolution and reports that a pool 1.4 times larger lifts the hit rate from 22% to 93% at a fixed number of users [77].

H Frozen predictions: the full record

This appendix gives every prediction frozen for the later tests (Tables 17 and 18) with its source, its scoring rule and what it is blind to, and the outcomes in hand in detail; §7 summarizes them, §5.5, §5.3 and §5.4 report the H200, rescue and basin outcomes, and Table 12 gives the GitHub push times. Each registered test has a FILLED registration committed and tagged before its first job, except that the Llama-3.1-8B registration (`1559f8e`) was pushed 47 s before its jobs were submitted but not tagged. BASIN-1's reached GitHub at 20:04:30 CDT on 2026-10-30, 68 minutes before its jobs were submitted; R1's and R4's at 22:39:02, 22 minutes before theirs; and BASIN-2's, TOOLSLOW's and SHED's at 02:10:48 and 02:28:09 CDT on 2026-10-31, before their jobs were submitted at 04:21 (Table 9). Every outcome is reported against these rules, misses included.

Out-of-sample predictions. The model's predictions for an H200 at the same pool, for pools of 150k and 600k tokens, for Llama-3.1-8B, for the registered basin tests and for the registered rescue tests were frozen and pushed before any of those results was read by the agents that derived it (the agent that operated the H200 cell and the first author had seen the H200 closed-curve values for $n \leq 32$, which include the predicted peak, §7; tag `r13-theory-pred`, commit `997d2c4`, pushed at 02:05:30 CDT on 2026-10-31; `notes/PREDICTIONS_r13_theory.md`, `configs/r13_theory_predict.yaml`), with primaries, estimators, tolerances and refutation rules fixed in advance (`notes/THEORY_r13_scoring.md`). A second set, computed by new drivers (`analysis/theory/pred2/`) that call the frozen model's functions unchanged (tag `r13-theory-pred2`, commit `cfa7712`, pushed at 04:20:20 CDT; `notes/PREDICTIONS_r13_pred2.md`, `configs/r13_theory_pred2.yaml`) covers the BASIN-2, TOOLSLOW and SHED tests, whose jobs were submitted after that push, and a 600k-token H200 cell; these drivers, including the per-seed burst calibration behind BASIN-2's probability of 0.66 (0.42 uncalibrated), were written after the first freeze but before any of those jobs was submitted. Section 7 lists all of them with their tests. Of the outcomes in hand, the H200 peak ratio, peak position, 0.5-hit crossing and from-empty edge are within their tolerances, while the tail after the H200 peak is softer than predicted; BASIN-1 came out as predicted (probability of the exact outcome 0.879; PASS was also the simulator's call, BASIN-1 does not test the critical burst, and its predictions are blind by access only); and F2, applied to the fresh-seed from-empty runs, for which the first set declared no prediction (the second set's, computed after those runs had ended, is not blind), was pessimistic.

GPU generation. The H200 cell repeats the 300k SGLang cell on one H200 NVL, with SGLang's default attention backend for that GPU (FA3; the A100 cells used FlashInfer), so the comparison is of GPU and default backend together. Its only inputs beyond the A100 cell are its E0a measurements: the all-miss token rate fixes the speed factor $s = 2.742$ of Propo-

Table 17: Frozen predictions and their tests. Each prediction was committed under the tag named before any of its results was read by the agents that made it (for the H200 and BASIN-1 exceptions see their rows; the theory's fresh-seed probabilities were not declared for that test before it ran); scoring rules and tolerances were frozen with it (`notes/THEORY_r13_scoring.md` for the theory; the registration file or `configs/r11_*_predict.yaml` otherwise). "Theory": the model of §6, tag `r13-theory-pred` (§6.5), files under `results/theory/merged/`; "pred2": new drivers calling the same frozen model, tag `r13-theory-pred2` (commit `cfa7712`, pushed 04:20:20 CDT, before the first BASIN-2, TOOLSLOW or SHED job by its freeze record; files under `results/theory/pred2/` at that tag). "Simulator": the calibrated simulator of §4.4. $\omega_{\text{ref}} = \mu^*/\lambda_{\text{ref}}$ (turns-based), with λ_{ref} the cell's measured E0a rate (pool sweep: the A100 300k value, 0.0715 sessions/s; H200 600k: the H200 300k value, 0.194 sessions/s; Llama: the model's a priori λ_{miss}); hit is the fraction of all turns of a closed run that hit (first turns always miss, so at most 0.932), the Stage C estimator; stuck means a hit fraction below 0.2 over the evaluation window.

Test	Frozen prediction	Source	Outcome
H200 NVL, SGLang, 300k pool: curve	$\omega_{\text{ref}} = 1.92$ (band 1.65–1.92 over the five cost variants; fluid 1.66–1.96; A100: 3.27 predicted, 3.15 measured); all-miss onset $n_b = 41.3$ (A100: 32.7); 0.5-hit crossing at $n = 37.3 \pm 3$; hit 0.67 (F1) at the peak, $n \approx 32.5$ (0.69 at $n = 32$, the value the freeze record gives). Refuted if $\omega_{\text{ref}} \geq 2.4$, hit$(32) < 0.5$ or hit$(48) \geq 0.5$; the first two are not blind to the first author, who had seen $n \leq 32$.	theory, `h200_300k*.json`; $s = 2.742$ fitted to the H200's all-miss token rate	Measured, not a registered test (window $(1, 1.3)$ λ_{ref}, too narrow for the registration rule): $\omega_{\text{ref}} = 1.89$ (sessions-based 1.99); peak at $n = 32$; crossing ≈ 38.9 (38.8 on the nine-point curve); hit 0.74 at $n = 32$, 0.22 at 48. No rule fires
H200: cliff bracket, $n = 36, 44, 52$	Hit, fluid: 0.56 at $n = 36$ (variants 0.52–0.57), 0 at 44 and 52; F1: 0.53 (0.50–0.53), 0.08, 0.002 (from the frozen curves, not the freeze record). At the original points $n = 40$ and 48: fluid 0.27 and 0, F1 0.28 and 0.01	theory, `h200_300k*.json` (closed curve)	Hit 0.61, 0.32, 0.16 (crossing ≈ 38.9), and 0.46, 0.22 at $n = 40$, 48: closest at 36; from 40 the tail is softer than predicted, so $n_b = 41.3$ is early. Blind in time (36, 44, 52); by access only (40, 48)
H200: from-empty edge	F2 edge 1.39 λ_{ref} (1.24–1.40 over the cost variants); scored on the original E0b runs 17168377–82, not the later reseeds	theory, `h200_300k*.json`	Stable at 1.3, unstable at 1.4 λ_{ref} (nominal): edge in $[1.3, 1.4)$, contains 1.39. Realized rates: ten runs monotone, edge 1.496–1.503, so F2 about 7% pessimistic, as on the A100. Reseeds (post hoc): 1.3 stable in 3 of 3 seeds, 1.4 in 2 of 3 (F2 probability of staying stable: 0.72, 0.48). Original runs blind by access only
H200 NVL, 600k pool	pred2 (jobs submitted at 02:50–02:51 CDT, before its push, not eligible to start before 05:00; first start 05:27): $n_b = 69.9$, as Proposition 2(c) gives from the 600k A100 onset and s; $\omega_{\text{ref}} = 3.20$ (fluid 3.27); crossing 66.6; F2 edge 2.57 λ_{ref}	pred2, `h200_600k.json`	[pending: Stage C 17169672–80, E0b 17169682–87]
Llama-3.1-8B, SGLang, A100, 300k	Theory, a priori: $M/E_S = 27.8$ for any speed, $n_b = 32.0$, crossing 30.9, $\omega_{\text{ref}} = 3.37$, edge 2.46 λ_{miss}; after the cell's λ_{miss} is measured, every quantity comes from the frozen speed table. Refuted if the crossing lies outside the table's value ± 3, or $\mu(48)$ is within 10% of the peak. Simulator, costs scaled to the measured all-miss rate: peak at $n = 20$, $\omega_{\text{ref}} = 3.52$, crossing 30.6, edge 2.5 λ_{miss}, paired test passes at every candidate rate.	theory, `llama31_8b_300k*.json`; simulator, tag `r12-llama31-predictions-frozen`, `notes/PREDICTIONS_llama31.md`	Registered (`1559f8e`, pushed 07:49:15 CDT on 10-31; 1.35, 1.65 λ_{ref}, cap 20). [pending: Stage C 17166814–16, E0b 17169565–70, E2 17170409–26]
Pool sweep, SGLang, A100, 150k and 600k	$\omega_{\text{ref}} = 1.87$ and 5.47; $n_b = 18.3$ and 61.2; crossing 16.9 and 60.0; F2 edge 1.09 and 4.53 λ_{ref} (second criterion 1.47 and 4.82). Refuted if $\omega_{\text{ref}} \geq 2.5$ or hit$(24) \geq 0.5$ at 150k, or $\omega_{\text{ref}} < 4.5$ or hit$(48) < 0.5$ at 600k. No fitted parameter.	theory, `a100_pool150k.json`, `a100_pool600k.json`	[pending: Stage C 17168864–69, E0b 17168870–81]
Fresh-seed E0b, 300k, seeds 3–5, 2.0 and 2.5 λ_{ref}	Simulator: stable in 3 of 3 at both rates, so the seed-0 bracket is not replicated. Theory (F2): unstable with probability 0.15 and 0.69 per seed.	simulator, tag `r11-basin-reg`; theory, `a100_300k.json`	Not replicated: stable in 3 of 3 at both rates (simulator: 6 of 6 calls). Realized-rate edge, 12 runs: 2.665–2.726 λ_{ref}. F2 pessimistic (all three stable at 2.5: 0.03)
BASIN-1, 300k, 1.65 λ_{ref}, bursts 25, 100 (registered)	Simulator: pass (100 stuck and trapped, 25 good, each in 3 of 3). Theory: stuck with probability 1.000 and 0.042 per seed; pass with probability 0.995.	registration and simulator, tag `r11-basin-reg`; theory, `b1_basin_300k.json`	PASS: 100 stuck and trapped, 25 good, each in 3 of 3 (simulator: 6 of 6 calls). Theory: probability of this outcome 0.879; blind by access only ($n = 100$ runs ended before the push; $n = 25$ runs in progress at it)

Table 18: Frozen predictions and their tests (continued from Table 17, with the same conventions).

Test	Frozen prediction	Source	Outcome
BASIN-2, bursts of 40, 50 and 60 (registered)	Theory: stuck with probability 0.139, 0.547 and 1.000 per seed (critical fresh burst 53.4). Rule: bracketed if 40 is good and 60 stuck, each in at least 2 of 3 seeds. pred2, conditioning on each seed's burst: stuck with probability 0.78, 0.86, 1.00 (seed 0), 0.36, 1.00, 1.00 (seed 1) and 0.09, 0.22, 0.81 (seed 2) at 40, 50, 60; the rule holds with probability 0.66 (bands 0.23–0.88; 0.94 for average bursts)	registration, tag `r14-basin2-reg`; theory, `b1_basin_300k.json`; pred2, `b2_basin.json`	[pending: BASIN-2 verdict]
Rescue R1, 300k, cap 20 engaged at 1800 s, 1.35 and 1.65 λ_{ref} (registered)	Simulator: rescued in 20 of 20 seeds at both rates, release at 3490–5830 and 4050–6050 s. Theory: rescued, release at 3830 and 4360 s (±15%), relapse probability 0.002 and 0.025; the 7200 s deadline is missed if the paused sessions complete below 32% and 42% of their fluid rate. Refuted if NOT RESCUED. A release outside ±15% is a miss by the scoring declaration, but the headline table of `notes/PREDICTIONS_r13_theory.md`, frozen at the same tag, also lists it under "falsified if".	registration, tag `r11-rescue-reg`; simulator, tag `r11-rescue-pred`; theory, `r1_rescue_300k.json`	PASS at both rates: rescued 6 of 6, no relapse, no session shed; BURST controls stuck 6 of 6. Release 3980–4550 s (1.35) and 4430–4920 s (1.65). Simulator: every call held; releases inside its band and within 600 s of each seed's exact prediction in 6 of 6. Theory: verdict held (rescued 6 of 6); release within ±15% in 5 of 6, all late; the exception, 1.35 λ_{ref} seed 0 at +18.8%, is a miss by the scoring declaration and a refutation by the headline table's rule. Blind in time to both
Rescue R4, native pool, cap 64 at 2100 s, 4.0 λ_{ref} (registered)	Simulator: rescued in 16 of 20 seeds, seeds 0–2 rescued; the engage time was moved from 2400 s (13 of 20) after that prediction and before any rescue run. Theory: rescued, release at 5040 s (±15%); the 7200 s deadline is missed if the paused sessions complete below 51% of their fluid rate. Release outside ±15%: a miss by the scoring declaration, and the only falsifier listed in the headline table.	as R1; theory, `r4_rescue_native.json`	PASS: rescued 3 of 3, no relapse, no session shed; BURST controls stuck 3 of 3. Release 5980–6140 s. Simulator: calls held; releases inside its band but 800–1750 s after each seed's exact prediction. Theory: verdict held (rescued 3 of 3); release 19–22% late, outside ±15% in 3 of 3: a miss by the scoring declaration, refuted by the headline table's rule. Blind in time to both
SHED, 300k, BURST at 1.35 λ_{ref}, arrivals stopped for τ from 1800 s (registered)	Rule: τ = 600 s stuck and τ = 3600 s good, in 2 of 2 seeds; sessions refused (about 58 and 348) reported against none for the cap. Simulator: a full stop from 1800 s lasting 3600 s rescued 10 of 10, dropping 328–378 sessions; one lasting 3000 s rescued 8 of 10. pred2: the shortest rescuing stop lasts 1795 s (Eq. (1) from RR's 161 sessions at 1800 s; RR alone: 1928 s); τ = 600 s stuck (1.00), τ = 3600 s good (0.998); the rule holds with probability 0.997.	registration, tag `r14-extra-reg`; simulator, tag `r11-rescue-pred`; pred2, `shed.json`	[pending: SHED verdict]
TOOLSLOW, 300k, NOBURST, 1.35 λ_{ref}, $Z + \delta$ in [1200, 2100) s (registered)	Rule: δ = 10 s good and δ = 30 s stuck, each in at least 2 of 3 seeds. pred2: δ = 10 s good with probability 0.71 per seed, δ = 30 s stuck (1.00); the rule holds with probability 0.80 (band 0.63); critical δ 16.8 s (fluid), 16.4 s (stochastic).	registration, tag `r14-extra-reg`; pred2, `toolslow.json`	[pending: TOOLSLOW verdict]
LPM-INF, 300k, BURST, 1.65 λ_{ref}, no 128-request fallback (exploratory)	Simulator: no collapse on seeds 0–2 (token hit fraction 0.08–0.10), including seeds 0 and 1, on which stock LPM collapsed; the frozen rule reads this as the fallback causing that collapse, and the registered rule still calls each run trapped.	tag `r11-lpminf-pred`, `configs/r11_lpminf_predict.yaml`	Consistent with the rule: no collapse on seeds 0, 1 (token hit 0.59, 0.42; stock LPM 0.000 in 4 of 4); seed 2 uninformative (queue $\leq$ 128). No run recovered. Simulator: direction right, level missed

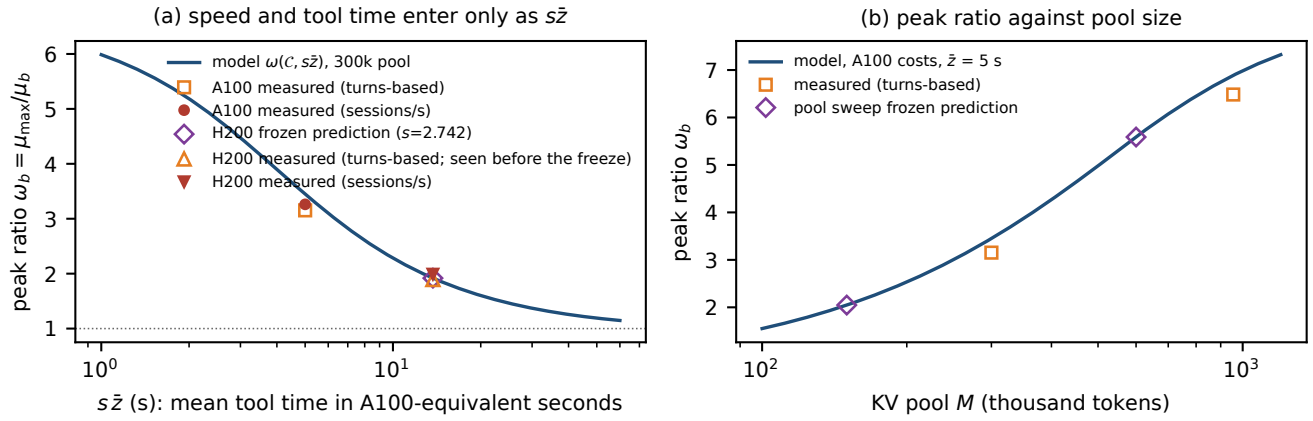


Figure 9: Frozen predictions of the peak ratio $\omega_b = \mu^*/\mu_b$ (fluid model, in model units). (a) Against $s\bar{z}$, the mean tool time in A100-equivalent seconds, for the 300k pool (Proposition 3); squares and dot: the A100, measured (turns-based, sessions/s); diamond: the H200 prediction ($s = 2.742$). (b) Against the pool size, A100 costs, $\bar{z} = 5$ s; squares: measured cells (turns-based); diamonds: the pool-sweep predictions. Triangles: the H200 measured peak ratio against its λ_{ref}, 1.89 turns-based and 1.99 in sessions/s (Table 17), values the first author had seen before the freeze. The pool-sweep outcomes are pending. Table 17 quotes the primary predictions (F1, against λ_{ref}), which are lower than the plotted fluid values by 0.1% (H200) to 8.6% (150k).

sition 2(c), the turns-based λ_{ref} (0.194 sessions/s) is the denominator of the peak ratio, and the 64-to-32-session ratio of all-miss token rates sets one cost variant. The model predicts that the peak ratio falls to 1.92 while the all-miss onset moves out to 41.3 sessions (Fig. 9); the frozen refutation rules are a peak ratio of 2.4 or more, a hit fraction below 0.5 at $n = 32$, or one of 0.5 or more at $n = 48$. Three extra closed runs at $n = 36$, 44 and 52, submitted after the freeze, bracket the predicted crossing at $n = 37.3$, and a 600k-token H200 cell, with the 600k A100 cell of the pool sweep, completes a two-by-two test of Proposition 3. KAIROS shows qualitatively that a slower clock tips a fixed arrival rate into thrashing [101], and VAMP locates KV capacity boundaries on H100 and H200 GPUs [55]; the new parts would be the measurement in units of λ_{miss} and the opposite movement of the onset and the peak ratio. The Claude Code agent that operated the H200 cell (it submitted and monitored that cell's jobs) saw its closed-curve values for $n = 16$–32 (jobs 17166818–22) at about 22:28 CDT on 2026-10-30, before the freeze, and showed them to the first author; the coordinating agent that drafted the paper and dispatched the other agents, and the agents that derived and froze the model, did not see them. These values include the region of the predicted peak ($n \approx 32.5$), so the peak-ratio rule ($\omega_{\text{ref}} \geq 2.4$), the hit fraction at the peak and hit(32) are not blind to the first author. The closed runs at $n = 36$, 44 and 52 were submitted at 02:33 CDT, after the freeze, and are blind in time. Those at $n = 40$, 48, 96 and 300 and the original from-empty runs had ended before the freeze; by the record no one read them until the curve and the bracket were combined at 02:30, but, as for BASIN-1's runs, no file-access record can confirm this, so they are blind by access only. No blind simulator prediction exists for this cell. The edge is scored on the original E0b runs (jobs 17168377–82); four reseeds (17169668–71, at 1.3 and 1.4 λ_{ref}) were added after those runs had been read and are reported separately.

GPU generation: outcome. Section 5.5 reports the measurement and §7 scores it against the frozen model (Table 17); the details they omit follow. In completed sessions the H200 peak is 0.386 sessions/s, 1.65 times the A100's. The unstable 1.4 λ_{ref} run ended with a queue of 50.4, rising by 49 over the last 1500 s. On the nine-point curve the 0.5-hit crossing lies at 38.8. The log-likelihood of the six original E0b outcomes (stable at 1.1, 1.2 and 1.3, unstable at 1.4, 1.55 and 1.75 λ_{ref}), which the scoring rules ask for under both edge criteria, is −1.35 under F2 and −2.82 under the second criterion (a level crossing, edge 1.57 λ_{ref}; its probabilities interpolated linearly between its grid points). No refutation rule fires: hit(32) = 0.74 and hit(48) = 0.22. The model is closest to the tail at $n = 36$ (hit 0.61 against 0.56 fluid; the cost variants give 0.52–0.57, and F1 0.50–0.53); from $n = 40$ on, F1 gives 0.28, 0.08, 0.01 and 0.002 at $n = 40$, 44, 48 and 52, and the measured turns-based μ is 1.12–1.29 times the F1 prediction. The frozen notes expected Assumption MF, which does not concentrate at this pool on the H200, to bias the peak upward; the peak was matched (F1 0.371 sessions/s against 0.367 turns-based), and the miss is in the tail instead (§6.6).

Pools and a second model. The pool sweep repeats Stage C and E0b on the A100 with pools of 150k and 600k tokens, with no fitted parameter; its 150k from-empty edge discriminates between F2 and a second edge criterion, which predict 1.09 and 1.47 λ_{ref}. The Llama-3.1-8B cell keeps the engine, the GPU and the 300k pool. The model's a priori costs for it come from published architecture ratios; the pool part $M/E_S = 27.8$ does not depend on the model's speed; and once the cell's all-

miss rate is measured, every predicted quantity is read from a frozen table indexed by the speed factor.

The basin. Two registered BURST-only tests at 1.65 λ_{ref} on the 300k pool vary the burst size: BASIN-1 with 25 and 100 sessions and BASIN-2 with 40, 50 and 60. The theory predicts whether a run is stuck, with a hit fraction below 0.2 over the evaluation window; it does not model the slope test of the registered trap rule, which is co-primary only for BASIN-1's 100-session cell. The model's critical fresh burst is 53.4 sessions, and its per-seed probabilities of a stuck run are 0.04, 0.14, 0.55 and 1.0 for 25, 40, 50 and 60 or more sessions; the stationary-composition estimate, 21.3 sessions, would have 40 stuck. BASIN-2's frozen rule calls the boundary bracketed if 40 stays good and 60 is stuck, each in at least two of three seeds. The second prediction set (pred2, pushed at 04:20:20 CDT on 2026-10-31; the nine BASIN-2 jobs were submitted at 04:21) conditions these probabilities on each seed's burst, rebuilt with the client's own plan generator. Seed 0's first 40 burst sessions are long and seed 2's short, so the probabilities of a stuck run at 40, 50 and 60 sessions become 0.78, 0.86 and 1.00 for seed 0, 0.36, 1.00 and 1.00 for seed 1, and 0.09, 0.22 and 0.81 for seed 2; the rule then holds with probability 0.66 (0.23–0.88 over the ways of mapping a plan to a burst size), against 0.94 with every burst at its mean composition. The critical burst and the stationary estimate are both defined at n_2, where $\mu(n)$ does not determine them (§6.6), so BASIN-2 tests the heuristic F3, not a consequence of the propositions.

The basin: BASIN-1 outcome. Section 5.4 reports the outcome and §7 scores it. The 100-session runs were stuck (hit fraction 0.000 over $[5700, 8700)$ s) and also trapped by the co-primary slope rule, in 3 of 3 seeds. BASIN-1 does not test the critical burst of 53.4 sessions; BASIN-2 does.

Fresh-seed from-empty runs: outcome. Sections 5.1 and 7 and Appendix A report the outcome: both rates stable in 3 of 3 fresh seeds, so the frozen replication rule is not met, and in realized rates the edge lies between 2.665 and 2.726 λ_{ref}. In realized arrivals the fresh seeds received 2.01, 2.15 and 1.97 λ_{ref} and 2.51, 2.67 and 2.47 λ_{ref}. The second edge criterion in the frozen 300k file did better than F2: its probabilities of instability, 0.03 at 2.0 and 0.37 at 2.5 λ_{ref}, give all six runs stable a log-likelihood of −1.47, against −4.01 under F2, and all three stable at 2.5 λ_{ref} a probability of 0.25. That file (tag `r13-theory-pred`) was pushed before five of the six runs ended but declares no prediction for this test.

Rescue and shedding. Two registered tests engage a cap inside an established trap and release it once the gate has been empty for 300 s: R1 on the 300k pool (cap 20 at 1800 s, at 1.35 and 1.65 λ_{ref}) and R4 with the native pool (cap 64 at 2100 s, at 4.0 λ_{ref}). The simulator and the model both predict rescue. At the fluid rate of the paused sessions, whose next turns miss, the model predicts release at 3830, 4360 and 5040 s (±15%; with F1, 3920, 4480 and 5250 s); it predicts release before the 7200 s deadline only if those sessions complete at no less than 32, 42 and 51% of that rate. The gate pauses sessions only at tool-call boundaries and defers work rather than dropping it. Pausing whole programs at tool boundaries is ThunderAgent's and NVIDIA Dynamo's mechanism [31, 53]; CONCUR cuts its window of active agents multiplicatively when it detects thrashing (hit rate below 0.2 with KV usage above 0.5) to exit that regime, and MISA-T contracts its cap temporarily on a confirmed hit-rate decline, both in closed or bounded populations [18, 45]; KAIROS bounds aggregate context and, when bursty arrivals have already broken the bound, defers new agents and boosts the clock, but reports no recovery from an established trap [101]; inhibiting new traffic until a bistable channel returns to its lower state was proposed by Carleial and Hellman [14], and shedding load is the usual response in production [46]. The new parts are a rescue from an established trap under open arrivals, a measured test that the good state persists after the cap is released, and a rescue condition and release times frozen beforehand; §5.3 reports the outcome. The SHED test measures the alternative: arrivals stop for $\tau = 600$ or 3600 s from 1800 s, refusing about $\lambda\tau = 58$ or 348 sessions, against none for the cap.

Rescue: outcome. Sections 5.3 and 7 report the outcome and its scoring; the per-run values follow (Table 16). The simulator's release times on the exact plans were 3960, 3470 and 3940 s at 1.35 λ_{ref} and 4900, 4440 and 4280 s at 1.65, against the engine's 4550, 3980 and 4400 s and 4920, 4430 and 4570 s, and 4250, 5180 and 4410 s for R4, against 6000, 5980 and 6140 s; all nine releases lie inside its 20-seed bands. Against the model's 3830 and 4360 s, the R1 releases were +18.8, +3.9 and +14.9% and +12.8, +1.6 and +4.8% late; against 5040 s, the R4 releases were +19.0, +18.7 and +21.8% late (against the F1 value, 5250 s: +14.3, +13.9 and +17.0%). The engine's first phase in R4 was slower than the fluid rate but far from collapsed. The smoke run ended before the model's push, but the model made no prediction for it, and by their record the agents that derived the model had no Nova access and opened no rescue output before the push.

Causes. TOOLSLOW lengthens every tool call that starts in $[1200, 2100)$ s by $\delta = 10$ or 30 s in the NOBURST arm at 1.35 λ_{ref}, a trigger that involves neither a burst nor a lowered clock. Its registered rule expects the 10 s slowdown to leave the engine good and the 30 s slowdown to leave it stuck, each in at least two of three seeds. In the frozen model (pred2) the slowdown acts through capacity and population: for δ above 13.7 s the slowed curve's peak lies below $\lambda = 0.0966$ sessions/s, so it has no good state (at $\delta = 30$ s the peak is the all-miss level $\mu_b = 0.0718$ sessions/s); at $\delta = 30$ s the population reaches 46.9 sessions by 2100 s, above $n_2 = 32.2$, while at $\delta = 10$ s it reaches 21.5, near the slowed curve's good state (21.8 sessions, against 8.5 without the slowdown). A pass would show that a temporary increase in tool latency, with no burst, can trigger the trap; it would not separate the longer gap from the capacity loss and the larger population that come with it. LPM-INF disables stock LPM's 128-request fallback (§5.6)

with a job-local hook that raises the limit to 10^9 and changes nothing else; the BURST arm at 1.65 λ_{ref} is repeated on seeds 0–2 with byte-identical arrival plans. The simulator predicted (pushed at 21:02:46 CDT on 2026-10-30, 65 minutes before the jobs were submitted) that LPM then avoids the all-miss collapse on the two seeds on which stock LPM collapsed, in a partial-hit state that the registered rule still calls trapped.

LPM-INF: outcome (exploratory). Appendix G gives the outcome in full: on the two informative seeds LPM without its fallback did not collapse, no run recovered by the registered rule, and the simulator got the direction and missed the level.

I Simulator studies

I.1 Robustness of the paired test

The real test fixes one workload, one burst size and one horizon; we varied each in the calibrated simulator (mixed batching). Each workload gets its own window [1, *hi*] in multiples of its own λ_{miss} (*hi*: the largest multiple at which all 10 from-empty runs stay stable) and is tested one third and two thirds of the way across it; a point passes if each arm meets its criterion in at least 7 of 10 seeds, counted separately per arm (the registered real-engine rule instead counts seeds in which all three arms succeed). The repository holds the per-run summaries of these simulations, from which every number below is computed, but not the 100 s window series behind them.

Tool-time laws. With exponential tool times of mean 1, 2 and 5 s, a lognormal law (mean 5 s, coefficient of variation 2) and a bimodal law (80% at 1 s, 20% at 20 s), λ_{miss} stays within 0.0718–0.0730 sessions/s: with the token profile fixed, it does not depend on the tool-time law. Shorter tool times raise the peak ($\mu^*/\lambda_{\text{miss}}$ is 5.25, 4.57 and 2.97 for means of 1, 2 and 5 s) and widen the window (*hi* = 3.0, 2.5, 2.0); a heavy-tailed or a bimodal law of about the same mean (5 s and 4.8 s) leaves the edge at 2.0. The test passes at all 10 points (Fig. 10): NOBURST and CAPPED succeed in 10 of 10 seeds everywhere, and BURST is trapped in 7 to 10 of 10.

Tool times from real agent traces. We replaced the tool-time law by empirical tables from SyFI TraceLab, real sessions of two commercial coding agents [105], and from mini-SWE-agent v2.0.0 [87, 96] on the Verified subset of SWE-bench [22, 52], whose tool times are far more dispersed than ours (coefficients of variation 4.3–4.7 against 1). λ_{miss} moved by −0.8% to +1.3%. The peak moved from 20 in the exponential control to 32 (TraceLab per turn), 40 (idle gap) and 24 (mini-SWE-agent), and the cap to 32, 40 and 20; at every point of these three laws the CAPPED arm recovered in 10 of 10 seeds. Of the six points of the three laws, five passed; the failure, TraceLab per turn at 1.40 λ_{miss}, is the slope rule: 6 of 10 BURST runs met it, all 10 were stuck below a hit fraction of 0.2, and on 20 fresh seeds (post hoc) 17 met it. Appendix G

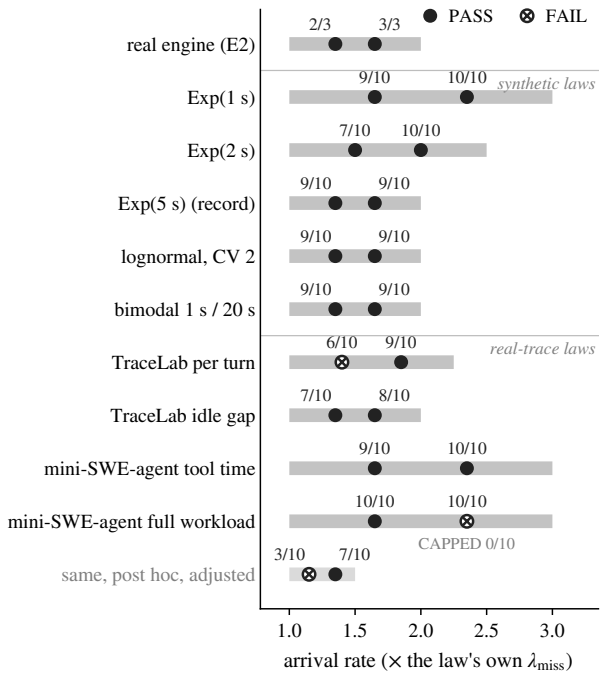


Figure 10: The paired test across tool-time laws (simulator). Gray bar: from-empty window [1, *hi*] in multiples of each law's λ_{miss}. Markers: test points, filled = pass, crossed = fail (each arm counted separately, in ≥ 7 of 10 seeds); number = BURST runs trapped of 10. Top row: the real engine (3 seeds; registered rule, all three arms in at least 2 of the same seeds). Bottom row: labelled post-hoc rerun of the mini-SWE-agent full workload. The two mini-SWE-agent full-workload rows use different units: the declared row multiples of its declared λ_{miss} (0.0120 sessions/s), the post-hoc row multiples of the long-window λ_{miss} (0.0141 sessions/s), in which the declared 1.65, 2.35 and 3.0 are 1.40, 1.99 and 2.55.

gives the trace statistics and a secondary workload with mini-SWE-agent's turn and token counts.

Burst size and horizon. At 1.35 and 1.65 λ_{miss}, a burst of 25 sessions never traps (0 of 10), 50 is a coin flip (5 and 6 of 10 trapped), and from 100 up every run is stuck below a hit fraction of 0.2. In runs extended to 30,000 s after the burst, no trapped BURST run ever returned to a 100 s window above a hit fraction of 0.5, all 20 ended with 468 to 1065 waiting requests, and no NOBURST run ever exceeded 50. The first-principles model's critical fresh burst, 53.4 sessions at 1.65 λ_{ref} (§6), lies between the simulator's coin-flip size and the size from which every run is stuck, a sweep that the agents who derived the model knew when they froze it (Appendix C). On the engine, the registered test BASIN-1 found 25 sessions good and 100 stuck, and trapped by the registered rule, in 3 of 3 seeds each, as both the simulator and the model predicted, which places the boundary between 25 and 100 sessions; BASIN-2, with 40, 50 and 60 sessions, is

pending (§5.4).

The slope rule is the fragile part. Over the 100 BURST runs of the tool-time test the registered rule labels 90 trapped and 10 indeterminate, while all 100 are stuck at zero hits with an all-miss token rate. At 1.35 λ_{miss} with 20 fresh seeds, 3 are indeterminate (Wilson interval [0.05, 0.36]; median slope $0.77(\lambda - \lambda_{\text{miss}})$ against a threshold of 0.5), so the real engine's one in three there is not surprising. The threshold presumes that a trapped engine completes sessions near λ_{miss}, but the registered SGLang BURST runs at 300k completed 1.13–1.35 λ_{ref} in the evaluation window (§3.3); the indeterminate one, seed 1, received 1.27 λ_{ref} there instead of the nominal 1.35 and completed 1.28, so its backlog stayed flat. Over 31,200 s runs the rule labels only 10 to 12 of 20 stuck BURST runs trapped, depending on the window. Post hoc, a rule on the hit fraction alone would have called all six real E2 BURST runs trapped, and all six exploratory vLLM runs, of which the registered rule calls four trapped (§5.2); the vLLM runs reuse the SGLang runs' seeds, and both of their indeterminate runs are again seed 1, so the two engines' labels are not independent. We report the registered rule. Its token criterion has the opposite weakness at high multiples of λ_{miss}, where a trapped engine prefills more than the all-miss token rate: 1.106–1.124 times it at 4.0 λ_{ref} with the native pool, against a tolerance of 1.15 (§5.2).

The cap must follow the measured curve. The right cap, 20 for the workload of record at 300k, moves to 32 or 40 with the TraceLab tool-time laws (it stays at 20 with mini-SWE-agent's) and to 64 with the native pool, where it recovered in 6 of 6 registered runs. With mini-SWE-agent's full workload the peak falls to 12, and a cap of 20 would sit past the peak; a cap c above the new peak leaves $(\mu(c), \mu^*)$ open in the quasi-static model (§3.4).

I.2 Published mechanisms in the earlier simulator

In the fluid model of §3.3 the recovery threshold from deep congestion is at most μ_∞, so a mechanism can raise it only as far as it raises μ_∞. We compare mechanisms by $\mu(300)/\lambda_{\text{miss}}$ as a proxy for μ_∞ (Fig. 11) in the earlier, uncalibrated simulator. Each rule was compared, by reading the code and not running it, with the released code where it exists (PEEK; the Continuum preview, which omits the paper's estimator) or with the paper (CONCUR, for which we found no code); where a difference was small and unambiguous we added a projection of the released rule beside the historical row, and the differences that remain are stated below. That simulator's assumed costs are about 1.5 times the fitted ones (λ_{miss} 0.0474 against 0.0722 sessions/s), and its curve peaks at 4.07 times its own λ_{miss}, against about 3.0 in the calibrated simulator and 3.26 λ_{ref} on the engine. Its absolute rates therefore do not carry over. As in Stage C, all sessions start at $t = 0$, but each run is measured over 25,000 s after a 5000 s warmup (static caps: 30,000 s), about four mean session lifetimes; FCFS reads 1.04 λ_{miss} over the first half of the window and 1.01 over the second, so the synchronized-start bias of §8 is a few percent here. These are results about the rules in a workload with private prefixes and exponential 5 s tools, not benchmarks of the named systems.

Orderings. FCFS gives 1.024 ± 0.020 with hit fraction 0.000 and the GPU always busy; with the 1.5% standard error of λ_{miss} included the interval is 1.02 ± 0.04, consistent with Proposition 6, which covers this simulator's request-level FCFS. Pure LPM (without SGLang's 128-request fallback) gives 1.174 ± 0.023 and recovers from 300 cold sessions over 150,000 s up to 1.1 λ_{miss}, not at 1.25. PEEK [95] protects prefixes needed by queued requests and adds a cache-aware scheduler. Our earlier rows implemented only the protection under FCFS order. As a soft ranking, which is how PEEK's eviction hook protects (finite priorities), it gives the same throughput as FCFS seed for seed: queued requests' KV is protected but still evicted before FCFS admits them (2,831–3,128 such evictions per run), so no request is admitted with its KV, consistent with Proposition 6. A hard pin, a counterfactual PEEK does not apply, gives 0.919. The released scheduler serves warm requests first; projected onto single-session prefixes with its eviction score, it gives 1.176 ± 0.021, equal to pure LPM within 0.001 ± 0.014, and recovers from none of 5 bursts at 2 λ_{miss}. Here PEEK is an LPM-class ordering, not FCFS; we did not evaluate it on shared-prefix traffic, for which it is designed.

TTL pins. Continuum keeps a program's KV after a non-final turn for a time-to-live chosen by a cost model, which may be zero [58]; its released preview omits that estimator and would rarely pin our 5 s-mean tool calls. Our rows are fixed, unconditional TTLs of $\tau = 5$, 10 and 20 s on every non-final turn, with Continuum's waiting order (pinned programs first). Under the preview's release rule they give 2.584 ± 0.035, 3.505 ± 0.064 and 4.376 ± 0.048; under the rule we read in the paper's deadlock-prevention text, $\tau = 10$ and 20 s give 1.16–1.17; with plain FCFS order the pins fall below λ_{miss}, consistent with Proposition 6. These are results about configured pins, not about Continuum (Appendix G).

Adaptive caps. CONCUR adds 2 to a window on active agents when KV usage is below 0.2 and halves it when usage exceeds 0.5 and the hit rate is below 0.2, pausing and resuming agents at generation and tool boundaries [18]; we found no released code. Our session-gate surrogate of this equation gives 3.009 ± 0.081 and recovers from 5 of 5 bursts, under a usage signal that counts only live sessions; if finished sessions' KV, still cached and evictable, counts as usage, 12 of 18 cells stall at the surrogate's minimum window (Appendix G). So 3.0 is the surrogate's rate, not deployed CONCUR's.

Static caps. By Proposition 1 a cap c holds a large population at $\mu(c)$: 4.04 ± 0.25 at $c = 20$, 2.39 ± 0.35 at $c = 28 \approx N_{\text{lb}}$

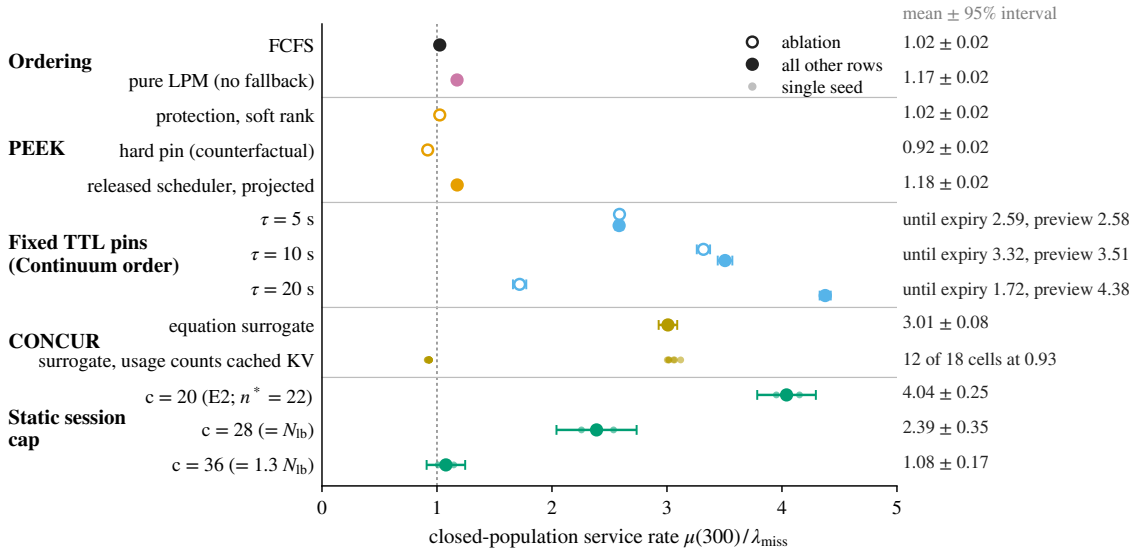


Figure 11: $\mu(300)/\lambda_{\text{miss}}$, our proxy for the congested service rate μ_∞, per mechanism in the earlier, uncalibrated simulator: mean over 10 seeds with 95% t intervals, in units of that simulator's λ_{miss} (0.0474 sessions/s; not comparable with Figs. 2–6); the intervals omit the 1.5% standard error of λ_{miss}. Hollow: ablations (PEEK's queued-prefix protection under FCFS order, as a soft rank and as a hard-pin counterfactual PEEK does not apply; fixed pins held until expiry even with the GPU idle). Filled: all other rows; for PEEK, a projection of its released scheduler; for the pins, fixed TTLs under the preview's release rule (values: held until expiry, preview). LPM is pure LPM, without SGLang's 128-request fallback. CONCUR: a surrogate of its control equation; small dots: 18 cells in which finished sessions' KV counts as usage. Static caps $c = 20$ (E2's cap; this simulator's n^* is 22), 28, 36: $\mu(c)$ of the closed FCFS curve (3 other seeds, not paired with the other rows).

and 1.08 ± 0.17 at $c = 36$ (3 seeds), against a peak of 4.07 at $n = 22$. The cliff makes the choice fragile: a cap about a quarter above N_{lb} is back near λ_{miss}.

Admission by memory demand. ThunderAgent periodically compares the programs' KV demand with the capacity, counting a program in a tool call at a weight that decays with its tool time (2^{-t} in its experiments); when demand exceeds capacity it pauses whole programs, those in tool calls and those with the shortest contexts first, and restores them when demand falls, and both of its watermarks are set to 1 in its evaluation [53]. NVIDIA Dynamo ships a port that pauses programs at tool boundaries, by default when utilization reaches 0.95 of the pool, down to 0.80, and resumes them at 0.85 or below [31]. KAIROS bounds the aggregate context of admitted agents at 0.95 of capacity and admits pending agents again below 0.9 [101], and AgentKV, from its abstract, admits by a predicted capacity ratio [3]. These are implemented forms of a session bound, sized by memory demand rather than count and applied to prevent the trap (KAIROS also defers new agents and boosts the clock once bursty arrivals have broken its bound, but reports no recovery from an established trap); we have not evaluated them. Their two thresholds form a controller's dead band, which keeps it from toggling; the hysteresis of Corollary 1 is a property of the system, whose good state persisted on the engine after a cap was removed (§5.3). Proposition 1 concerns a FIFO count cap whose sessions keep their slots, so it does not predict a gate that bounds current demand and pauses admitted programs, and the static-cap rates above are not a prediction for any of these systems.

Why the ranking comes out this way. Under request-level FCFS, with or without protection that keeps FCFS order, μ_∞ cannot exceed λ_{miss} when (A3) holds (Proposition 6); we conjecture (Remark in Appendix E) that pure resident-first orderings raise it through tool calls shorter than the congested lifetime of idle KV, and for pure LPM the simulator agrees. A static cap c holds the engine at $\mu(c)$ (Proposition 1), so at most at the peak, 4.07 at $n = 22$ (3 seeds). Pins at $\tau = 20$ s gave 4.38, above every static cap we ran and above the peak, though within the peak's 3-seed interval (±0.36). In this simulator they also serve pinned programs first and keep a returning session's KV for at least τ, and the set of sessions they bound depends on memory, not on a count; we have not separated these effects. On the real engine, stock LPM recovered from a 200-session burst at 1.35 λ_{ref} in 4 of 6 runs (§5.6); that finite-burst result does not compare directly with these deep-congestion rates, and in deep congestion stock LPM stops resorting its queue, so, on our reading of its source, Proposition 6 bounds its μ_∞ by λ_{miss} when (A3) holds.